

FernUniversität in Hagen
Fakultät für Mathematik und Informatik
LG Kooperative Systeme

UDP Transport Options

Implementierung und Analyse von RFC 9868 in Rust

Masterarbeit
im Studiengang Master Informatik

vorgelegt von
Alan Bernstein

Matrikelnummer: 2068834

Erstprüfer: Prof. Dr. Christian Icking

Zweitprüferin: Dr. Lihong Ma

Hagen, 15.09.2026

Inhaltsverzeichnis

Abbildungsverzeichnis	v
Tabellenverzeichnis	vi
Abkürzungsverzeichnis	vii
1 Einleitung	1
1.1 Motivation	1
1.2 Problemstellung	2
1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum	2
1.2.2 Unidirektionalität	2
1.2.3 Herausforderungen	2
1.3 Zielsetzung	3
1.4 Abgrenzung und Umfang	4
1.5 Aufbau der Arbeit	4
1.6 KI-Einsatz	5
2 Grundlagen	6
2.1 IPv4	6
2.2 Das User Datagram Protocol (UDP)	10
2.2.1 Protokollstruktur	11
2.2.2 Eigenschaften	12
2.3 Erweiterungsmuster im Vergleich	13
2.4 Die Surplus Area	14
2.4.1 Lage und Kompatibilität	14
2.4.2 Aufbau und Ausrichtung	15
2.4.3 OCS und Fehlerbehandlung	17
2.4.4 TLV-Format und Empfangsentscheidung	18
2.4.5 Entwurfsprinzipien und Optionsklassen	20
2.5 Betriebssystem und Netzpfad	22
3 Methodik und KI-gestützte Entwicklung	27
3.1 Anlass des KI-Einsatzes und veröffentlichte Fälle	27
3.2 Forschungsdesign und KI-gestützte Arbeitsweise	28
3.3 Bekannte Schwächen der Sprachmodelle	34
3.4 Testarchitektur für FF1	34

3.4.1	Prüfmittel und ihre Reichweite	35
3.4.2	Prüfkette bis zur Freigabe	36
3.5	Reproduzierbarkeit und Grenzen	37
4	Analyse und Anforderungsmodell	38
4.1	Anwendung der Extraktionsmethode	38
4.2	Anforderungskatalog und FF1-Maßstab	39
4.3	FF2-Pfad- und Auswertungsmodell	40
4.4	Technologieauswahl	42
4.4.1	Programmiersprache	42
4.4.2	Zugang zur Surplus Area	44
4.4.3	Mitschnitt und Auswertung	45
4.4.4	Fuzzing und formale Gegenprobe	45
5	Entwurf und Implementierung	47
5.1	Vorgehen der Implementierung	47
5.2	Gesamtarchitektur	48
5.3	Sendepfad	50
5.4	Empfangspfad	53
5.5	Optionsverarbeitung	58
5.6	Testarchitektur	61
5.7	Betriebs- und Sicherheitsgrenzen	64
5.8	Bewusste Grenzen	66
6	Evaluation und Diskussion	67
6.1	Test- und Messkonzept	67
6.2	Beobachtbarkeit und Middlebox-Verträglichkeit auf realen Pfaden . .	71
6.2.1	Kontrollierte Stufen: Loopback, eigene Topologien, Tunnel . .	71
6.2.2	Öffentliche Pfade in Europa	72
6.2.3	Interkontinentale Pfade	78
6.2.4	Mechanismenklassen und Folgen	79
6.3	Soll-Ist-Abgleich mit RFC 9868	81
6.4	Reflexion der KI-gestützten Arbeitsweise	87
6.5	Diskussion und Grenzen der Aussagekraft	88
7	Fazit und Ausblick	90
7.1	Beantwortung der Forschungsfragen	90
7.2	Ausblick	91
7.3	Schlusswort	91
A	Konformitätsmatrix	93

Literaturverzeichnis	98
Eidesstattliche Erklärung	103

Abbildungsverzeichnis

2.1	Der IPv4-Header nach RFC 791 [5]. Hervorgehoben sind die beiden Längenangaben, auf denen die Bestimmung der Surplus Area beruht; gestrichelte Felder sind nur vorhanden, wenn IHL über dem Minimalwert 5 liegt.	7
2.2	Schematische Zerlegung eines UDP-Datagramms in drei IPv4-Fragmente	9
2.3	Der UDP-Header nach RFC 768 [1]	11
2.4	Der Pseudo-Header der UDP-Prüfsumme für IPv4 [1]	12
2.5	IP Transport Payload und Surplus Area eines Datagramms mit Total Length 60	14
2.6	Anatomie eines Datagramms mit UDP Options	15
2.7	Paritätsentscheid für das Pad-Byte vor dem OCS	16
2.8	Empfangsentscheidung für die Surplus Area im Standardfall	20
2.9	Die zwei Sendepfade durch den Linux-Kernel [4, 17]	23
2.10	Der Empfangspfad durch den Linux-Kernel [4, 17]	24
2.11	Messmodell des Datagrammwegs	26
3.1	Aufbau der Untersuchung und Zuordnung der Forschungsfragen . . .	29
3.2	Von den Normquellen zum geprüften Anforderungsbestand	30
3.3	Kernzyklus eines Implementierungsschritts bis Schritt 18 (Ausnahmen gestrichelt)	32
3.4	Prüfprozess im Sollzustand	36
4.1	Verteilung der markierten normativen Absätze über die Abschnitte .	38
5.1	Komponenten der Bibliothek und ihre Abhängigkeiten	49
5.2	Entstehung des Beispieldatagramms in vier Schritten	50
5.3	Empfangspipeline der Bibliothek mit ihren Ergebnisarten	55
5.4	Entwicklungs- und Nachweisartefakte des Implementierungs-Repositorys	61
6.1	Ankunfts-TTL je Richtung über die Kampagnen-Mitschnitte der europäischen Messpfade	70
6.2	Kontrollierte Messumgebungen der Pfadstufen 1 bis 3	72
6.3	Europäisches Messdreieck mit zwei Endpunkt- und zwei beobachteten Transit-Netzen	73
6.4	Reroute-Fenster im Szenario S-53 und Aufteilung des Quellportraums auf die ECMP-Zweige	74

6.5	Prüfsummen-Gate-Kreuzzellen je Messrichtung	75
6.6	Zugangspfad über den Mobilfunk-Hotspot in den Betriebsarten NAT und Bridge	76
6.7	Interkontinentale Messendpunkte mit Belegstufe und Rundlaufzeiten .	78
6.8	Schicksal der Surplus-Klasse auf den Hinwegen nach Asien-Pazifik je Quelle	79
6.9	Kantenmodell eines Messpfads mit den beobachteten Attributionsin- tervallen der drei Mechanismenklassen	80
6.10	Drei beobachtete Schicksale von Beispieldatagrammen auf realen Pfaden	80

Tabellenverzeichnis

2.1	Muster für nachträgliche Protokollerweiterungen	13
2.2	OCS-Rechnung einer 6-Byte-Surplus-Area bei geradem Startoffset . .	18
3.1	Operationalisierung der Forschungsfragen	29
3.2	Rollen bei Erzeugung, Prüfung und Freigabe	32
3.3	Schwächen der Sprachmodelle und methodische Antworten	34
4.1	Auszug aus dem RFC-Arbeitsindex des Implementierungs-Repositorys	39
4.2	Implementierungsumfang	41
4.3	Messinfrastruktur	42
4.4	Vergleich der Sprachkandidaten	43
4.5	Entscheidungsmatrix für den Zugang zur Surplus Area unter Linux .	45
5.1	Reichweite der Prüfmittel	63
6.1	Messläufe der Arbeit nach Pfad, Stand und Einstufung	68
6.2	Summen der Konformitätsmatrix	82
6.3	Fragmentierung und Soft-State: Norm, Umsetzung und Messbeleg . .	86
7.1	Synthese der Forschungsfragen im geprüften Umfang	90
A.1	Prüfung der 67 Matrixzeilen gegen RFC 9868 und den Implementie- rungsstand	93

Abkürzungsverzeichnis

APC Additional Payload Checksum.

AUTH Authentication Option.

DTLS Datagram Transport Layer Security.

EOL End of List.

FRAG Fragmentation Option.

MDS Maximum Datagram Size.

MRDS Maximum Reassembled Datagram Size.

NOP No Operation.

OCS Option Checksum.

PCAP Packet Capture.

PMTU Path Maximum Transmission Unit.

TIME Timestamp Option.

1. Einleitung

1.1 Motivation

Die ursprüngliche Spezifikation des User Datagram Protocol (UDP), RFC 768 aus dem Jahr 1980, umfasst 663 Wörter [1]. Die Erweiterung RFC 9868 wurde im Oktober 2025 veröffentlicht und führt unter dem Titel *Transport Options for UDP* erstmals einen allgemeinen Optionsmechanismus für UDP ein [2]. Zwischen beiden Dokumenten liegen 45 Jahre; die Klartextfassung des RFC 9868 ist mehr als 27-mal so umfangreich wie die des RFC 768.

UDP besitzt einen acht Byte langen Header aus vier 16-Bit-Feldern: Quellport, Zielport, Länge und Prüfsumme [1]. RFC 9868 beschreibt UDP als weit verbreitet und stellt fest, dass der ursprüngliche Header keinen Raum für Optionen bietet [2, Sec. 4].

RFC 9868 greift Funktionen auf, die UDP ursprünglich der Anwendung oder einer darüberliegenden Schicht überließ: Transportfragmentierung, eine zusätzliche Nutzdatenprüfsumme sowie Optionen für Größen- und Zeitinformationen [2, Sec. 5, 11.3 und 11.8]. Für ihre Bereitstellung kommen grundsätzlich drei Wege in Betracht:

- Erweiterung innerhalb der Nutzdaten: Jede Anwendung definiert ein eigenes Rahmenformat in der UDP-Nutzlast.
- Ein neues Transportprotokoll: Alle gewünschten Mechanismen können so von Beginn an vorgesehen werden.
- Erweiterung des bestehenden Formats: UDP selbst erhält einen Optionsmechanismus, der für Bestandssysteme möglichst unsichtbar bleibt.

RFC 9868 wählt den dritten Weg (die Muster hinter diesen Wegen vergleicht Kapitel 2) und nutzt dafür die sogenannte *Surplus Area*: den möglichen Bereich zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms. Da das UDP-Length-Feld die Länge des UDP-Datagramms angibt und diese kleiner sein kann als die *IP Transport Payload*, also alles, was im IP-Datagramm auf den IP-Header folgt, entsteht ein bisher ungenutzter Bereich für Erweiterungen (Abschnitt 2.4). Auf dieser Grundlage definiert RFC 9868 ein erweiterbares Rahmenwerk für Transportoptionen.

Damit stellt sich die Frage, die diese Arbeit als Leitfrage begleitet:

Wie lässt sich UDP um Transportoptionen erweitern, ohne seine grundlegenden Eigenschaften aufzugeben?

Die Arbeit untersucht diese Frage anhand einer Referenzimplementierung und auf ausgewählten realen Netzpfeilen.

1.2 Problemstellung

Die Implementierung von UDP-Optionen gemäß RFC 9868 wirft drei Problemfelder auf, die diese Arbeit behandelt: die Beschaffenheit der Surplus Area selbst, die bewusste Unidirektionalität des Protokolls sowie die praktischen Implementierungsherausforderungen vom Zugriff aus dem Benutzerraum bis zum Verhalten von Zwischensystemen. Die folgenden Abschnitte behandeln sie in dieser Reihenfolge.

1.2.1 Die Surplus Area als ungenutzter Erweiterungsraum

RFC 9868 lässt den UDP-Header unverändert, verlangt aber die getrennte Auswertung von UDP- und IP-Länge. Den genauen Aufbau der daraus entstehenden Surplus Area behandelt Abschnitt 2.4.

1.2.2 Unidirektionalität

RFC 9868 stellt explizit klar: „*UDP is unidirectional; UDP Options do not change that fact.*“ [2, Sec. 6].

Für Mechanismen, die Bidirektionalität voraussetzen, fordert RFC 9868 eine getrennte Spezifikation [2, Sec. 6]. RFC 9869 definiert einen solchen Einsatz von REQ und RES für die Pfad-MTU-Ermittlung [3, Sec. 3]. Diese Vorgabe schließt einen Datenaustausch in beiden Richtungen nicht aus. Sie bedeutet, dass die UDP-Optionsschicht keine Antwort selbstständig erzeugt. Nicht das Protokoll, sondern die Anwendung oder eine in ihrem Auftrag arbeitende Bibliothek löst eine Antwort aus.

1.2.3 Herausforderungen

Die zentrale Herausforderung bei der Implementierung von UDP-Optionen liegt im Zugriff auf die Surplus Area. Die in RFC 9868 getesteten Systeme Linux, macOS und Windows Cygwin lieferten nur die durch das UDP-Length-Feld begrenzten Nutzdaten an die Anwendung und verwarfen die Surplus Area still. Ein ebenfalls dokumentiertes eingebettetes Gerät wich davon ab und reichte das gesamte IP-Datagramm weiter [2, Sec. 18]. Den Weg eines UDP-Pakets durch den Linux-Kernel bis zu dieser Socket-Schnittstelle dokumentieren Stephan und Wüstrich [4].

Eine weitere Herausforderung stellt die Kompatibilität mit Zwischensystemen dar. RFC 9868 berichtet Interoperabilitätstests, in denen Pakete mit Surplus Area Geräte zur Netzadressübersetzung (NAT, *Network Address Translation*) passierten, nennt aber auch ein Intrusion Detection System, das die Diskrepanz zwischen UDP-Length und IP-Length in der Standardeinstellung als Angriff meldet [2, Sec. 18]. Ob reale Netzpfade solche Pakete tatsächlich durchlassen, ist eine empirische Frage dieser Arbeit (FF2).

1.3 Zielsetzung

Die Problemstellung verdichtet sich zu zwei getrennten Hürden: Ein UDP-Options-Paket muss normgerecht erzeugt und empfangen werden, und seine Surplus Area muss die untersuchten Netzpfade unverändert überstehen.

Eine normgerechte Implementierung sagt nichts darüber aus, ob Zwischensysteme Datagramme mit Surplus Area unverändert passieren lassen. Ebenso nützt ein durchlässiger Netzpfad wenig, wenn Erzeugung und Auswertung der Optionen fehlerhaft sind. Aus dieser Trennung ergeben sich zwei Forschungsfragen:

- **FF1:** Welche Anforderungen des RFC 9868 lassen sich unter Linux und IPv4 mit einer Raw-Socket-Implementierung im Benutzerraum vollständig, teilweise oder nicht erfüllen?
- **FF2:** In welchem Umfang bleibt die *Surplus Area* auf den untersuchten realen Netzpfeiden bis zur Gegenstelle unverändert erhalten?

Das Ziel dieser Arbeit ist es, beide Fragen zu beantworten. Dazu entsteht zunächst eine an RFC 9868 ausgerichtete und systematisch auf Konformität geprüfte Bibliothek für UDP-Transportoptionen in Rust. Die auf Linux und IPv4 begrenzte Implementierung arbeitet im Benutzerraum mit Raw Sockets. Sie umfasst Parser und Serialisierer für UDP-Optionen nach dem TLV-Schema (*Type-Length-Value*), die Validierung der Optionsprüfsumme (*Option Checksum*, OCS) gemäß Abschnitt 9 des RFC 9868 sowie die verpflichtend zu unterstützenden Optionen EOL, NOP, APC, FRAG, MDS, MRDS, REQ und RES [2, Sec. 10].

Die Verifikation erfolgt mehrstufig: Komponententests prüfen die einzelnen Bestandteile, Integrationstests den Loopback-Pfad, und für reine Regeln wird ergänzend eine Lean-Spezifikation gepflegt. Lean dient als formale Gegenprobe für Prüfsummenarithmetik sowie Längen- und Anordnungsregeln; das Verhalten von Raw Sockets, Betriebssystemen und Zwischensystemen bleibt Gegenstand empirischer Prüfungen. Dieses Prüfprogramm schafft die Grundlage für die Beantwortung von FF1.

Für FF2 vergleicht die Evaluation gewöhnliche UDP-Datagramme mit gleich großen Datagrammen, die eine Surplus Area enthalten, auf ausgewählten realen Netzpfaden. Sie unterscheidet zwischen unveränderter Zustellung, Entfernung der Surplus Area und Paketverlust. Die Schlussfolgerungen bleiben auf die beobachteten Pfade begrenzt.

1.4 Abgrenzung und Umfang

Diese Arbeit setzt einen bewussten Rahmen. Sie beschränkt sich auf eine Umsetzung des RFC 9868 im Linux-Benutzerraum für IPv4. Gegenstand sind das TLV-Rahmenwerk, die OCS sowie die verpflichtend zu unterstützenden Optionen. Die Beschränkung auf den Benutzerraum ist eine vorab gesetzte Rahmenbedingung der Aufgabenstellung. Abschnitt 4.4 begründet die dafür gewählte Umsetzung mit Raw Sockets gegenüber kernelnahen Verfahren.

Außerhalb des Umfangs liegt der REQ/RES-Anwendungsfall zur Pfad-MTU-Ermittlung, den RFC 9869 definiert [3]. Die Implementierung unterstützt nur die in RFC 9868 festgelegten Formate von REQ und RES. Ebenfalls nicht implementiert wird die TIME-Option. RFC 9868 beschreibt zwar ihr Format und den Anwendungszweck, die Schätzung der Paketumlaufzeit (*Round-Trip Time*, RTT), legt aber kein nutzendes Protokoll fest. Die Kennungen für AUTH, UCMP und UENC sind in RFC 9868 lediglich reserviert und werden daher ebenfalls nicht implementiert (AUTH [2, Sec. 11.9], UCMP [2, Sec. 12.1], UENC [2, Sec. 12.2]). Kernel-Module werden nicht entwickelt.

IPv6 wird weder implementiert noch auf Konformität geprüft. Die hier untersuchten Mechanismen des RFC 9868 (*Surplus Area*, OCS, TLV-Optionen und FRAG) lassen sich vollständig über IPv4 zeigen. Die Implementierung und ihre Konformitätsprüfung beziehen sich daher auf IPv4; in den Pfadmessungen erscheint IPv6 nur als Kontrollgröße der Messumgebung (Abschnitt 4.3).

1.5 Aufbau der Arbeit

Nach dieser Einleitung folgen sechs Kapitel, die drei Blöcke bilden:

- **Grundlagen und Methodik:** Kapitel 2 führt die technischen Grundlagen ein: das Internet Protocol, UDP, Erweiterungsmuster von Transportprotokollen sowie RFC 9868 mit der *Surplus Area* und seinen Entwurfsprinzipien. Kapitel 3 legt die Forschungsmethodik offen, bevor mit ihr gearbeitet wird: das Forschungsdesign, die RFC-Auswertungsmethode und der Einsatz von KI.

- **Anwendungsteil:** Kapitel 4 überführt RFC 9868 mit dieser Methode in ein prüfbares Anforderungsmodell und begründet die Technologieauswahl. Kapitel 5 führt Entwurf und Implementierung der Bibliothek komponentenweise zusammen. Kapitel 6 berichtet die Ergebnisse zu beiden Forschungsfragen sowie eine deskriptive Auswertung der KI-gestützten Arbeitsweise.
- **Schluss:** Kapitel 7 wiederholt die Forschungsfragen, beantwortet sie einzeln und zieht die Bilanz zur Leitfrage.

Diese lineare Kapitelfolge ist eine rekonstruktive Darstellungsstruktur und kein Abbild des tatsächlichen Arbeitsprozesses. Die Umsetzung verlief iterativ und agenten-gestützt, sodass Entwurf, Implementierung und Test einander fortlaufend bedingten.

1.6 KI-Einsatz

Diese Arbeit ist unter durchgehendem Einsatz künstlicher Intelligenz (KI) entstanden. Der Einsatz betrifft Text und Software. Auf der Textseite erstellten KI-Werkzeuge Entwürfe, überarbeiteten Formulierungen, übernahmen die LaTeX-Formatierung, setzten die Abbildungen als TikZ-Grafiken um und bedienten die LaTeX-Werkzeugkette. Zudem prüften sie den Text wiederholt gegen den Primärtext des RFC 9868. Auf der Softwareseite unterstützten sie Planung, Implementierung, Tests und Verifikation der Bibliothek. Der Autor entschied über die Übernahme der Ergebnisse und verantwortet sie.

Zum Einsatz kommen zwei KI-Werkzeuge (harness), die ein Sprachmodell mit Dateizugriff und Kommandoausführung verbinden: Claude Code von Anthropic und Codex von OpenAI. Ihre Rollenverteilung, die bekannten Schwächen dieser Arbeitsweise und die Prüfkette vor der Übernahme eines Ergebnisses behandelt Kapitel 3.

2. Grundlagen

IPv4 und UDP sind die beteiligten Protokolle (Abschnitt 2.1, Abschnitt 2.2), gefolgt von den Erweiterungsmustern (Abschnitt 2.3), der Surplus Area (Abschnitt 2.4) sowie Betriebssystem und Netzpfad (Abschnitt 2.5).

2.1 IPv4

Das Internet Protocol (IP) vermittelt Datagramme zwischen Endsystemen und bildet die Vermittlungsschicht für UDP [5]. Für RFC 9868 ist es nötig, weil erst **UDP Length** und das durch den IP-Header bestimmte Datagrammende Lage und Größe der Surplus Area festlegen [2, Sec. 7].

Diese Arbeit beschränkt sich auf IPv4. Bei IPv4 bestimmen **IHL** und **Total Length** die IP-Nutzlast; bei IPv6 ergeben sich ihre Grenzen aus dem Basis-Header und der Kette der Extension Header [2, Sec. 7][6]. Auch Randwerte unterscheiden sich: Endpunkte müssen mindestens zwei Fragmente zerlegen und wieder zusammensetzen können, von denen jedes in die Ethernet-MTU von 1 500 Byte passt [2, Sec. 11.4]. Die zugehörige lokale MRDS-size beträgt mindestens 2 926 Byte bei IPv4 und 2 886 Byte bei IPv6 [2, Sec. 11.6]; bei IPv6-Jumbograms mit **UDP Length** null sind UDP Options nicht nutzbar [2, Sec. 22]. Die Grundstruktur des Mechanismus bleibt gleich, Implementierung und Evaluation betrachten jedoch nur IPv4 (Abschnitt 1.4).

Der Abschnitt behandelt deshalb nur die für RFC 9868 und die Messung nötigen IPv4-Felder: **IHL**, **Total Length**, **Protocol**, Adressen, Header-Prüfsumme, Fragmentierungsfelder und **Time to Live**.

Den Ausgangspunkt bildet der Header; seinen Aufbau zeigt Abbildung 2.1 [5].

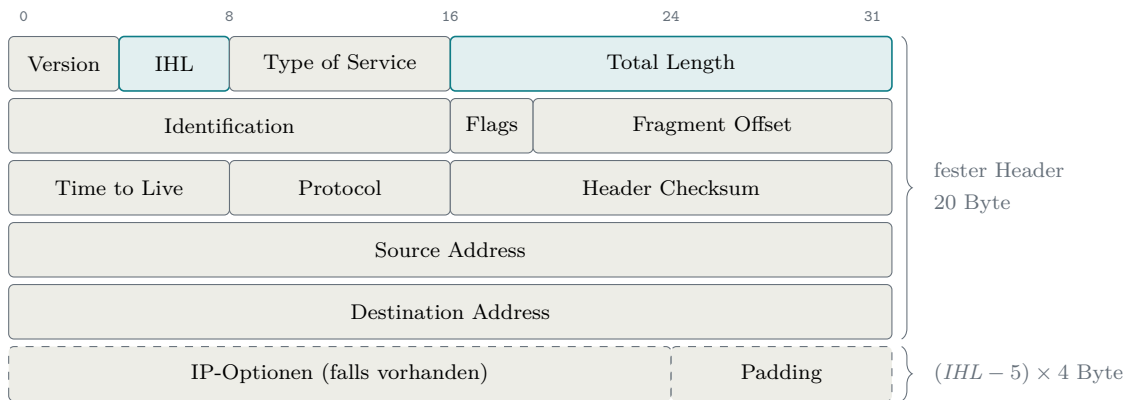


Abbildung 2.1: Der IPv4-Header nach RFC 791 [5]. Hervorgehoben sind die beiden Längenangaben, auf denen die Bestimmung der Surplus Area beruht; gestrichelte Felder sind nur vorhanden, wenn IHL über dem Minimalwert 5 liegt.

Im Zentrum stehen die beiden Längenangaben der ersten Zeile. **Total Length** gibt die Gesamtlänge des Datagramms in Byte an, Header und Nutzlast zusammen; als 16-Bit-Feld erlaubt es höchstens 65 535 Byte.

Die zweite Längenangabe, die **IHL** (Internet Header Length), beziffert die Länge des Headers. Sie zählt dabei nicht in Byte, sondern in 32-Bit-Wörtern zu je 4 Byte. Der Grund liegt in der Breite des Feldes: Mit seinen 4 Bit kann es höchstens den Wert 15 darstellen. In Byte gezählt würde das nicht einmal für den festen Headeranteil von 20 Byte reichen; in 32-Bit-Wörtern gezählt beschreibt derselbe Wertebereich dagegen Header bis 60 Byte [5]. Die Headerlänge ergibt sich daher als

$$\text{Headerlänge} = IHL \times 4 \text{ Byte.} \quad (2.1)$$

Der Minimalwert 5 entspricht dem optionslosen 20-Byte-Header, der Maximalwert 15 einem Header von 60 Byte. Eine IPv4-Headerlänge ist damit stets ein Vielfaches von 4; diese unscheinbare Eigenschaft kehrt bei der Ausrichtung der UDP Options wieder.

Zwischen festem Header und Nutzlast können zudem IP-Optionen liegen. Sie stellen Steuerfunktionen für Sonderfälle bereit, die die gewöhnliche Kommunikation nicht braucht, und dienen als zusätzliche Felder zur Erweiterung der IP-Header [5]: **Record Route** etwa sammelt die Adressen der durchlaufenen Router ein, **Internet Timestamp** deren Zeitstempel. In der Praxis werden solche Optionen kaum genutzt. Wie ein Router optionstragende Pakete behandelt, hängt von seiner Architektur und Konfiguration ab: Er kann sie mit Leitungsgeschwindigkeit weiterleiten oder filtern oder die Optionen im allgemeinen Prozessor oder in Spezialhardware verarbeiten; vor allem die Verarbeitung im allgemeinen Prozessor birgt bei großen Paketmengen ein Überlastrisiko. Zugleich ist das Verwerfen optionstragender Pakete verbreitete

Praxis [7, Sec. 1 und 3.1]. Zudem fasst der 40-Byte-Optionsraum bei **Record Route** höchstens neun Adressen und damit viele reale Pfade nicht [5]. Abschließend gilt, dass erst die IHL und keine Konstante den Beginn der Nutzlast festlegt.

Aus den beiden Längenangaben **Total Length** und IHL ergibt sich die Größe, auf die sich RFC 9868 durchgängig bezieht: Die Nutzlast eines IPv4-Datagramms beginnt nach $IHL \times 4$ Byte und endet bei **Total Length**. Für ein UDP-Datagramm fordert RFC 9868 deshalb [2, Sec. 7]¹

$$UDP\ Length \leq Total\ Length - Headerlänge. \quad (2.2)$$

Die rechte Seite der Ungleichung ist die Länge dieser Nutzlast, also der Platz hinter dem IP-Header. Die linke Seite ist die Länge, die das UDP-Datagramm für sich beansprucht. Die Bedingung verlangt damit, dass das UDP-Datagramm vollständig in die Nutzlast des IP-Datagramms passt; UDP darf nie mehr Platz beanspruchen, als das IP-Datagramm hergibt. Bei Gleichheit füllt das UDP-Datagramm die Nutzlast exakt aus; so arbeiten klassische Sender. Bleibt **UDP Length** dagegen zurück, ist die Differenz genau der Raum, den Abschnitt 2.4 als Surplus Area einführt.

Von den übrigen Feldern braucht die Arbeit nur wenige. **Protocol** benennt das Transportprotokoll der Nutzlast; der Wert 17 steht für UDP. Die **Header Checksum** sichert ausschließlich den Header, nicht die Nutzlast; deren Fehlererkennung bleibt der UDP-Prüfsumme überlassen (Abschnitt 2.2.1). Quell- und Zieladresse geben Absender und Empfänger des Datagramms an. Sie werden später noch einmal wichtig: Die UDP-Prüfsumme bezieht beide Adressen über einen sogenannten Pseudo-Header in ihre Berechnung ein. **Version** und **Type of Service** berühren den Optionsmechanismus nicht und werden nicht weiter vertieft.

Auch **Time to Live** spielt für die Optionen keine Rolle, kehrt in der Evaluation aber als Messgröße wieder. Jede weiterleitende Instanz verringert den Wert um mindestens eins und verwirft das Datagramm bei null [5]. Die Differenz zwischen gesendetem und empfangenem Wert entspricht deshalb nur unter der Annahme eines Dekrements von genau eins je Weiterleitung der Zahl der durchlaufenen IPv4-Router; Kapitel 6 nutzt sie unter dieser Annahme, um die Stabilität der Messpfade zu prüfen. Die zweite Zeile des Headers schließlich (**Identification, Flags, Fragment Offset**) gehört vollständig zur Fragmentierung, der sich der Rest dieses Abschnitts widmet.

Wie groß ein Datagramm praktisch werden kann, entscheidet allerdings nicht das 16-Bit-Längenfeld, sondern der Übertragungsweg. Jede Technik der Sicherungsschicht

¹ In Ausnahmefällen liegen zwischen IPv4- und UDP-Header noch Shim-Header, etwa bei IPsec oder IPComp; das Feld **Protocol** zeigt dann nicht UDP an, und die Obergrenze verringert sich zusätzlich um die Gesamtlänge dieser Shim-Header [2, Sec. 7]. Diese Arbeit betrachtet solche Ketten nicht weiter.

begrenzt die Nutzlast eines einzelnen Rahmens; diese Obergrenze heißt Maximum Transmission Unit (MTU) und liegt für Ethernet üblicherweise bei 1500 Byte. Streng genommen ist die MTU also eine Eigenschaft der Sicherungsschicht. Ihre Folgen trägt jedoch die Vermittlungsschicht, denn ein Datagramm durchquert auf dem Weg zum Ziel mehrere Netze mit möglicherweise unterschiedlichen MTUs, und IP muss festlegen, was mit einem Datagramm geschieht, das nicht in den nächsten Rahmen passt [5].

IPv4 beantwortet das mit Fragmentierung. Ist ein Datagramm größer als die MTU und das Flag **DF** (Don't Fragment) nicht gesetzt, zerlegt die IP-Schicht es in mehrere Fragmente. Das übernimmt nicht nur der Absender: Bei IPv4 darf auch jeder Router unterwegs ein Datagramm zerlegen, das nicht in sein nächstes Netz passt [5]. Jedes Fragment erhält einen eigenen IP-Header, übernimmt die **Identification** des Originals, trägt in **Fragment Offset** seine Position in Einheiten von 8 Byte und setzt das Flag **MF** (More Fragments), das nur beim letzten Fragment fehlt. Der Empfänger sammelt zusammengehörige Fragmente anhand von Quelladresse, Zieladresse, Protokoll und **Identification** und setzt das Datagramm vor der Übergabe an die Transportschicht wieder zusammen [5]. Ist **DF** dagegen gesetzt, verwirft ein Router das zu große Datagramm und meldet dies dem Absender per ICMP; auf dieser Rückmeldung baut die Path MTU Discovery auf, mit der ein Sender die kleinste MTU seines Pfades ermittelt [8, Sec. 3.2].

Für diese Arbeit ist an der Fragmentierung vor allem eine Konsequenz bedeutsam: Der UDP-Header gehört aus Sicht von IP zur Nutzlast und landet deshalb vollständig im ersten Fragment. Was das für die übrigen Fragmente bedeutet, zeigt Abbildung 2.2 am Beispiel eines in drei Teile zerlegten Datagramms.

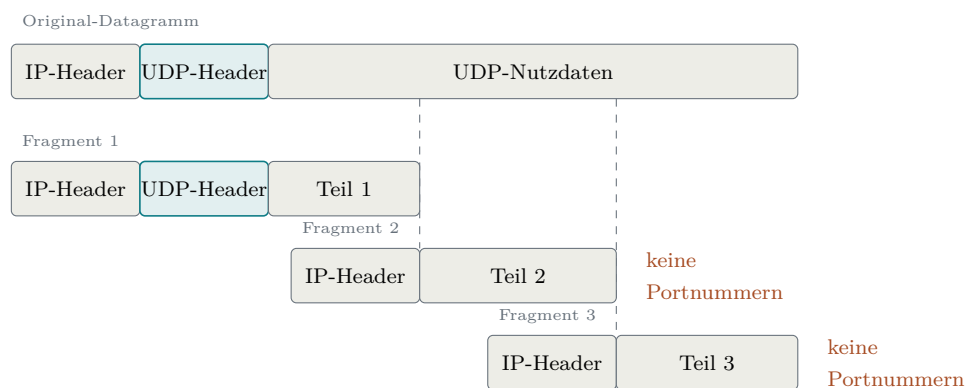


Abbildung 2.2: Schematische Zerlegung eines UDP-Datagramms in drei IPv4-Fragmente

Aus Abbildung 2.2 geht hervor, dass nur das erste Fragment den UDP-Header enthält: Alle Folgefragmente tragen keine Portnummern. Zwischensysteme wie NATs und Firewalls, die Pakete anhand vollständiger Header zuordnen oder prüfen, können

solche Fragmente nicht einordnen und verwerfen sie teils pauschal; zudem macht der Verlust eines einzelnen Fragments das gesamte Datagramm unbrauchbar. RFC 8085 wertet beides als grundsätzliches Problem der IP-Fragmentierung, das jedes Transportprotokoll trifft, und rät UDP-Anwendungen deshalb, IP-Fragmentierung zu vermeiden [8, Sec. 3.2]. Dass der Rat gerade an UDP geht, hat einen Grund: Während TCP seinen Datenstrom selbst in passende Segmente zerlegt und der IP-Fragmentierung so meist entgeht [9], erzeugt UDP je Nachricht genau ein Datagramm (Abschnitt 2.2.2); ein zu großes Datagramm kann bislang nur die IP-Schicht zerteilen. Genau diese Lücke schließt die FRAG-Option von RFC 9868: Sie verlagert die Fragmentierung auf die Transportschicht und wiederholt die Portnummern in jedem Fragment [2, Sec. 11.4]; den Verlust einzelner Fragmente behebt allerdings auch FRAG nicht. Darauf kommt Kapitel 6 zurück.

Der folgende Abschnitt wendet sich dem Protokoll zu, das RFC 9868 tatsächlich erweitert: UDP.

2.2 Das User Datagram Protocol (UDP)

Das User Datagram Protocol (UDP) ist ein verbindungsloses, nachrichtenorientiertes Transportprotokoll. Es arbeitet ohne vorherigen Handshake und bietet selbst keine Garantie für Zustellung, Reihenfolge oder Duplikaterkennung [1]. Zuverlässigkeit und Staukontrolle muss ein nutzendes Protokoll bei Bedarf oberhalb von UDP bereitstellen [8, Sec. 3.1 bis 3.3]. UDP verwendet die IP-Protokollnummer 17 und ist als STD 6 standardisiert. Bemerkenswert ist, dass RFC 768 seit seiner Veröffentlichung im August 1980 über 45 Jahre hinweg keine formelle Aktualisierung erfahren hat. Erst RFC 9868 trägt den Header-Eintrag `Updates: 768` und erweitert damit den ursprünglichen UDP-Standard erstmalig direkt [2].

Historisch geht UDP auf die Aufspaltung des ursprünglichen *Transmission Control Program* zurück, das Cerf, Dalal und Sunshine Ende 1974 noch als monolithische Einheit aus Netzwerk- und Transportfunktionen beschrieben [10]. Mit deren Trennung in IP und TCP entstand Raum für eine schlanke, transaktionsorientierte Alternative ohne den Overhead von TCP [1]. Der UDP-Kern blieb über die folgenden Jahrzehnte unverändert, und Begleitdokumente wie die Einsatzrichtlinien von RFC 8085 klärten Randfragen, ohne den Standard formell zu ändern [8]. Die Motivation, ihn nun doch direkt zu erweitern, benennt RFC 9868 selbst: UDP zählt zu den meistgenutzten Protokollen ohne Raum für Header-Optionen, und weil inzwischen viele Protokolle UDP direkt nutzen, wäre eine Zwischenschicht oberhalb von UDP für Bestandsanwendungen kaum noch auszurollen [2, Sec. 4 und 5].

2.2.1 Protokollstruktur

Ein UDP-Datagramm besteht aus einem Header mit fester Länge von 8 Byte und den darauf folgenden Nutzdaten; den Aufbau des Headers zeigt Abbildung 2.3 [1].

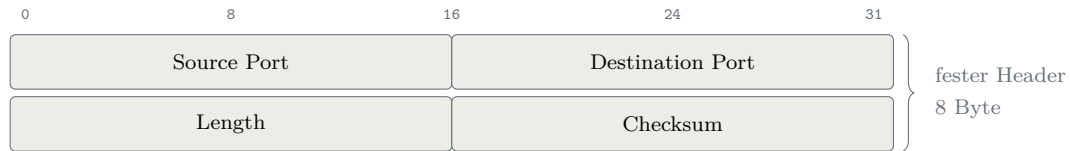


Abbildung 2.3: Der UDP-Header nach RFC 768 [1]

Aus Abbildung 2.3 geht hervor, dass der gesamte Header aus genau vier Feldern zu je 16 Bit besteht; unter ihnen das Längenfeld, für das Abschnitt 2.1 mit Gleichung 2.2 bereits die Obergrenze aufgestellt hat. Die vier Felder im Einzelnen:

- **Source Port:** Die Portnummer des sendenden Prozesses. Dieses Feld ist optional; wird es nicht benötigt, wird es auf null gesetzt [1]. In der Praxis nutzen Anwendungen den *Source Port* zur Identifikation des Absenders, damit der Empfänger Antworten an den korrekten Port zurücksenden kann.
- **Destination Port:** Die Portnummer des Zielprozesses auf dem empfangenden Host. Zusammen mit der IP-Zieladresse ermöglicht dieses Feld die *Demultiplexierung* eingehender Datagramme an die korrekte Anwendung.
- **Length:** Die Gesamtlänge des UDP-Datagramms in Byte, bestehend aus Header und Nutzdaten. Der Minimalwert beträgt 8, was einem Datagramm ohne Nutzdaten entspricht [1]. Da das Feld 16 Bit breit ist, ergibt sich eine theoretische Maximalgröße von 65 535 Byte.
- **Checksum:** Eine Prüfsumme zur Fehlererkennung über einen Pseudo-Header, den UDP-Header und die Nutzdaten; ihr Verfahren und ihre Sonderwerte erläutert der folgende Absatz.

Prüfsumme. Die Prüfsumme dient der Fehlererkennung: Mit ihr erkennt der Empfänger, ob einzelne Bits des Datagramms auf dem Weg verändert wurden. Diese Prüfung auf der Transportschicht ist nötig, weil nicht jede Teilstrecke eines Pfades eigene Fehlererkennung bietet und Bits auch im Speicher eines Routers kippen können [11]. Für IPv4 darf der Sender die Prüfsumme durch den Feldwert null ungenutzt lassen; bei IPv6 ist sie grundsätzlich verpflichtend, abgesehen von eigens geregelten Tunnel-Ausnahmen [8, Sec. 3.4 und 3.4.1]. Für die Berechnung summiert der Sender Pseudo-Header, UDP-Header und Nutzdaten als 16-Bit-Wörter im Einerkomplement auf (ein Übertrag wird am niederwertigen Ende wieder hinzugezählt); das invertierte Ergebnis bildet den Feldwert [1, 12]. Der Empfänger summiert dieselben Wörter

einschließlich des Prüfsummenfelds und erwartet lauter Einsen, also `0xFFFF`; jedes andere Ergebnis zeigt einen Übertragungsfehler an [12].

Dabei hilft eine Eigenheit des Einerkomplements: Es kennt zwei Darstellungen der Null, `0x0000` und `0xFFFF`. Diese Doppelrolle nutzt UDP für die Sonderwerte des Feldes: eine übertragene `0x0000` bedeutet, dass der Sender keine berechnet hat. Ergibt die Berechnung selbst den Wert null, wird stattdessen `0xFFFF` übertragen. Dadurch bleibt im Einerkomplement die Zahl Null erhalten, und der Sonderwert bleibt eindeutig [1]. Dabei ist zu beachten, dass UDP Fehler nur erkennen und nicht beheben kann; ein beschädigtes Datagramm wird typischerweise verworfen [11].

Pseudo-Header. Die UDP-Prüfsumme schützt nicht nur die Transportschichtdaten, sondern bezieht über einen *Pseudo-Header* auch Informationen der Vermittlungsschicht ein. Dieser Pseudo-Header ist kein Teil des Datagramms und steht an keiner Stelle im Paket: Sender und Empfänger bauen ihn jeweils nur für die Berechnung im Speicher auf und stellen ihn dem UDP-Header dabei gedanklich voran [1]. Dieses Konzept stellt sicher, dass ein Datagramm, das durch einen IP-Fehler an den falschen Host oder das falsche Protokoll zugestellt wird, erkannt und verworfen werden kann. Seinen Aufbau zeigt Abbildung 2.4 [1].

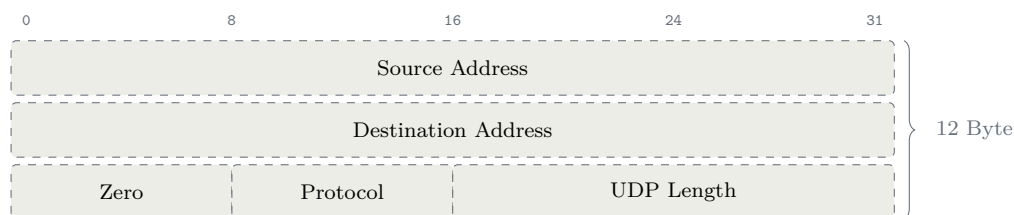


Abbildung 2.4: Der Pseudo-Header der UDP-Prüfsumme für IPv4 [1]

Der Pseudo-Header besteht aus beiden IP-Adressen, einem Nullbyte, **Protocol** und der wiederholten **UDP Length**. Sender und Empfänger bilden diese nicht übertragenen zwölf Byte aus den jeweils vorliegenden Werten [1]. Ohne Umschreibung sind die Werte an beiden Enden gleich; eine NAT kann Adressen oder Ports samt UDP-Prüfsumme anpassen [2, Sec. 11.3]. Das Nullbyte ergänzt das 8-Bit-Protokollfeld zu einem 16-Bit-Wort, wie ein Nulloktett Nutzdaten ungerader Länge für die Rechnung auffüllt [1, 12]. Ein veränderliches Feld wie **Time to Live** wäre ungeeignet, weil Sender und Empfänger dann verschiedene Summen bildeten (Abschnitt 2.1).

2.2.2 Eigenschaften

UDP ist nachrichtenorientiert: Jede Sendeoperation erzeugt ein Datagramm, das der Empfänger als Einheit entgegennimmt. TCP stellt dagegen einen Bytestrom

bereit, segmentiert ihn selbst und überlässt der Anwendung die Kennzeichnung von Nachrichtengrenzen [9, Sec. 2.2 und 3.7]. Klassisches UDP hält zudem keine Sequenznummern, Fenstergrößen oder Zeitgeber je Kommunikationsbeziehung vor. Nur die Reassemblierung von FRAG-Datagrammen bildet in RFC 9868 eine begrenzte Ausnahme [2, Sec. 6].

Da kein Verbindungsaufbau nötig ist, kann die erste Nachricht sofort gesendet werden [8, Sec. 1]. UDP garantiert weder Zustellung noch Reihenfolge und wiederholt verlorene Datagramme nicht [1]. Später eintreffende Datagramme werden auf UDP-Ebene deshalb nicht wegen eines Verlusts zurückgehalten; dort entsteht kein *Head-of-Line-Blocking* wie bei einem gesicherten Bytestrom. Benötigte Zuverlässigkeit oder Reihenfolge muss die Anwendung selbst herstellen [8, Sec. 3.3].

Diese Eigenschaften eignen sich für latenzempfindliche oder verlusttolerante Anwendungen. Dem festen UDP-Header fehlt jedoch ein eigener Erweiterungsraum. RFC 9868 nutzt dafür die Surplus Area, ohne den Header zu verändern (Abschnitt 2.4).

2.3 Erweiterungsmuster im Vergleich

Ein Protokoll ohne freie Bits oder Versionsfeld lässt sich nur erweitern, wenn die Grenze zwischen Kopf, Nutzdaten und Zusatzraum anderweitig erkennbar ist. Tabelle 2.1 vergleicht die drei hier relevanten Muster.

Tabelle 2.1: Muster für nachträgliche Protokollerweiterungen

Muster	Lage und Kennzeichnung	Voraussetzung	Verhalten alter Empfänger	Grenze
TCP- und IPv4-Headeroptionen	zwischen festem Header und Nutzdaten; Offset im Header	das ursprüngliche Format besitzt ein Offsetfeld	Bestandteil des bereits definierten Headerformats; auf UDP nicht übertragbar	bei TCP höchstens 40 Byte [9]
Teredo-Trailer in den UDP-Nutzdaten	hinter dem inneren IPv6-Paket; dessen Länge markiert das Ende	Nutzdaten besitzen eine eigene, bekannte Grenze	im Teredo-Protokoll auswertbar; für beliebige UDP-Altempfänger In-Band-Daten	innerhalb der UDP-Nutzdaten [13, 14]
UDP Options in der Surplus Area	hinter UDP Length , aber vor dem IP-Datagrammende	redundante Längenangaben von UDP und IP	gewöhnliche Empfänger stoppen an UDP Length ; dokumentierte Ausnahmen bleiben	bis zum IP-Datagrammende [2, Sec. 4 und 18]

Das erste Muster kennen TCP und IPv4 (Abschnitt 2.1); beide teilen dabei auch das Füllmuster aus den Ein-Byte-Codes NOP (No Operation) und EOL (End of Option

List) [5, 9]. UDP besitzt dagegen vier feste 16-Bit-Felder und kein Offset- oder Ankündigungsfeld. Eine Headererweiterung wäre daher nicht abwärtskompatibel. Eine In-Band-Konvention scheitert bei beliebigen Bestandsanwendungen, weil sie die Zusatzfelder als Nutzdaten lesen [2, Sec. 4]. RFC 9868 nutzt stattdessen, dass UDP seine Länge wiederholt: Endet UDP **Length** vor der IP-Nutzlast, bildet der Rest die Surplus Area. Dieser Mechanismus bleibt mit Teredo-Trailern kompatibel. Der historische Grund für das UDP-Längenfeld ist ungeklärt; RFC 9868 nennt vorgeschlagene Erklärungen, die nicht zu früheren UDP-Fassungen und zur zeitgleichen Multiplex-Konstruktion passen, und lässt die Ursache offen [2, Sec. 4]. Für bestehendes UDP ist damit nur der Surplus-Trailer allgemein einsetzbar. Seinen Aufbau behandelt der folgende Abschnitt.

2.4 Die Surplus Area

2.4.1 Lage und Kompatibilität

UDP **Length** umfasst nur den UDP-Header und die UDP-Nutzdaten. Die Längenangaben des IP-Headers können einen größeren Transportbereich ausweisen (Abschnitt 2.1). RFC 9868 nennt die gesamte Nutzlast hinter dem IP-Header *IP Transport Payload* und den Teil zwischen dem Ende der UDP-Nutzdaten und dem Ende des IP-Datagramms *Surplus Area*; dort liegen die UDP Options. Die Lage beider Bereiche zeigt Abbildung 2.5 an einem Beispieldatagramm mit **Total Length** 60, **IHL** 5 und UDP **Length** 32 [2, Sec. 7]:

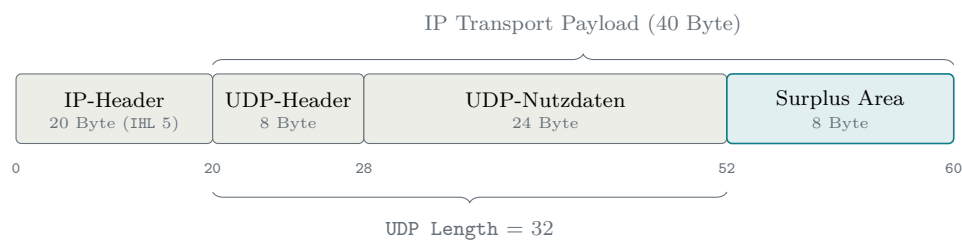


Abbildung 2.5: IP Transport Payload und Surplus Area eines Datagramms mit **Total Length** 60

Abbildung 2.5 zeigt drei Grenzen ab Byte 0 des IP-Datagramms: Die IP Transport Payload beginnt nach dem IP-Header, UDP **Length** deckt von dort nur UDP-Header und Nutzdaten ab, und die Surplus Area reicht bis zum IP-Datagrammende. Im Beispiel bleiben von $60 - 20 = 40$ Byte IP Transport Payload nach UDP **Length** = 32 genau 8 Byte. Allgemein gilt

$$\text{Surplus-Länge} = \text{Total Length} - \text{IP-Headerlänge} - \text{UDP Length}. \quad (2.3)$$

Die Obergrenze aus Gleichung 2.2 verhindert negative Werte; bei Überschreitung ist das Datagramm ungültig und wird still verworfen [2, Sec. 10]. Ohne Optionen ist die Differenz null. RFC 768 verlangt diese Gleichheit jedoch nicht; RFC 9868 nutzt die seit 1980 bestehende Freiheit [2, Sec. 7].

Ein Altempfänger verarbeitet gewöhnlich nur die durch `UDP Length` begrenzten Nutzdaten und überliest den Rest. RFC 9868 dokumentiert neben diesem Verhalten getesteter Standardsysteme auch Abweichungen: Ein eingebettetes Gerät reichte das ganze IP-Datagramm weiter, ein Erkennungssystem meldete die Differenz als Angriff [2, Sec. 18]. Die Erweiterung ist daher für Endsysteme typischerweise, nicht ausnahmslos, abwärtskompatibel; reale Pfade untersucht Kapitel 6. Für IP bleibt die Surplus Area gewöhnliche Transportnutzlast innerhalb von `Total Length`; neue IP-Felder oder eine geänderte IPv4-Verarbeitung entstehen nicht [2, Sec. 16].

Die Surplus Area darf an jedem gültigen Byte-Offset, auch einem ungeraden, beginnen. Das Ausrichtungsproblem wird in ihr gelöst (Abschnitt 2.4.2). `UDP Length` bleibt eine Länge und markiert zugleich den *trailer options offset* [2, Sec. 7]. Damit erhält ein unverändertes Feld eine zusätzliche Bedeutung [2, Sec. 21].

2.4.2 Aufbau und Ausrichtung

Trägt die Surplus Area Optionen, ist sie vollständig strukturiert; ungedeutete Restbytes gibt es dann nicht. Sie beginnt mit der *Option Checksum* (OCS), einem 2 Byte großen Prüfsummenfeld an der ersten 2-Byte-Grenze der Area, gemessen am Beginn des IP-Datagramms. Beginnt die Surplus Area an einem ungeraden Offset, stellt genau ein Nullbyte vor dem OCS diese Ausrichtung her (das *Pad*). Hinter dem OCS folgt die Optionsliste im TLV-Format (Type-Length-Value, Abschnitt 2.4.4); bei gewöhnlichen Datagrammen läuft sie bis zum Ende der Surplus Area oder endet vorzeitig mit EOL, worauf nur noch Nullbytes folgen [2, Sec. 8]. Nur beim UDP-Fragment endet sie früher, denn dort folgt hinter den Optionen noch der Fragmentdatenbereich (Abschnitt 2.4.5). Diese Anatomie zeigt Abbildung 2.6 an einem Datagramm mit 5 Byte Nutzdaten und einer einzelnen Option:

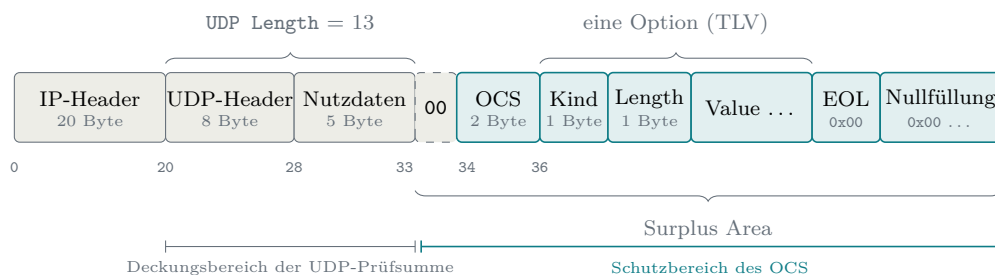


Abbildung 2.6: Anatomie eines Datagramms mit UDP Options

Aus Abbildung 2.6 geht hervor, wie die Bausteine ineinandergreifen: `UDP Length` = $8 + 5 = 13$ lässt die Surplus Area bei Byte $20 + 13 = 33$ beginnen. Dieser Offset ist ungerade, also rückt ein Pad-Byte das OCS auf die 2-Byte-Grenze bei Byte 34, und die erste Option beginnt bei Byte 36. Die unterste Ebene der Abbildung zeigt zugleich, dass sich die beiden Prüfbereiche im Datagramm lückenlos ergänzen: Die UDP-Prüfsumme endet an `UDP Length` (hinzu kommt nur ihr Pseudo-Header, Abschnitt 2.2.1), das OCS schützt genau den Teil, den die UDP-Prüfsumme nie sieht [2, Sec. 8].

Für diese Stelle formuliert RFC 9868 verbindliche Regeln [2, Sec. 8]: Der Sender darf Optionen an jedem gültigen `UDP-Length`-Offset beginnen lassen, und Ausrichtungsbytes vor dem OCS müssen null sein. Findet der Empfänger dort einen anderen Wert, greift zum ersten Mal eine Grundregel, die alle weiteren Prüfungen dieses Abschnitts teilen: Im Standardfall kostet ein Fehler in der Surplus Area nur die Optionen. Der Empfänger verwirft die Optionsliste still und stellt die Nutzdaten trotzdem zu, wie es auch ein Altempfänger täte [2, Sec. 8 und 9]. Standardfall heißt dabei: Das Datagramm selbst ist gültig, denn eine falsche `UDP Length` oder eine fehlgeschlagene UDP-Prüfsumme verwirft es komplett, noch bevor Optionen betrachtet werden, während eine zulässig ungenutzte Prüfsumme (Abschnitt 2.2.1) kein Fehler ist [2, Sec. 10 und 14]; die Anwendung hat kein strengeres Verhalten angefordert; und es ist kein UDP-Fragment, denn die Sonderwege von FRAG und UNSAFE behandelt Abschnitt 2.4.5. Die 2-Byte-Ausrichtung selbst begründet der RFC pragmatisch: Das OCS wird, wie der nächste Unterabschnitt zeigt, über 16-Bit-Wörter berechnet, und die Ausrichtung erspart dieser Rechnung das Vertauschen von Bytes [2, Sec. 8].

Ob ein Pad-Byte nötig ist, entscheidet allein die Parität des Startoffsets; es gibt nur zwei Fälle, kein Pad oder genau eines. Den Unterschied zeigt Abbildung 2.7 an zwei sonst gleichen Datagrammen, deren Nutzdaten sich um ein einziges Byte unterscheiden:

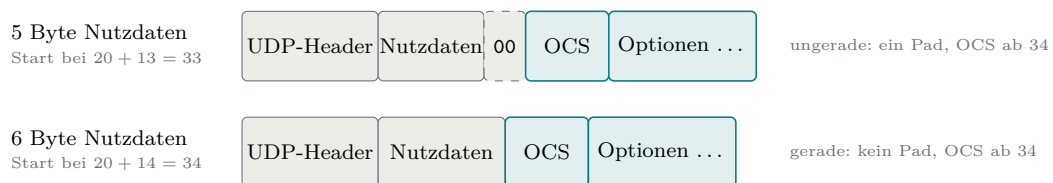


Abbildung 2.7: Paritätsentscheid für das Pad-Byte vor dem OCS

Aus Abbildung 2.7 geht hervor, dass bereits ein einziges Nutzdatenbyte über das Pad entscheidet: Bei einem IP-Header ohne Optionen liegt der Startoffset bei $20 + \text{UDP Length}$, sodass allein die Parität von `UDP Length` den Ausschlag gibt. An der Größe der Surplus Area hängt zugleich, ob sie überhaupt Optionen trägt: Nötig sind mindestens 2 Byte bei geradem und 3 Byte bei ungeradem Start, damit

das ausgerichtete OCS vollständig hineinpasst. Ein kleinerer Überschuss enthält schlicht keine Optionen und ist kein Fehlerfall; passt genau das OCS hinein und sonst nichts, ist die Optionsliste leer und das Datagramm ebenfalls gültig [2, Sec. 7 und 8]. Nutzdaten braucht es dagegen keine: Auch ein Datagramm mit `UDP Length` 8, also leerem Nutzdatenteil, kann Optionen tragen [2, Sec. 8]; davon macht später die FRAG-Option Gebrauch, deren Fragmente ihre Daten vollständig in der Surplus Area transportieren.

2.4.3 OCS und Fehlerbehandlung

Das OCS selbst ist eine gewöhnliche Internet-Prüfsumme mit derselben Einerkomplementarithmetik, die Abschnitt 2.2.1 für die UDP-Prüfsumme beschreibt [12]. Zwei Eigenheiten unterscheiden die beiden. Erstens schützt das OCS genau den Bereich, den die UDP-Prüfsumme ausspart: Die Summe läuft ab dem ausgerichteten OCS-Feld über den Rest der Surplus Area, wobei das OCS-Feld selbst als null gilt; das Pad davor wird getrennt auf null geprüft und ist so mitgeschützt [2, Sec. 8 und 9]. Zweitens geht zusätzlich die Länge der gesamten Surplus Area einschließlich des Pads als vorzeichenloser 16-Bit-Summand in die Rechnung ein, obwohl dieser Wert in keinem eigenen Feld des Pakets steht [2, Sec. 9]. Das folgt demselben Gedanken wie der Pseudo-Header aus Abschnitt 2.2.1: Wie die UDP-Prüfsumme dort die `UDP Length` einbezieht, bezieht das OCS die Surplus-Länge ein, und beide Seiten können diesen Summanden unabhängig voneinander aus den Längensummanden ableiten. Der Sender trägt das invertierte Ergebnis in das Feld ein. Abschnitt 9 verlangt ein OCS ungleich null, sobald die UDP-Prüfsumme ungleich null ist, und kennzeichnet ein ungenutztes OCS mit null [2, Sec. 9]. Eine ausdrückliche Senderegeln für eine errechnete Null enthält der Abschnitt nicht; weil der Nullwert die unbenutzte Form kennzeichnet, bleibt für ein verwendetes OCS im Einerkomplement nur die Abbildung auf `0xFFFF`, analog zur UDP-Prüfsumme [1]. Der Empfänger summiert die Wörter der Surplus Area einschließlich des OCS-Felds sowie den Längensummanden und erwartet wie in Abschnitt 2.2.1 lauter Einsen, also `0xFFFF` [12].

Tabelle 2.2 führt diese Rechnung an einem bewusst kleinen Beispiel vollständig vor.

Die Optionswörter stehen für zwei `NOP`, ein `EOL` und ein Nullbyte. Die zwei `NOP` dienen nur der Rechnung; als Füllung vor `EOL` rät RFC 9868 von ihnen ab. Ein regulärer Sender schreibe `EOL` unmittelbar hinter das OCS [2, Sec. 11.1].

Diese Bauart hat einen praktischen Hintergrund: Manche fehlerhafte Middleboxes berechnen die UDP-Prüfsumme über die gesamte IP-Nutzlast statt nur bis `UDP Length`. Weil das OCS über die Surplus Area und deren Länge gebildet und

Tabelle 2.2: OCS-Rechnung einer 6-Byte-Surplus-Area bei geradem Startoffset

Schritt	Sender	Empfänger
Optionswörter hinter dem OCS	0x0101, 0x0000	0x0101, 0x0000
Längensummand	0x0006	0x0006
OCS-Wort	0x0000 als Platzhalter	empfangenes 0xFE8
Einerkomplementsumme	0x0107	0xFFFF
Ergebnis	Invertierung ergibt 0xFE8	OCS ist gültig

anschließend invertiert wird, neutralisiert es den Beitrag der fälschlich einbezogenen Surplus Area: Mit gesetztem OCS fällt die UDP-Prüfsumme in beiden Rechnungen gleich aus; an dieser fehlerhaften Ausdehnung des Prüfbereichs scheitert das Datagramm bei solchen Geräten also nicht [2, Sec. 9]. In erster Linie dient das OCS ohnehin dem Erkennen zufälliger Fehler und anderweitiger, nicht standardkonformer Nutzungen der Surplus Area; ein Schutz gegen gezielte Angriffe ist es ausdrücklich nicht [2, Sec. 9]. Wie die UDP-Prüfsumme ist das OCS zudem bedingt abschaltbar: Der Wert 0x0000 bedeutet ungenutzt, zulässig ist das nur, wenn auch die UDP-Prüfsumme null ist, und Implementierungen müssen standardmäßig ein echtes OCS setzen [2, Sec. 9]. Schlägt die Prüfung beim Empfänger fehl, gilt die Grundregel aus Abschnitt 2.4.2: Optionen still verwerfen, Nutzdaten bei bestandener oder zulässig ungenutzter UDP-Prüfsumme dennoch zustellen [2, Sec. 9].

2.4.4 TLV-Format und Empfangsentscheidung

Die Optionen hinter dem OCS folgen dem TLV-Muster, dessen Syntax RFC 9868 bewusst an TCP anlehnt (Abschnitt 2.3): Ein 1 Byte großes Feld **Kind**, benannt nach dem TCP-Vorbild, identifiziert die Option; das 1 Byte große Feld **Length** zählt die Gesamtlänge der Option einschließlich **Kind** und **Length** selbst; dahinter kann ein Wert folgen [2, Sec. 8 und 10]. Für die Größe des Werts ist das nur scheinbar eine enge Grenze: Im Standardformat fasst eine Option höchstens 254 Byte einschließlich **Kind** und **Length**; größere Optionen weichen auf ein erweitertes Format aus, in dem der Wert 255 im **Length**-Feld ein zusätzliches 16-Bit-Längenfeld ankündigt. Damit sind je Option höchstens 65535 Byte einschließlich der Kopffelder kodierbar [2, Sec. 10]. Eine zusätzliche Gesamtgrenze für den Optionsraum kennt RFC 9868 dagegen bewusst nicht: Wie das Entwurfsprinzip der fehlenden Längengrenze in Abschnitt 2.4.5 festhält, gilt allein die Grenze des UDP-Pakets selbst; bei einem nicht fragmentierten Datagramm begrenzt somit der im IP-Datagramm verfügbare Platz, wie viele Optionen es trägt. Auch die beiden Ein-Byte-Codes aus Abschnitt 2.3 kehren

als einzige Ausnahmen ohne Längenfeld wieder: **NOP** (Kind 1) dient mit impliziter Länge eins der Ausrichtung zwischen Optionen und darf sich dafür wiederholen, **EOL** (Kind 0) beendet die Liste vorzeitig; die Bytes dahinter muss der Sender bis zum Ende der Surplus Area beziehungsweise des Optionsbereichs eines UDP-Fragments auf null setzen [2, Sec. 8 und 11.1]. Der Empfänger darf diese Nullfüllung prüfen, muss es aber nicht; findet er dabei ein Byte ungleich null, verwirft er nach der Grundregel aus Abschnitt 2.4.2 die gesamte Optionsliste still [2, Sec. 11.1]. Variable Optionslängen und künftige Erweiterungen sind damit im Format selbst angelegt.

Für die Anzahl der Optionen gilt dasselbe Bild wie für ihre Länge: Eine feste Obergrenze gibt es nicht; ohne vorzeitiges **EOL** endet die Liste erst am Ende der Surplus Area. Begrenzend wirken stattdessen zwei Regelgruppen. Erstens die Wiederholungsregeln: Außer **FRAG**, **NOP** und den Experimentaloptionen soll jede Option höchstens einmal je Datagramm vorkommen, ein Empfänger wertet von einer solchen Option ohnehin nur die erste Instanz, und ein mehrfaches **FRAG** macht die gesamte Optionsliste ungültig [2, Sec. 10]; **NOP** wiederum soll höchstens siebenmal in Folge stehen und nicht als Ersatz für **EOL** samt Nullfüllung dienen [2, Sec. 11.2]. Zweitens die Vollständigkeitsregel: Jede Option muss ganz in die Surplus Area passen. Meldet ein **Length**-Feld eine Länge über deren Ende hinaus, gilt die gesamte Surplus Area als fehlerhaft, und der Empfänger verwirft still alle Optionen, nicht nur die angeschnittene [2, Sec. 10]; das verifizierte Erratum EID 8834 bestätigt diese Regel gegen eine ältere, weichere Formulierung in Abschnitt 14 [15]. Im Regelfall entsteht diese Lage nicht, denn der Sender muss die Surplus Area von vornherein so bemessen, dass das OCS und alle Optionen vollständig hineinpassen [2, Sec. 8].

Damit sind die Prüfschritte des Standardpfads beisammen. Abbildung 2.8 ordnet sie in der Reihenfolge, in der ein optionsfähiger Empfänger sie durchläuft, und stellt ihnen die vorgelagerte Prüfung des Datagramms selbst voran:

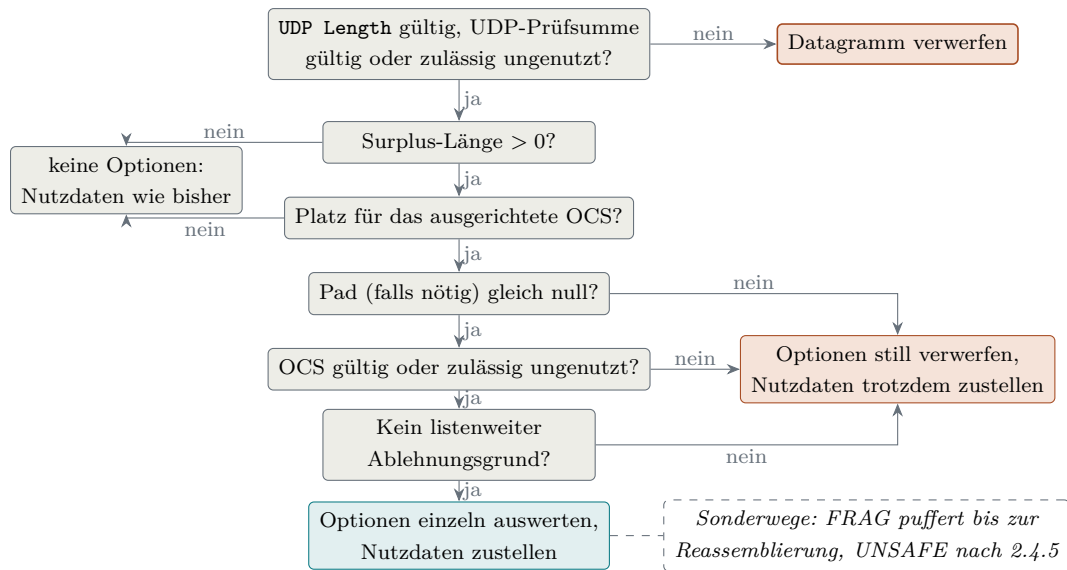


Abbildung 2.8: Empfangsentscheidung für die Surplus Area im Standardfall

Aus Abbildung 2.8 geht hervor, dass nur die vorgelagerte Prüfung des Datagramms selbst Nutzdaten kosten kann: Eine ungültige `UDP Length` oder eine falsche UDP-Prüfsumme verwirft das ganze Datagramm [2, Sec. 10 und 14]. Alle Prüfungen der Surplus Area treffen dagegen höchstens die Optionen: Die beiden Größenfragen führen auf den harmlosen Ausgang links, die drei Prüfungen dahinter auf den Fehlerausgang rechts; genau das ist die Grundregel aus Abschnitt 2.4.2 im Bild. Bei der Auswertung selbst wird einzeln ignoriert, was nur lokal stört: eine unbekannte `SAFE`-Option ebenso wie spätere Instanzen einer gewöhnlichen wiederholten Option. Die ganze Liste ist bei den strukturellen Längenfehlern aus dem vorigen Absatz und bei mehrfachem `FRAG` ungültig; daneben darf der Empfänger sie verwerfen, wenn verpflichtend zu unterstützende Optionen (im RFC *must-support*, ausgenommen `NOP` und `EOL`) nicht vor den übrigen `SAFE`-Optionen stehen, und er muss es, wenn seine freiwillige Nullfüllungsprüfung hinter `EOL` ein Byte ungleich null findet [2, Sec. 10 und 11.1]. Die gestrichelte Randnotiz markiert die Sonderwege: Ein UDP-Fragment wird erst gepuffert und nach der Reassemblierung zugestellt; `UNSAFE`-Optionen und strengere, von der Anwendung angeforderte Empfangsregeln behandelt der folgende Unterabschnitt.

2.4.5 Entwurfsprinzipien und Optionsklassen

Wie dieser Bereich genutzt werden darf, bindet RFC 9868 an sechs Entwurfsprinzipien, die ein gemeinsames Ziel haben: UDP soll durch die Optionen seinen Charakter nicht verlieren [2, Sec. 6]:

- **Zustandslos:** Nötiger Zustand gehört in die Anwendung oder in eine Bibliothek, die in ihrem Auftrag arbeitet. Die einzige, ausdrücklich begrenzte Ausnahme

ist die Zusammenführung von Fragmenten.

- **Unidirektional:** Die UDP-Schicht erzeugt nie von sich aus eine Antwort auf eine Option; ob und wann geantwortet wird, entscheidet die Anwendung oder eine Schicht beziehungsweise Bibliothek in ihrem Auftrag. Verfahren, die auf Antworten angewiesen sind, verlagert der RFC in eigene Dokumente. Als Einordnung dieser Arbeit, nicht als Begründung des RFC: Diese Zurückhaltung verhindert, dass schon der Grundmechanismus mit gefälschter Absenderadresse als Reflektor missbraucht wird; ein ausdrücklich aktiviertes Antwortverfahren muss diesen Missbrauch selbst abwehren, etwa durch die Ratenbegrenzung, die RFC 9869 für eigens als Antwort erzeugte Datagramme vorschreibt [3, Sec. 3.1].
- **Keine eigene Längengrenze:** Für den Optionsraum gilt allein die Grenze des UDP-Pakets selbst; bei einem nicht fragmentierten Paket ist das der im IP-Datagramm verfügbare Platz. Feste Grenzen werden erfahrungsgemäß überschritten; der RFC verweist auf die Erfahrung mit TCP-Optionen (40 Byte Optionsraum [9]) und IPv4-Adressen (32 Bit [5]). Grenzen darf eine Implementierung setzen, nicht die Spezifikation.
- **Kein Protokollersatz:** Optionen liefern Funktionen für den eigenen Verkehr einer Anwendung; sie sollen NTP, ICMP (insbesondere Echo) und Netzwerk-Testgeräte weder ersetzen noch nachbauen.
- **Rahmenwerk statt Protokoll:** RFC 9868 definiert Optionsformate auch dann, wenn sie noch kein vollständiges Verfahren ergeben; die Nutzung regeln andere Dokumente. Für REQ/RES übernimmt das RFC 9869 [3], für TIME bleibt sie bewusst offen. Das entspricht dem Muster von TCP, dessen Basisnorm den Optionsmechanismus festlegt, während einzelne Optionen in eigenen Dokumenten folgten.
- **Voreinstellung wie beim Altempfänger:** Empfangene Pakete werden standardmäßig auch dann zugestellt, wenn Prüfsummen-, Authentifizierungs- oder Entschlüsselungsprüfungen der Optionen scheitern; einzige Ausnahme sind UDP-Fragmente. Strengeres Verhalten muss die Anwendung ausdrücklich anfordern. Optionen sind ein Angebot, kein Filter.

Der gemeinsame Nenner der sechs Prinzipien: RFC 9868 legt die Optionsformate und ihre Grundverarbeitung fest, aber kein vollständiges Anwendungsprotokoll. UDP bleibt ein minimales Transportprotokoll, und jede zusätzliche Fähigkeit wird von der Anwendung ausdrücklich aktiviert [2, Sec. 6].

An das letzte Prinzip knüpft die zentrale Zweiteilung der Optionen an, und sie folgt unmittelbar aus der Abwärtskompatibilität: Weil ein Altempfänger die Surplus

Area überliest, muss jede Option damit rechnen, ignoriert zu werden. *SAFE*-Optionen (Kind 0 bis 191) sind genau dafür entworfen: Ein Empfänger, der sie nicht versteht, ignoriert sie, ohne dass sich die Bedeutung der Nutzdaten ändert. In diese Klasse gehören die Ein-Byte-Codes *NOP* und *EOL* ebenso wie *FRAG* [2, Sec. 10 und 11].

UNSAFE-Optionen (Kind 192 bis 255) können dagegen die Nutzdaten verändern oder eine Änderung ihrer Deutung signalisieren, etwa durch Kompression oder Verschlüsselung; ein Empfänger, der die Option nicht versteht, würde die Nutzdaten falsch deuten. RFC 9868 löst das mit einer Transportbeschränkung: *UNSAFE*-Optionen dürfen nur in UDP-Fragmenten auftreten, deren regulärer Nutzdatenteil leer ist; die transportierten Daten liegen im Fragmentdatenbereich der Surplus Area hinter der Optionsliste [2, Sec. 11.4]. Ein Altempfänger sieht dann nur ein leeres Datagramm und verwirft die Daten still, statt sie falsch zu deuten. Ein optionsfähiger Empfänger, der eine *UNSAFE*-Option nicht unterstützt, muss die reassemblierten Nutzdaten ebenso still verwerfen, und dasselbe gilt, wenn eine *UNSAFE*-Option außerhalb eines Fragment- oder Reassemblierungskontexts erscheint; zugestellt wird in beiden Fällen höchstens ein leeres Datagramm [2, Sec. 10, 12 und 14]. Die Einzelbewertung der 67 Matrixzeilen steht in Anhang A; die Umsetzung der Optionen beschreibt Kapitel 5.

Zwei Folgen dieser Prinzipien verdienen eine eigene Erwähnung. Erstens verteilt die *FRAG*-Option ein logisches Anwendungsdatagramm (im RFC *user datagram*) künftig gegebenenfalls auf mehrere IP-Datagramme. Zweitens nimmt der RFC einen ausdrücklich benannten Unterschied in Kauf: Ein Altempfänger sieht jedes UDP-Fragment als leeres Datagramm, ein optionsfähiger Empfänger das reassemblierte Datagramm mit Inhalt. Diese Zählungen gleicht der RFC nicht an, weil leere Datagramme als Lebenszeichen ohnehin unzuverlässig sind [2, Sec. 6 und 18].

2.5 Betriebssystem und Netzpfad

Ob ein Datagramm mit Surplus Area sein Ziel erreicht, entscheidet sich an zwei Stellen außerhalb des Protokolls: im Betriebssystem der beiden Endpunkte und auf dem Netzpfad dazwischen. Betrachtet wird dabei durchgehend der Linux-Netzwerkstapel für IPv4, auf dem die Referenzimplementierung dieser Arbeit läuft; andere Betriebssysteme und IPv6 bleiben ausgeklammert. Dieser Abschnitt stellt zunächst die zwei Socket-Sichten vor, mit denen der Kernel UDP-Verkehr an Anwendungen übergibt, und anschließend die Akteure des Pfades; am Ende fügt Abbildung 2.11 beides zu einem Gesamtbild zusammen. Allgemeine Netzwerkgrundlagen bleiben bewusst ausgespart: Es geht genau um die Kette, die die Messpfade dieser Arbeit tatsächlich durchlaufen.

Zwei Socket-Sichten. Beim gewöhnlichen UDP-Socket übernimmt der Kernel die Protokollarbeit. Empfangsseitig prüft er Länge und Prüfsumme, demultiplexiert nach Adresse, Port und Socketmerkmalen und liefert nur die Nutzdaten bis `UDP Length` [4]. Damit entspricht diese Sicht dem Altempfänger aus Abschnitt 2.4; getestete Standardsysteme schneiden die Surplus Area ab [2, Sec. 18]. Sendeseitig baut der Kernel beide Header und lässt hinter den Nutzdaten keinen Raum. Ein UDP-Socket kann die Surplus Area daher weder lesen noch erzeugen.

Der normale Kernelweg verläuft beim Senden von `write()` über `sock_sendmsg()` und `udp_sendmsg()` zu `ip_send_skb()` und `dev_queue_xmit()`; `udp_send_skb()` schreibt dabei den UDP-Header. Empfangsseitig führt der Weg von `netif_receive_skb()` über `ip_rcv()`, `udp_rcvmsg()` und `sock_rcvmsg()` zu `read()`; ausgeliefert wird erst nach der UDP-Prüfsumme [4, 16]. Diese Folge berichtigt zwei Beschriftungen der Vorlage (dort Abbildung 5): Auf der Socket-Schicht stehen `sock_sendmsg()` und `sock_rcvmsg()`, und der UDP-Sendepfad übergibt an `ip_send_skb()` statt `ip_queue_xmit()`.

Wer beides will, braucht die zweite Sicht: den *Raw Socket*, den Linux nur Prozessen mit der Berechtigung `CAP_NET_RAW` öffnet [17]. Er zweigt auf der IP-Schicht ab: Der Sender speist dort ein selbst gebautes Datagramm ein, der Empfänger erhält vor dem UDP-Pfad eine Kopie; die folgenden Abbildungen trennen beide Richtungen. Er arbeitet unterhalb von UDP direkt mit IP-Datagrammen: Die Anwendung schreibt den UDP-Header selbst, und die IP-Längenangabe bemisst der Kernel an der übergebenen Pufferlänge statt am UDP-Datagramm; alles, was die Anwendung hinter das UDP-Datagramm in den Puffer legt, wird so zur Surplus Area [17]. Mit der Socket-Option `IP_HDRINCL` baut sie darüber hinaus den IP-Header selbst, statt ihn vom Kernel erzeugen zu lassen [17]; diesen Weg nutzt die Referenzimplementierung, die damit das vollständige Datagramm selbst baut (Abschnitt 4.4). Die vertauschten Rollen beider Sendewege zeigt Abbildung 2.9:

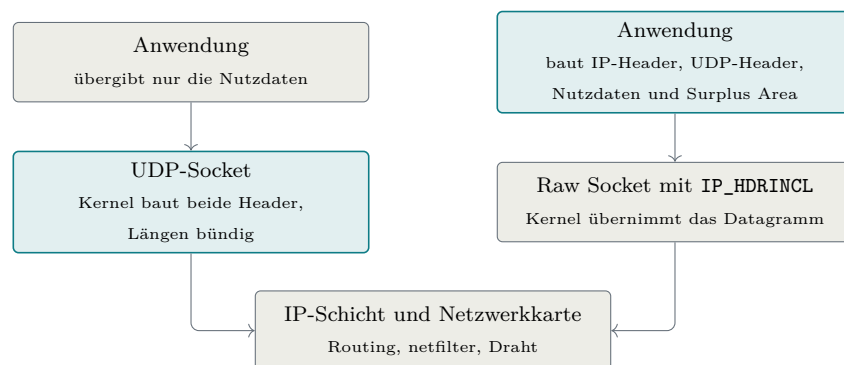


Abbildung 2.9: Die zwei Sendepfade durch den Linux-Kernel [4, 17]

Aus Abbildung 2.9 geht hervor, dass die Arbeitsteilung kippt: Am UDP-Socket

baut der Kernel beide Header und bemisst die Längen bündig, am Raw Socket liefert die Anwendung das fertige Datagramm ab, und nur auf dem Raw-Socket-Weg entsteht hinter den Nutzdaten Platz für eine Surplus Area. Der Unterbau bleibt derselbe: Beide Wege münden in die IP-Schicht mit ihren netfilter-Stationen und enden an der Netzwerkkarte [4].

Ganz entlässt der Kernel den Raw-Sender allerdings nicht aus seiner Obhut, und diese Restarbeit prägt den Sendepfad der Implementierung. Routing, Nachbarauflösung und Link-Schicht bleiben Kernelsache, und im übergebenen IP-Header trägt der Kernel `Total Length` und die IP-Header-Prüfsumme stets selbst ein; UDP-Header und Surplus Area lässt er selbst dagegen unangetastet, die netfilter-Stationen dieses Wegs können das Datagramm gleichwohl noch verwerfen oder verändern [4, 17]. Zugleich fragmentiert dieser Pfad nicht: Mit `IP_HDRINCL` ist ein Datagramm auf die MTU des Interfaces begrenzt, ein größerer Sendeaufruf schlägt fehl [17]. Für RFC 9868 ist das kein Verlust: IP-Fragmentierung sollen UDP-Anwendungen ohnehin vermeiden (Abschnitt 2.1) [8, Sec. 3.2], und für große Nutzlasten stellt der RFC stattdessen die FRAG-Option auf Transportebene bereit.

Beim Empfang entscheidet sich, an welcher Stelle ein Datagramm geprüft wird; den Weg vom Draht zu den beiden Sichten zeichnet Abbildung 2.10 nach:

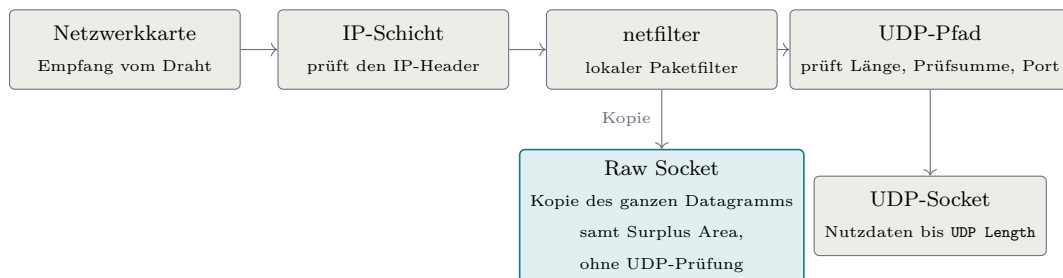


Abbildung 2.10: Der Empfangspfad durch den Linux-Kernel [4, 17]

Aus Abbildung 2.10 geht hervor, dass sich der Weg hinter dem Paketfilter gabelt: Der Raw Socket erhält eine Kopie des gesamten IP-Datagramms samt Surplus Area, und zwar bevor der UDP-Pfad des Kernels Länge, Prüfsumme und Portzuordnung geprüft hat [4, 17]. Ungeprüft heißt dabei: frei von jeder UDP-Prüfung; den IP-Header hat die IP-Schicht zu diesem Zeitpunkt dagegen bereits geprüft, wie die Abbildung zeigt. Ein Datagramm mit falscher UDP-Prüfsumme oder einer `UDP Length`, die über die IP-Nutzlast hinausweist, erreicht den Raw Socket deshalb unverändert. Die Prüfungen, die der Kernel dem UDP-Socket abnimmt, muss eine Raw-Empfangsschicht deshalb vollständig selbst ausführen, und zwar in der Reihenfolge, die RFC 9868 vorgibt: erst Mindestlänge und Längenkonsistenz des UDP-Headers, dann die UDP-Prüfsumme, erst danach Surplus-Ortung und OCS [2, Sec. 10 und 14]; wie die

Implementierung das umsetzt, zeigt Kapitel 5. Zwei Arbeiten behält der Kernel dagegen auch hier. Die netfilter-Hooks bleiben beiden Sichten vorgeschaltet: Ein lokaler Paketfilter kann ein Datagramm verwerfen, bevor irgendeine Anwendung es sieht [4]. Und ankommende IP-Fragmente setzt die IP-Schicht bereits vor der Zustellung wieder zusammen, auch für Raw Sockets; eine Raw-Empfangsschicht muss die IPv4-Reassemblierung deshalb nicht nachbauen [4, 17].

Akteure auf dem Netzpfad. Zwischen den Endpunkten kann das Datagramm auf *Middleboxes* treffen. RFC 8085 nennt damit Zwischensysteme wie NATs und Firewalls, die zusätzliche Funktionen ausführen und typischerweise Zustand je Fluss führen [8, Sec. 3.5]. Für die Interpretation der späteren Messungen genügt der Sammelbegriff nicht, denn die Akteure setzen an verschiedenen Stellen an und hinterlassen verschiedene Spuren. Im Zugangsnetz ersetzt die NAT eines Heimrouters oder Hotspots Adressen und Ports und bindet jeden Fluss an einen Zustandseintrag mit Zeitschranke; ihre Provider-Variante *Carrier-Grade NAT* (CGNAT) teilt dieselben öffentlichen Adressen zusätzlich unter vielen Kunden auf. Firewalls verwerfen an Netzgrenzen und Endsystemen nach Regelwerk, etwa anhand von Ports und Protokollen. Leicht zu übersehen sind die letzten beiden Akteure: die Virtualisierungsschicht eines Testrechners, deren eigener Netz- und NAT-Stack Kopffelder auf erwartete Normwerte umschreiben kann, und das Transitnetz selbst, die AS-Kette zwischen den Zugangsnetzen, in der einzelne Netze filtern.

Ob ein einzelner Akteur UDP Options kennt, lässt sich von außen nicht feststellen. Unbekannte Zwischensysteme können eine Surplus Area unverändert weiterleiten, verändern, entfernen oder das gesamte Datagramm verwerfen. Zustandsbehaftete NATs und Firewalls können dabei je Richtung und Messzeitpunkt unterschiedlich reagieren [8, Sec. 3.5]. Eine Kapselung, etwa in einen WireGuard-Tunnel, verbirgt das innere Datagramm vor den Akteuren zwischen den Tunnelendpunkten, die nur noch das äußere Tunnelpaket behandeln. Welche dieser Eingriffe auf den untersuchten Pfaden beobachtet wurden, bewertet Kapitel 6.

Diese Kette erklärt zugleich, warum RFC 9868 seine Optionen ausgerechnet in einem Bereich ablegt, den Altempfänger nie lesen. Weil Middleboxes bevorzugt durchlassen, was sie kennen, weichen Protokollerweiterungen in Bereiche aus, die Bestandssysteme nicht deuten; dieses Erstarren des Netzes unter seinen Zwischensystemen wird als *Ossifikation* bezeichnet. Unsichtbar ist die Surplus Area allerdings nur für die Endpunkte alter Anwendungen; für die Geräte auf dem Pfad bleibt sie ohne Verschlüsselung sichtbar [2, Sec. 16], und RFC 9868 rechnet ausdrücklich mit deren Eingriffen: Fehlerhaft rechnende Middleboxes begründen Details der OCS-Berechnung (Abschnitt 2.4) [2, Sec. 9], und UDP-Relays können eine Surplus Area

beim Weiterleiten vollständig abstreifen [2, Sec. 25.6].

Alle Stationen zusammen ordnet Abbildung 2.11 zu einem Messmodell des Weges von der sendenden Anwendung bis zu dem Raw-Empfänger, der die Surplus Area auswertet; je nach Pfad können einzelne Stationen fehlen oder zusammenfallen.

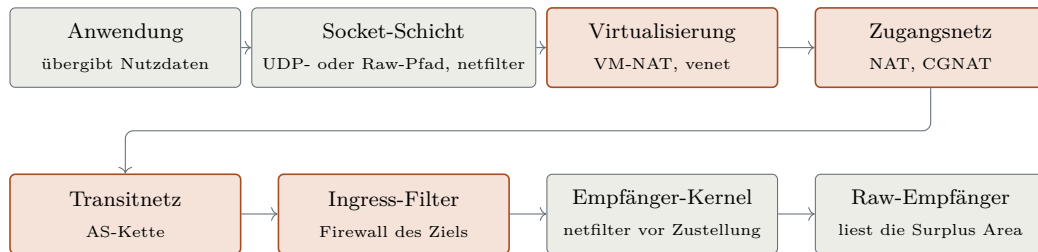


Abbildung 2.11: Messmodell des Datagrammwegs

Aus Abbildung 2.11 geht hervor, dass zwischen Socket-Schicht und Empfänger-Kernel bis zu vier Stationen liegen, die das Datagramm außerhalb der Endpunkte berühren: Sie greifen teils mit Zustand ein und sind von außen nicht direkt beobachtbar. Für die Feldmessungen folgt daraus die Interpretationsregel dieser Arbeit: Geht ein optionsbehaftetes Datagramm verloren oder kommt es verändert an, ist der Befund zuerst dieser Kette zuzuordnen, bevor er der Implementierung oder dem Standard angelastet wird. Messpunkte an beiden Enden grenzen ihn dabei auf ein Intervall der Kette ein; welches Gerät eingreift, bleibt ohne zusätzliche Messpunkte oder unabhängige Evidenz offen (Kapitel 6).

Wireshark und tshark 4.6 dekodieren RFC 9868 nicht: Ihre UDP-Ansicht endet ohne Feld, Trailer oder Warnung an **UDP Length**. Die Kampagnen benötigen deshalb einen eigenen Prüfer für PCAP-Dateien (Kapitel 6); der fehlende Dissektor zeigt zugleich die geringe bisherige Werkzeugunterstützung.

Die Auswertungsmethode folgt in Kapitel 3.

3. Methodik und KI-gestützte Entwicklung

Große Sprachmodelle unterstützten die Auswertung des RFC-Primärtexts, den Entwurf und die Implementierung, die Analyse der Messkampagnen und die Prüfung von Zwischenergebnissen. Wissenschaftlich belastbar ist dieses Vorgehen nur, wenn die Arbeit offenlegt, wie Modellausgaben geprüft werden und wer die Ergebnisse verantwortet. Diese Regeln müssen vor der Ergebnisbewertung feststehen. Deshalb steht dieses Kapitel vor der RFC-Analyse.

3.1 Anlass des KI-Einsatzes und veröffentlichte Fälle

Diese Arbeit entsteht als Einzelarbeit und verbindet die Auswertung einer neuen Norm, systemnahe Programmierung, formale Modellierung und Messkampagnen. Der Einsatz von KI-Werkzeugen wird deshalb nicht beiläufig erwähnt, sondern vorab offengelegt und mit Prüfregeln versehen. Zur Einordnung dienen neun veröffentlichte Fälle, in denen KI-Systeme, darunter Sprachmodelle, an Forschungs- und Entwicklungsergebnissen beteiligt waren. Sie bilden keine systematische Literaturübersicht, belegen keine einheitliche Prüfkette und erlauben keine Aussage über Häufigkeit oder Wirksamkeit eines Verfahrens. Die folgende Auflistung nennt nur die in den Quellen ausdrücklich dokumentierten Rollen: den Beitrag des KI-Systems, die prüfende Instanz und die Grenze der jeweiligen Prüfung.

- **AlphaFold:** Ein Vorhersagemodell bestimmte Proteinstrukturen; der blinde CASP14-Vergleich mit experimentell ermittelten Strukturen prüfte die Ergebnisse, eine allgemeine Prüfkette entsteht daraus nicht [18].
- **Davies u. a.:** Ein KI-System schlug Hypothesen und Kandidatenmuster für mathematische Probleme vor; beteiligte Mathematiker prüften durch Gegenbeispielsuche und führten den klassischen Beweis [19].
- **FunSearch:** Ein Sprachmodell erzeugte ausführbare Programmkandidaten, die ein ausführbarer Auswerter einzeln bewertete; das Ergebnis gilt nur für die kodierte Bewertungsfunktion [20].
- **AlphaProof:** Ein Modell erzeugte formale Beweiskandidaten, die der Kern des Beweisassistenten Lean einzeln nachprüfte; jeder Beweis gilt für die formalisierte

Aussage und ihre Annahmen [21].

- **Knuths Zyklen:** Ein Modell konstruierte die Lösung eines Zyklenproblems; Knuth führte den Beweis anschließend selbst, und unabhängig davon lag bereits eine veröffentlichte Lean-Formalisierung von Morrison vor [22].
- **Zeta-Schranke:** Ein Modell lieferte ein mathematisches Ergebnis samt gemeinsamer Lean-Formalisierung; zwei Anthropic-Mathematiker und zwei externe Fachleute prüften es, ein dokumentierter Blindvergleich fand nicht statt [23].
- **Einheitsabstand:** Ein Modell lieferte einen Gegenbeweis zu einer Vermutung der diskreten Geometrie; externe Mathematiker prüften ihn und veröffentlichten eine verdichtete, von Menschen verifizierte Fassung [24].
- **C-Parameter:** Ein Modell steuerte unter Aufsicht Rechnungen, Numerik und Manuskriptteile bei; Physiker prüften mit einer Gegenprobe durch Monte-Carlo-Simulationen, und der Preprint legt den Einsatz offen [25].
- **Bun-Migration:** Bei der Migration von Bun von Zig nach Rust erzeugten und prüften Modelle derselben Familie den Quelltext; trotz getrennter Rollen blieb ein Fehler unentdeckt, nach dem Merge wurde undefiniertes Verhalten gemeldet und das Prüfwerkzeug Miri in die kontinuierliche Integration aufgenommen [26–28].

Die Fälle belegen nur, dass die Prüfinstanz zum Artefakt passen muss. Normtext, ausführbare Tests, formale Beweise, Messdaten und menschliche Freigabe haben verschiedene Aufgaben; ihre tatsächliche Nutzung muss dokumentiert sein. Diese Arbeit folgt außerdem der ausdrücklichen Offenlegung des C-Parameter-Falls (Abschnitt 1.6) und versteht auch eine mehrstufige Prüfung nicht als Garantie. Sie wendet denselben Maßstab auf sich selbst an: Jede Artefaktart erhält eine passende Prüfinstanz, und die Arbeitsteilung zwischen Autor und Werkzeugen steht fest, bevor Ergebnisse bewertet werden. Beides legen die folgenden Abschnitte offen.

3.2 Forschungsdesign und KI-gestützte Arbeitsweise

Die Untersuchung verbindet die Auswertung der Norm, die Referenzimplementierung und eine empirische Pfaduntersuchung. Abbildung 3.1 zeigt, wie die beiden Forschungsfragen auf diesen Teilen aufbauen:

Aus Abbildung 3.1 geht die Doppelrolle der Referenzimplementierung hervor: Für FF1 ist sie der Prüfgegenstand, und das aus den Normquellen abgeleitete Anforderungsmodell bildet den Maßstab; für FF2 dient dieselbe Implementierung als

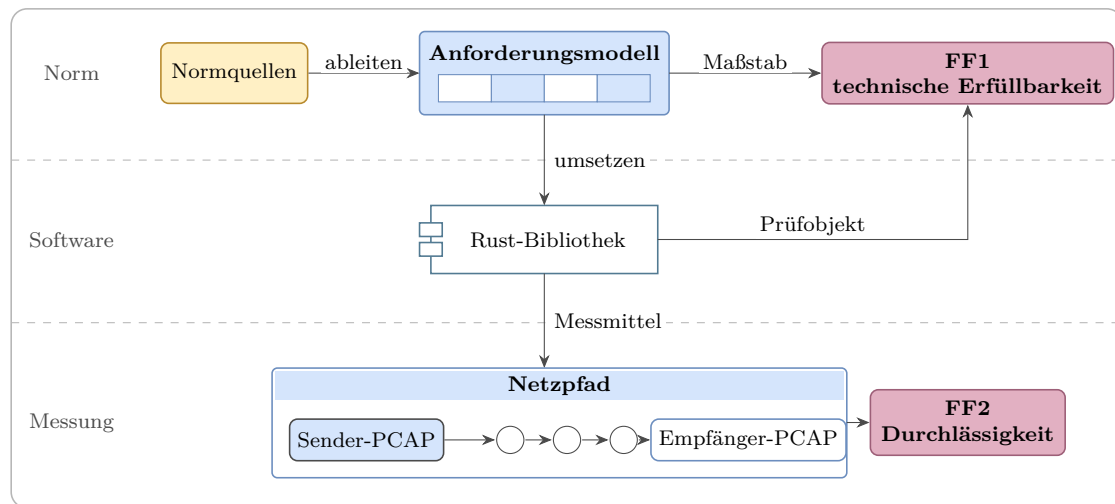


Abbildung 3.1: Aufbau der Untersuchung und Zuordnung der Forschungsfragen

Messmittel. Beide Bewertungen bleiben getrennt: Eine konforme Implementierung belegt keine Zustellbarkeit entlang realer Netzpfade, und eine unveränderte Zustellung belegt keine RFC-Konformität.

Die Forschungsfragen benötigen deshalb verschiedene Untersuchungseinheiten und Entscheidungsregeln. Tabelle 3.1 legt diese Regeln fest, bevor die Ergebnisse betrachtet werden:

Tabelle 3.1: Operationalisierung der Forschungsfragen

FF	Untersuchungs-einheit	Nachweis	Bewertung und Grenze
FF1	jede anwendbare normative Anforderung im Projektumfang	je nach Status: Normfundstelle sowie Quelltext und Prüfung oder Beleg einer technischen Grenze	vollständig, teilweise oder nicht erfüllbar; nicht anwendbar bleibt unbewertet
FF2	kontrolliertes Szenario je Pfad, Richtung und Messzeitpunkt	Paketmitschnitte an Sender und Empfänger, Sendemanifest und dokumentierte maschinelle Auswertung	Surplus erhalten, verändert oder entfernt; Datagramm verworfen oder Lauf ungültig. Gilt nur für Pfad, Richtung und Messzeitpunkt

Aus Tabelle 3.1 geht hervor, dass FF1 je anwendbarer Anforderung und FF2 je kontrolliertem Szenario entschieden wird. Ein Szenario kann ein einzelnes Kontroll- und Optionsdatagramm, mehrere Wiederholungen oder bei FRAG mehrere Wire-Datagramme enthalten. Für FF1 wird zunächst die Anwendbarkeit bestimmt. Anwendbare **MUST**- und **SHOULD**-Aussagen gehen in die Bewertung ein; eine Abweichung von **SHOULD** benötigt eine dokumentierte Begründung. Eine **MAY**-Funktion wird nur bewertet, wenn sie für die Referenzimplementierung gewählt wurde.

Wie diese Regeln auf die einzelnen Anforderungen angewandt werden, legt Abschnitt 4.2 fest; die Auswertungsregeln der Messläufe für FF2 stehen in Abschnitt 6.1. Beide Regelwerke stehen damit ebenfalls vor der Ergebnisbewertung fest.

Der Maßstab für FF1 entsteht aus dem Primärtext. Normativ ist RFC 9868; RFC 768 liefert ergänzend das UDP-Format, RFC 1071 als informative Referenz die Rechenregeln der Prüfsummen. Die Schlüsselwörter **MUST**, **SHOULD** und **MAY** werden nach BCP 14¹ gelesen. Nur die großgeschriebenen Formen tragen die dort festgelegte besondere Bedeutung; normative Aussagen können die Schlüsselwörter aber auch auslassen [29, 30]. RFC 9868 stellt Absätzen mit solchen Wörtern zusätzlich das Zeichenpaar >> voran. Die eigene Zählung von 103 markierten Absätzen ist deshalb nur eine Kontrollzahl und keine vollständige Anforderungsliste.

Jede so gewonnene Aussage wird nach demselben Muster erfasst: Fundstelle, Normstufe, Anwendbarkeit und Projektumfang. Dieses Muster ist zugleich das Zeilenschema des Anforderungskatalogs im Implementierungs-Repository, sodass jede Katalogzeile ihren RFC-Abschnitt mitführt. Zwei Nachbarquellen bleiben davon getrennt. RFC 9869 beschreibt die Pfad-MTU-Bestimmung (DPLPMTUD, *Datagram Packetization Layer Path MTU Discovery*) mit UDP-Optionen und liegt außerhalb des Implementierungsumfangs [3]. Von den Errata ist das verifizierte technische EID 8834 angewendet; es löst den Widerspruch zwischen den Abschnitten 10 und 14 zur Behandlung überlanger Optionen [15]. Das redaktionelle EID 8708 bleibt für eine spätere Dokumentausgabe zurückgestellt [31]. Abbildung 3.2 fasst diesen Weg zusammen:

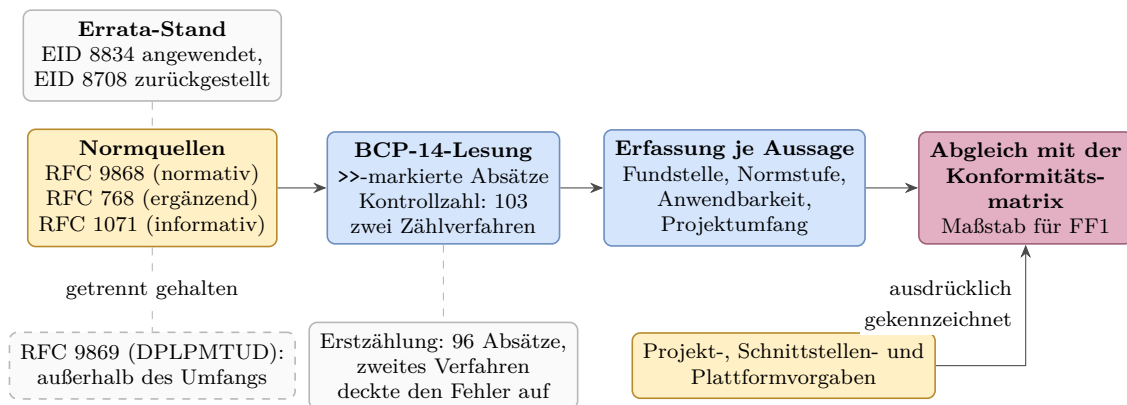


Abbildung 3.2: Von den Normquellen zum geprüften Anforderungsbestand

Aus Abbildung 3.2 geht hervor, dass der Maßstab aus zwei getrennt gehaltenen Zuflüssen entsteht: der normativen Lesung mit ihrem Errata-Stand und den ausdrücklich gekennzeichneten eigenen Projekt-, Schnittstellen- und Plattformvorgaben.

¹ BCP steht für Best Current Practice. BCP 14 umfasst RFC 2119 und dessen Aktualisierung RFC 8174.

Diese Trennung verhindert, dass eine lokale Entwurfsentscheidung nachträglich als Forderung des RFC erscheint. Die Kontrollzahl sichert den normativen Zufluss mit zwei Zählverfahren ab: Die Erstzählung ergab 96 Absätze, erst das zweite Verfahren deckte den Fehler auf. Ein Werkzeug, das den RFC-Text maschinell zerlegt, gibt es nicht; Zählung und Einordnung entstanden lesend.

Diese Erfassung liegt nicht im Quelltext, sondern in versionierten Textdokumenten des Implementierungs-Repositorys. Sie sind ausdrücklich an die Werkzeuge gerichtet: Die zentrale Vertragsdatei nennt in ihrem ersten Satz das Modellwerkzeug und menschliche Mitwirkende als Adressaten.² Damit tragen die Dokumente den Wissensstand zwischen den Arbeitssitzungen und sind zugleich der Maßstab, an dem eine Modellausgabe geprüft wird. Der Preis dieser mehrfachen Ablage ist belegt: Ein Abgleich am Primärtext fand vier falsche RFC-Angaben in vier Vertragsdokumenten zugleich (Abschnitt 5.1).

Die Versionsgeschichte schreibt diese Vorgaben dem Autor zu: Der erste Commit des Repositorys legte die Roadmap und die Schrittdateien an, der zweite am selben Tag den Anforderungskatalog sowie die Architektur- und Formatreferenz. Die Roadmap bezeichnet sich als Masterplan, der Schritt für Schritt mit einem Menschen in der Schleife ausgeführt wird: ein geprüfter Commit je Schritt. Der Autor legt damit Plan, Maßstab und Abschlusskriterien fest und gibt jedes Ergebnis frei; die Werkzeuge arbeiten innerhalb dieser Vorgaben. Zwei Grenzen bleiben: Die Sammelcommits verschmelzen die innere Reihenfolge eines Schritts, und die Commit-Historie trennt den Beitrag von Autor und Werkzeug je Änderung nicht auf.

Denselben Aufbau hat der Plan. Jeder Hauptschritt bekommt eine eigene Datei nach festem Schema mit Ziel, Anforderungen, Lean-Pflichten, Plan, Aufgaben und Abschlusskriterien; alle Schrittdateien lagen vor dem ersten Funktionsschritt vor. Wo Lean-Pflichten bestehen, verlangen neun Schrittdateien wörtlich die Spezifikation vor der Umsetzung und den Beweis danach; Abbildung 3.3 zeigt den Kernzyklus:

² Implementierungs-Repository, Commit `f1cfe52`: `CLAUDE.md`, `docs/requirements.md`, `docs/plan/ROADMAP.md` sowie die Schrittdateien unter `docs/plan/steps/`.

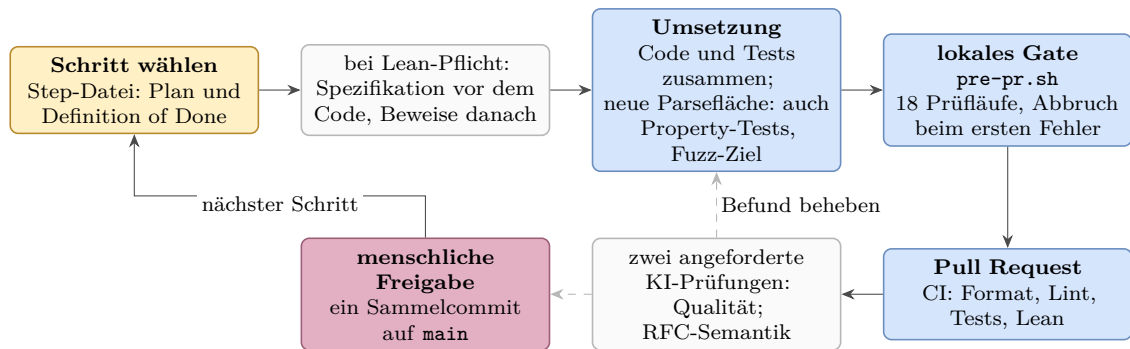


Abbildung 3.3: Kernzyklus eines Implementierungsschritts bis Schritt 18 (Ausnahmen gestrichelt)

Sprachmodelle sind in den Untersuchungen Werkzeuge, aber keine bewertende Instanz. Maßgeblich sind die Normquellen, die Messdaten, ausführbare Prüfungen und die Freigabe durch den Autor. Tabelle 3.2 trennt die Rollen von Autor und Werkzeugen nach Aufgabe und Grenze:

Tabelle 3.2: Rollen bei Erzeugung, Prüfung und Freigabe

Beteiligter	Aufgabe	Grenze
Autor	legt Plan, Anforderungskatalog, Architektur- und Formatvorgaben sowie die Schrittdateien fest; entscheidet, gewichtet, gibt frei und verantwortet	jedes Ergebnis gilt erst nach seiner Freigabe
Claude Code (Anthropic)	setzt innerhalb dieser Vorgaben um: Entwürfe, Quelltext, Tests und Analysen	Modellausgaben sind keine Nachweise
Codex (OpenAI)	getrennter Zweitprüfer	Herstellertrennung beweist keine Unabhängigkeit

Beide Modellwerkzeuge verbinden ein Sprachmodell mit Dateizugriff, Kommandoausführung und Repository. Die Herstellertrennung soll vollständig gleiche Fehlermuster verringern, beweist aber keine Unabhängigkeit. Eine Prüfung durch dasselbe Modell ist keine getrennte Zweitprüfung. Änderungen durch den Zweitprüfer erfordern einen eigenen, vom Autor freigegebenen Auftrag und zählen dann nicht als Prüfung desselben Ergebnisses.

Die Art der Prüfung richtet sich nach der Art der Aussage. Protokollaussagen werden gegen RFC-Primärtext und Errata geprüft. Aussagen über die Implementierung werden durch Quelltext und ausführbare Prüfungen gestützt. Pfadbefunde benötigen Paketmitschnitte und Kampagnenprotokolle. Literaturbehauptungen werden an der

jeweiligen Primärquelle geprüft. Ein Hashwert belegt nur die Unverändertheit eines Artefakts, nicht die Wahrheit seines Inhalts.

Im Umgang mit den Modellen gilt die Haltung, sie wie fähige, aber unerfahrene Entwickler zu behandeln, deren Arbeit grundsätzlich geprüft wird. Aus diesem Grund wurden drei Arbeitsregeln schriftlich festgehalten:

1. Modellergebnisse werden gegen die jeweilige Primärquelle geprüft, nie gegen eigene Zusammenfassungen.
2. Aufträge werden neutral formuliert, ohne das Modell in eine gewünschte Richtung zu lenken.
3. Widerspruch wird ausdrücklich ermutigt: Ein Modell, das „nein“ sagt oder Unsicherheit meldet, ist wertvoller als eines, das gefällig zustimmt.

Diese Regeln beschreiben den Sollprozess. Welche Prüfungen tatsächlich stattfanden, wird nicht aus ihm abgeleitet, sondern aus den vorhandenen Protokollen rekonstruiert (Abschnitt 3.5). Warum diese Trennung nötig ist, zeigt der folgende Abschnitt.

3.3 Bekannte Schwächen der Sprachmodelle

Der Einsatz von Sprachmodellen bringt bekannte Schwächen mit. Tabelle 3.3 ordnet ihnen die beobachtete Wirkung und die methodische Antwort zu.

Tabelle 3.3: Schwächen der Sprachmodelle und methodische Antworten

Schwäche	Wirkung und Beleg	Antwort der Methodik
Halluzination	überzeugend formulierter, aber falscher Inhalt; Trainings- und Bewertungsverfahren können Raten gegenüber eingestandener Unsicherheit belohnen [32]	Prüfung an der Primärquelle, nicht an eigenen Zusammenfassungen
Gefälligkeit (Sycophancy)	erkennbare Erwartungen können Widerspruch unterdrücken; das Verhalten tritt bereits bei reinen Vortrainingsmodellen auf, steigt bei größeren Modellen, bleibt nach Training mit menschlicher Rückmeldung bestehen und verursacht in Versuchen über elf Modelle messbare Schäden [33, 34]	neutrale Aufträge, ausdrücklich erwünschter Widerspruch und getrennter Zweitprüfer
Verfügbarkeit	Cloud-Dienste können ausfallen, gedrosselt oder verändert werden; wechselnde Modelle und nichtdeterministische Antworten verhindern die vollständige Reproduktion einer Modellausgabe	dauerhafte, ohne den Dienst prüfbare Artefakte: Quelltext, Tests, Protokolle und Messdaten

Halluzination und Gefälligkeit können zusammenwirken, wenn eine falsche Aussage unwidersprochen bleibt. Die Gegenmaßnahmen setzen deshalb an Auftrag, Prüfinstanz und Artefakten an. Keine Aussage gilt, weil ein Modell sie behauptet. Konformitätsmatrix und Testarchitektur prüfen die Implementierung; auch sie belegen nur die untersuchten Eigenschaften und keinen allgemeinen Korrektheitsbeweis.

Die Antworten der Tabelle sind Verfahrensregeln, und Verfahrensregeln führen Menschen und Modelle aus. Ohne weiteres Zutun urteilen nur die ausführbaren Prüfungen und der Beweisprüfer. Wie diese Prüfungen für FF1 aufgebaut sind, legt der folgende Abschnitt fest.

3.4 Testarchitektur für FF1

Die aktuelle Prüfkette wird lokal gestartet, verbindet aber lokale und entfernte Prüfungen. Sie ist der Sollzustand vor dem Öffnen oder Aktualisieren eines Pull

Requests. Historisch entstand diese Kette schrittweise.

Dieser Abschnitt beschreibt ausschließlich die Prüfung der Entwicklung, also die Nachweiseite von FF1. Ein bestandener Entwicklungstest sagt nichts über reale Netzpfade aus, und eine Pfadmessung ersetzt keinen Entwicklungstest. Die Pfaduntersuchung für FF2 verwendet die fertigen Programme, Sendemanifeste sowie Paketmitschnitte an Sender und Empfänger; ihre Ergebnisklassen legt bereits Tabelle 3.1 fest, das Pfad- und Auswertungsmodell folgt in Abschnitt 4.3, die Belegregeln und die Auswertung der Messläufe in Abschnitt 6.1. Der eigene Mitschnittprüfer wird dabei wiederverwendet; die Bewertungsregeln beider Untersuchungen bleiben getrennt.

3.4.1 Prüfmittel und ihre Reichweite

Zu den festen Prüfungen gehören Formatierung, statische Analyse, Dokumentationsbau sowie Unit- und Integrationstests. Drei weitere Prüfmittel tragen besondere Aufgaben. Ein Property-Test formuliert eine Eigenschaft, die für alle Eingaben einer Klasse gelten soll, und prüft sie an vielen maschinell erzeugten Fällen statt an ausgewählten Beispielen [35]. Ein Fuzz-Ziel ist eine Funktion, die eine beliebige Bytefolge entgegennimmt und damit die echten Verarbeitungspfade der Bibliothek aufruft, vom Zerlegen über das Serialisieren bis zum Aufbau ganzer Datagramme; der abdeckungsgeleitete Fuzzer erzeugt und verändert solche Eingaben fortlaufend und bevorzugt jene, die neue Programmpfade erreichen [36, 37]. Fuzzing findet Fehler, aber keine Fehlerfreiheit; es läuft als lokale Pflichtstufe und nicht in der kontinuierlichen Integration. Lean ist ein Beweisassistent: Regeln werden als Modell formuliert, Aussagen darüber als Theoreme bewiesen, und ein kleiner Kern prüft jeden Beweisschritt nach [38]. Die Lean-Prüfung baut diese Modelle und kontrolliert ihre Axiome. Bewiesen ist damit das handgeschriebene Modell mit seinen Annahmen, nicht der Rust-Quelltext; den gebauten Umfang beziffert Abschnitt 5.6.

Eine besondere Gefahr sind Fehler, die Implementierung und Test gemeinsam haben. Sender und Empfänger der Bibliothek verwenden getrennte Module für Serialisierung und Parsing, beruhen aber auf denselben projektspezifischen Formatannahmen, etwa der gemeinsamen Tabelle der `Kind`-Werte und der Längenkonstanten. Ein gemeinsamer Entwurfsfehler in diesen Annahmen kann deshalb jeden Test bestehen. Die Wire-Prüfung verringert diese Selbstbezüglichkeit: `tcpdump` zeichnet die tatsächlich gesendeten Bytes auf. Ein getrenntes Python-Programm dekodiert IPv4, UDP und die UDP-Optionen und berechnet die Prüfsummen. `tshark` bestätigt zusätzlich die IPv4- und UDP-Felder. Die im Projekt verwendete Version 4.6.4 besitzt jedoch keinen Dissektor für UDP-Optionen.

3.4.2 Prüfkette bis zur Freigabe

Das lokale Gate `pre-pr.sh` Skript fährt neun feste Test-Bahnen: Formatierung, statische Analyse, Dokumentationsbau, die Tests samt Property-Fällen, die Prüfung des eigenen Auswerters, das Lean-Tor sowie drei Bahnen auf einem separaten Linux-Zielsystem (lokale virtuelle Maschine, aka achim), das die Cross-Kompilierung, die Raw-Socket-Integration mit Root-Rechten und das tatsächlich gesendete Paketbild prüft. Dazu kommt je Fuzz-Ziel eine eigene Bahn; zusammen ergeben sich die 18 Prüfläufe aus Abbildung 3.3. Die kontinuierliche Integration (CI) wiederholt Formatierung, statische Analyse und Unit-Tests. Nach deren Erfolg baut sie die Lean-Modelle und stellt anschließend zwei getrennte Prüfanfragen: an Claude Code zu Codequalität und Sicherheit und an Codex zur RFC-9868-Semantik.

Der aktuelle Sollprozess ist in Abbildung 3.4 als Sequenzdiagramm dargestellt: Jeder Beteiligte erhält eine senkrechte Lebenslinie, die schmalen Balken zeigen seine aktiven Phasen, durchgezogene Pfeile sind Aufträge und gestrichelte Pfeile Rückmeldungen. Auch diese Darstellung bleibt bewusst auf die Kontrollpunkte und die Grenze ihrer technischen Erzwingung reduziert:

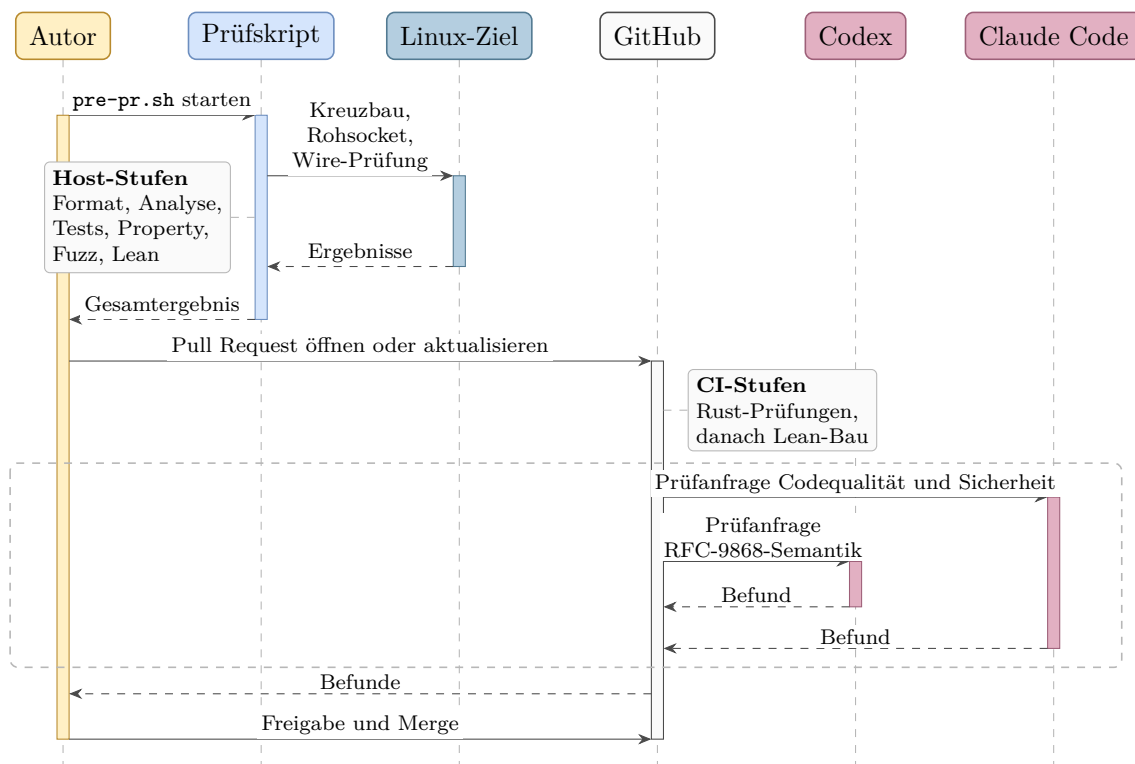


Abbildung 3.4: Prüfprozess im Sollzustand

Aus Abbildung 3.4 geht hervor, dass die Pfeilfolge nur die vorgesehene Reihenfolge vom Prüfskript über den Pull Request und die Rust/Lean-CI bis zur Freigabe durch den Autor zeigt; eine technische Merge-Bedingung ist sie nicht. Nach erfolgreicher

Rust/Lean-CI werden getrennte Zweitprüfungen angefragt.

Ob eine Prüfung tatsächlich stattfand, entscheidet allein ihre Protokollierung; deren Grenzen behandelt der folgende Abschnitt.

3.5 Reproduzierbarkeit und Grenzen

Aussagen über den Entstehungsprozess sind nur so belastbar wie ihre Protokollierung. Maßgeblich sind die vorhandenen Arbeitsprotokolle, also die Repository-Historie, die Prüf- und Kampagnenprotokolle sowie die Messdaten. Technische Ergebnisse sind reproduzierbar, soweit Quellstand, Werkzeuge, Eingaben und Ausgaben archiviert sind. Die genaue Folge nichtdeterministischer Modellausgaben ist es nicht.

Kapitel 4 verwendet die hier festgelegten Quellen- und Bewertungsregeln, um aus RFC 9868 das Anforderungsmodell für FF1 abzuleiten.

4. Analyse und Anforderungsmodell

Kapitel 3 legt die Quellenregeln fest. Dieses Kapitel wendet sie auf RFC 9868 an (Abschnitt 4.1), überführt den Anforderungskatalog in den FF1-Maßstab (Abschnitt 4.2) und bildet das FF2-Pfad- und Auswertungsmodell (Abschnitt 4.3). Die Technologieauswahl (Abschnitt 4.4) schließt die Analyse ab.

4.1 Anwendung der Extraktionsmethode

Die Auswertung wendet die in Abschnitt 3.2 festgelegte Methode auf den Primärtext an. Die 103 mit >> markierten Absätze dienen als Kontrollzahl; unmarkierte normative Festlegungen bleiben Teil des Maßstabs [29, 30]. Extrahiert wird satzgenau und nicht absatzweise, denn ein markierter Absatz kann mehrere Pflichten mit verschiedenen Adressaten bündeln. Jede extrahierte Aussage wird zu einem Prüfpunkt für den Anforderungsbestand des Implementierungs-Repositorys (Abschnitt 4.2).

Abbildung 4.1 zeigt, wie sich die 103 markierten Absätze auf die Abschnitte verteilen:

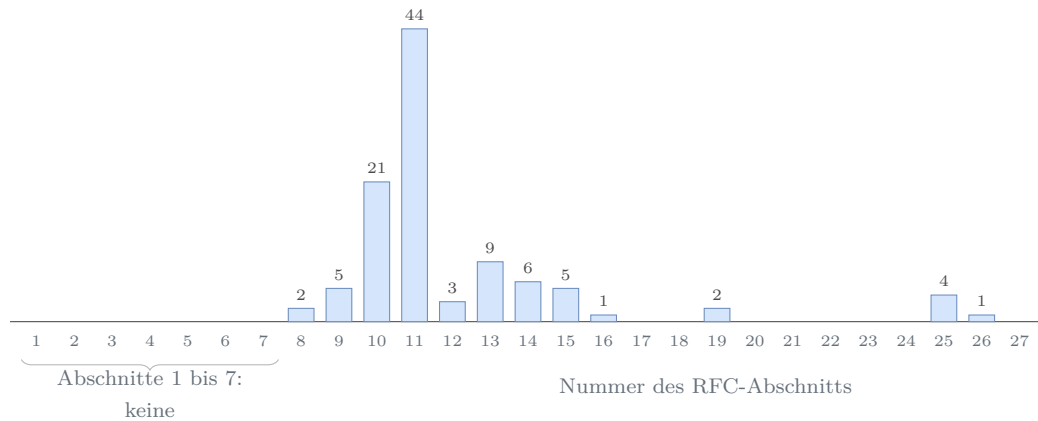


Abbildung 4.1: Verteilung der markierten normativen Absätze über die Abschnitte

Knapp zwei Drittel der markierten Absätze (65 von 103) liegen in den Abschnitten 10 und 11, also beim Optionsformat mit seinen Verarbeitungsregeln und bei den einzelnen Optionen; der Vorspann bis einschließlich Abschnitt 7 enthält keinen einzigen markierten Absatz, wohl aber eine Vorgabe des Projektumfangs: Abschnitt 7 verringert die Obergrenze der UDP `Length` um vorangehende Shim-Header. Die Implementierung verfolgt Shim-Ketten nicht und verarbeitet nur `Protocol 17`; das ist eine ausdrückliche Umfangsvorgabe dieser Arbeit, kein Verbot des RFC.

4.2 Anforderungskatalog und FF1-Maßstab

Das Implementierungs-Repository (Kapitel 5) führt in `docs/requirements.md` 49 funktionale und zwölf nichtfunktionale Anforderungen (FR-01 bis FR-50 ohne die entfallene Nummer FR-47, NFR-01 bis NFR-12) sowie eine Konformitätsmatrix; die Datei entstand vor dem ersten Funktionsschritt und wurde parallel zur Umsetzung fortgeschrieben. Die Matrix ist der Sache nach ein Arbeitsindex: Sie mischt markierte Normsätze, unmarkierte normative Festlegungen, Beschreibungen und Leitlinien. Die Prüfung aus Abschnitt 4.1 stuft deshalb jede Zeile satzgenau am Primärtext ein und überführt sie in die geprüfte Matrix in Anhang A. Tabelle 4.1 zeigt eine eigene übersetzte Auswahl; Normstufe und Abdeckung stehen im Wortlaut der Datei:

Tabelle 4.1: Auszug aus dem RFC-Arbeitsindex des Implementierungs-Repositorys

Fund- stelle	Normative Aussage	Normstufe	Abdeckung	FR/NFR
Sec. 8	Ausrichtungsbyte vor dem OCS ist null; sonst alle Optionen ignorieren und die Surplus Area still verwerfen	MUST	yes	FR-05
Sec. 9	OCS ungleich null, wenn die UDP-Prüfsumme ungleich null ist	MUST	yes	FR-21
Sec. 10	Optionen über 254 Byte nutzen das erweiterte Format; das kleinste Format ist empfohlen	MUST/ SHOULD	yes (send); local strict receive	FR-09, FR-14
Sec. 11.2	Höchstens sieben NOPs in Folge; übermäßige Folgen protokollieren und Ressourcen begrenzen	SHOULD	partial	FR-16, NFR-05
Sec. 12	UCMP, UENC und UEXP sind reservierte UNSAFE-Optionen	reserved	out	FR-39

Aus Tabelle 4.1 geht die Arbeitsteilung der Datei hervor: Die Konformitätsmatrix hält die Normseite fest, die FR- und NFR-Zeilen tragen die Umsetzung. Die Auswahl zeigt zugleich die Grenzen des Arbeitsindex. Die Zeile zu Abschnitt 11.2 bündelt die Ressourcengrenze aus Abschnitt 25.2 mit den NOP-Pflichten [2, Sec. 25.2].

Bewertet wird gegen den Maßstab von FF1: welche Anforderungen sich unter Linux und IPv4 im Benutzerraum „vollständig, teilweise oder nicht“ erfüllen lassen (Abschnitt 1.3). Die Bewertung nutzt drei Kategorien und eine Sonderklasse, die Tabelle 3.1 bereits verankert. **Vollständig** erfüllbar ist eine Anforderung, wenn sie unter den eigenen Vorgaben der Arbeit ohne Abstriche umsetzbar ist; diese Vorgaben sind die Plattform aus Abschnitt 1.3 (Linux, IPv4, Benutzerraum mit Raw Sockets) und die Protokollgrenze aus Abschnitt 4.1 (nur Protocol 17, keine

Shim-Ketten). **Teilweise** erfüllbar ist sie, wenn nur ein Teil erreichbar bleibt und die Ursache der Einschränkung belegt ist. **Nicht erfüllbar** ist sie, wenn dokumentiertes Betriebssystem- oder Schnittstellenverhalten die Umsetzung im Benutzerraum grundsätzlich ausschließt. **Nicht anwendbar** bleibt als Sonderklasse sichtbar, wird aber nicht bewertet; hierhin gehören Aussagen außerhalb des Umfangs und nicht gewählte MAY-Funktionen.

4.3 FF2-Pfad- und Auswertungsmodell

FF2 fragt, in welchem Umfang die Surplus Area auf realen Pfaden bis zur Gegenstelle erhalten bleibt (Abschnitt 1.3). Ausgewertet wird je Pfad, Richtung und Messzeitpunkt ein kontrolliertes Szenario aus Kontroll- und Optionsverkehr, das mehrere Wiederholungen oder bei FRAG mehrere Wire-Datagramme umfassen kann (Tabelle 3.1). Ein gültiger Lauf endet mit **erhalten**, **verändert**, **entfernt** oder **verworfen**; fehlerhafte Messungen erhalten den Status **Lauf ungültig** und gehen nicht in die Bewertung ein (Abschnitt 6.1). Die fünf **Pfadstufen** bilden getrennte Messklassen mit unterschiedlicher externer Netzbeteiligung:

1. **Loopback:** Sende- und Empfangspfad auf demselben Host; prüft Bibliothek und Messwerkzeuge ohne fremde Akteure.
2. **Lokale virtuelle Testumgebung:** Eine Ubuntu-VM unter VMware Fusion, mit Linux-Netzwerk-Namensräumen und **veth**.
3. **Tunnel:** Kapselung über einen realen Pfad (WireGuard); der äußere Pfad transportiert das innere Datagramm samt Surplus Area unbesehen zwischen den Tunnelendpunkten.
4. **Öffentliche Pfade in Europa:** Cloud-Endpunkte in Deutschland und Finnland sowie ein Mobilfunk-Zugangsnetz; reale Zugangs-, Transit- und Provider-Kanten.
5. **Interkontinentale Pfade:** Endpunkte in Nordamerika und Asien-Pazifik; lange Transitketten und die Netzstrukturen großer Cloud-Anbieter.

Vorversuche ohne beidseitige Mitschnitte und Ablaufprotokolle bleiben Piloten und werden nicht verallgemeinert (Abschnitt 6.1).

Die **Auswertung** verknüpft für jedes gerichtete Szenario Sendeversuch, Sender-manifest, Sender-PCAP, Empfänger-PCAP und Empfängerausgabe. Dadurch lassen sich Senderfehler, Veränderungen oder Verluste auf dem Pfad sowie Verwerfungen durch die Empfängerimplementierung unterscheiden. Jede Messung hat dabei genau zwei eigene Messpunkte: den Mitschnitt beim Sender und den Mitschnitt

beim Empfänger. Alles dazwischen, auf realen Pfaden etwa Zugangs-, Transit- und Provider-Kanten, hat keinen eigenen Messpunkt. Verlässt ein Datagramm den Sender nachweislich korrekt und kommt beim Empfänger verändert oder gar nicht an, ist die einzige belegbare Aussage: Die Abweichung ist irgendwo zwischen diesen zwei Punkten entstanden. Dieses Pfadstück zwischen den vorhandenen Messpunkten heißt **Attributionsintervall**, das Intervall, dem ein Befund zugeschrieben (attribuiert) werden darf. Einem einzelnen Gerät wird ein Befund nie zugeschrieben; dafür fehlen Messpunkte unmittelbar davor und dahinter.

Die Vorversuche dienten darüber hinaus dazu, mögliche Ursachen für beobachtete Veränderungen und Verluste zu formulieren und gezielte Folgemessungen zu planen. Da diese Hypothesen aus Messbeobachtungen entstanden, gehören die daraus abgeleiteten Mechanismenklassen zu den Ergebnissen dieser Arbeit; sie werden mit ihren Belegen in Kapitel 6 dargestellt.

Zwei Tabellen grenzen abschließend den Geltungsbereich ab: Tabelle 4.2 den Implementierungsumfang, Tabelle 4.3 die bewusst breitere Messinfrastruktur:

Tabelle 4.2: Implementierungsumfang

Merkmal	Festlegung	außerhalb des Umfangs
Betriebssystem	Linux	andere Betriebssysteme
Vermittlungsschicht	IPv4	IPv6
Ausführungsort	Benutzerraum mit Raw Sockets	Kernelimplementierungen
Artefakt	Bibliothek mit Mess- und Beispielprogrammen	eigenständiger Dienst; Protokollautomaten für REQ/RES und TIME
Optionsumfang	acht <i>must-support</i> -Optionen sowie generische Empfangsregeln für unbekannte SAFE- und UNSAFE-Arten	typisierte Erzeugung und Verarbeitung weiterer Arten wie TIME, AUTH, EXP, UCMP, UENC und UEXP

Tabelle 4.3: Messinfrastruktur

Baustein	Rolle in der Messung
Linux-Endpunkte: lokale virtuelle Maschine und Cloud-Instanzen in Europa, Nordamerika und Asien-Pazifik	Sende- und Empfangsendpunkte der Kampagnen; Mitschnitt mit <code>tcpdump</code>
Mobilfunk-Zugang über einen Hotspot	reale Zugangsnetzketten mit Hotspot-NAT; der Anteil eines möglichen CGNAT bleibt ohne zusätzlichen Messpunkt offen
macOS-Messpunkt <code>en0</code> (<code>access_bpf</code>)	zusätzlicher Beobachtungspunkt am Zugangsnetz; keine Implementierungsplattform
native IPv6-Kontrollen	Einordnung einzelner Pfadbefunde; außerhalb der Bibliothek
WireGuard-Tunnel	Kontrollpfad mit Kapselung (Pfadstufe 3)

macOS-Messpunkt, IPv6-Kontrollen und WireGuard-Tunnel erweitern nur die Messinfrastruktur; FF2 bewertet weiterhin Datagramme über IPv4 (Tabelle 4.3).

4.4 Technologieauswahl

Die Implementierung entsteht als einbindbare *Bibliothek* mit schmalen Mess- und Beispielprogrammen, nicht als eigenständiger Dienst und nicht als Kernelmodul. Diese Form folgt den Entwurfsprinzipien von RFC 9868, die nötigen Zustand und jede Antwort in der Anwendung oder einer in ihrem Auftrag arbeitenden Bibliothek verorten (Abschnitt 2.4.5) [2, Sec. 3 und 6], und eignet sich für die Messprogramme dieser Arbeit ebenso wie für spätere Anwendungen. Zu begründen sind die Sprache, der Zugang zur Surplus Area, die Messwerkzeuge sowie Fuzzing und formale Gegenproben.

4.4.1 Programmiersprache

Die Wahl der Programmiersprache folgt aus vier Bedingungen. Erstens braucht die Bibliothek im Benutzerraum Zugriff auf das vollständige IP-Datagramm über die durch UDP `Length` begrenzten Nutzdaten hinaus (Abschnitt 2.5); den gewählten Zugang begründet Abschnitt 4.4.2. Zweitens verlangt das Wire-Format bytegenaue Kontrolle: Der Serialisierer schreibt jedes Feld ausdrücklich in Netzwerkbyteordnung; native Strukturabbilder mit möglichem Padding dienen nicht als Wire-Format. Drittens verarbeitet der Parser Pakete, die jeder beliebige Absender im Netz erzeugt

haben kann; er liegt damit an der *Vertrauensgrenze* des Systems, an der eingehende Daten als potentiell bösartig gelten müssen, und muss auch fehlerhafte oder gezielt konstruierte Pakete robust verarbeiten. Viertens soll die Bibliothek ohne eigene Laufzeitumgebung und ohne automatische Speicherbereinigung (Garbage Collector) auskommen; das erleichtert das Einbetten und vermeidet GC-Pausen, macht das Zeitverhalten aber nicht allgemein vorhersagbar. Die ersten beiden Bedingungen folgen aus der Umsetzung im Benutzerraum, die letzten beiden sind Qualitätsziele dieser Arbeit.

Diese Bedingungen erfüllen *Systemprogrammiersprachen* wie C, C++, Zig und Rust: Sprachen mit hardwarenahem Zugriff auf Speicher und Betriebssystemschnittstellen, die ohne obligatorische Laufzeitumgebung zu nativem Maschinencode übersetzt werden. Sprachen mit Garbage Collector wie Go oder Java scheiden an der vierten Bedingung aus; das ist kein Eignungsurteil, sondern die Folge des bewusst gesetzten Qualitätsziels.

Alle vier Kandidaten bieten die nötige Kontrolle. Den Ausschlag gibt die Speichersicherheit an der Vertrauensgrenze. Sie schließt Zugriffe außerhalb eines Speicherbereichs (*Buffer Overflow*), auf freigegebenen Speicher (*Use-after-free*) und doppelte Freigaben (*Double Free*) aus. Ein *Memory Leak* gewährt dagegen keinen unzulässigen Zugriff und bleibt auch in Rust möglich. Miller berichtete 2019 auf Grundlage der seit 2004 gepflegten MSRC-Daten, dass rund 70 Prozent der jährlich von Microsoft mit CVE versehenen Schwachstellen Speicherfehler sind. Chromium nennt rund 70 Prozent unter 912 seit 2015 erfassten Stable-Channel-Fehlern hoher oder kritischer Schwere. Die NSA empfiehlt speichersichere Sprachen, darunter Rust [39–41].

Die entscheidungsrelevanten Eigenschaften fasst Tabelle 4.4 zusammen:

Tabelle 4.4: Vergleich der Sprachkandidaten

Kriterium	C	C++	Zig	Rust
Speichersicherheit	nein	nein	teilweise	ja in Safe Rust; unsafe vertragsabhängig
Schutzmechanismus	keiner	optional	Übersetzung, modusabhängige Laufzeit	Compiler, Laufzeit
Speicherverwaltung	manuell	manuell, RAII	Allokatoren	Ownership
Sprachfassung	ISO-Norm	ISO-Norm	vor 1.0	1.0 seit 2015

Rust verbindet als einziger Kandidat der Tabelle Safe-Rust-Garantien mit einer Speicherverwaltung ohne Garbage Collector; Bereichsprüfungen lösen zur Laufzeit einen definierten *panic* statt undefinierten Verhaltens aus [42, 43], während der

C-Standard bei Pufferüberläufen undefiniertes Verhalten kennt [44]. Diese Garantien setzen korrekte Verträge an jeder `unsafe`-Grenze voraus. Statische Werkzeuge können Quelltext ohne ausgeführten Programmpfad prüfen; nur dynamische Prüfungen und Fuzzing bleiben auf erreichte Pfade beschränkt [41, 45]. Zig prüft auch zur Übersetzungszeit; seine Laufzeitprüfungen hängen vom Build-Modus ab, und die Sprache hat noch keine Fassung 1.0 erreicht [46]. Die Migration von Bun von Zig nach Rust bildet dazu den Referenzfall aus Abschnitt 3.1 [26]. Für die Vertrauensgrenze dieser Bibliothek fiel die Wahl deshalb auf Rust.

Drei Rust-Mittel tragen die Implementierung. Der Parser liest die Surplus Area ohne Kopie als geborgte Byte-Ausschnitte, die ihren Empfangspuffer nicht überdauern können [43]. `enum` und `Result` bilden Optionsarten und Fehler vollständig ab. Wenige `unsafe`¹-Blöcke kapseln die Systemgrenzen; ihre Verträge prüft Kapitel 5. Externe Parser-Bibliotheken entfallen, weil die Mechanik von RFC 9868 selbst Untersuchungsgegenstand ist.

Safe Rust verhindert keine Protokollfehler. Ein Um-eins- und Offsetfehler bei ungeradem Beginn der Surplus Area überstand 23 selbstgeschriebene Tests und fiel erst im Pull-Request auf; er verletzte die Protokolllogik, nicht die Speichersicherheit. Danach erhielt jeder Schritt mit neuer Parsefläche Property-Tests und ein Fuzz-Ziel (Kapitel 5).

4.4.2 Zugang zur Surplus Area

Ein gewöhnlicher UDP-Socket kann die Surplus Area weder füllen noch auslesen (Abschnitt 2.5). Tabelle 4.5 vergleicht die verfügbaren Linux-Wege zum vollständigen Datagramm.

Gewählt wurden Raw Sockets, weil sie ohne eigenes Netzgerät und ohne Kernelmodul Zugriff auf das vollständige IP-Datagramm geben. Als Betriebssystem trägt Linux diese Entscheidung: Dort ist der Weg dokumentiert und im quelloffenen Kernel nachprüfbar, und dieselbe Plattform stellt alle Endpunkte der Kampagnen (Abschnitt 4.3). Der Preis ist, dass die Bibliothek IPv4-Kopf, Längen, UDP-Prüfsumme und Optionen selbst prüft. Beim Senden liefert sie mit `IP_HDRINCL` den IPv4-Kopf; der Kernel ergänzt einzelne Felder, fragmentiert den Sendepuffer aber nicht, sodass jedes Optionsdatagramm in die MTU passen muss [17]. Diese Folgen bestimmen den Entwurf in Kapitel 5.

¹ Das Rust-Schlüsselwort `unsafe` ist von der UNSAFE-Optionsklasse aus Abschnitt 2.4.5 zu unterscheiden.

Tabelle 4.5: Entscheidungsmatrix für den Zugang zur Surplus Area unter Linux

Weg	Sicht	Zusätzlicher Unterbau	Bewertung
Standard-UDP-Socket	UDP-Nutzdaten bis UDP Length	keiner	Surplus Area nicht zugänglich; ausgeschlossen [2, Sec. 18]
Packet Socket mit AF_PACKET/SOCK_RAW	vollständiger Rahmen der Verbindungsschicht	CAP_NET_RAW, Schnittstellen- und Link-Layer-Behandlung	vollständiger Rahmen, aber unnötige Bindung an die Verbindungsschicht [47]
TUN/TAP	IP-Pakete bei TUN, Ethernet-Rahmen bei TAP	virtuelles Netzgerät, Konfiguration und Routing	verändert die Topologie und ist mehr als eine einbindbare Bibliothek [48]
AF_XDP	Rahmen aus einer Empfangswarteschlange	XDP-Programm, UMEM und Warteschlangenbindung	leistungsfähig, aber zusätzlicher Kernel- und Speicheraufbau [49]
Kernelmodul	Verarbeitung im Kernel	passender Kernelbau, Laden und privilegierter Kernelraum	voller Zugriff, aber versionsgebunden und keine gewöhnliche Benutzerraumbibliothek [50]
Raw Sockets: Senden mit IP_HDRINCL, Empfang mit IPPROTO_UDP	vollständiges IP-Datagramm	CAP_NET_RAW; eigene IPv4-, UDP- und Optionsverarbeitung	gewählt: kleinster Unterbau mit der nötigen Sicht [17]

4.4.3 Mitschnitt und Auswertung

Die Kampagnenskripte zeichnen den Verkehr mit `tcpdump` in archivierbaren PCAP-Beweisdateien auf. Der eigene Prüfer aus Abschnitt 3.4 rechnet OCS und Optionstruktur aus den Bytes nach, weil Wireshark in der verwendeten Version 4.6 die Surplus Area nicht dekodiert (Abschnitt 2.5); `tshark` prüft unabhängig davon die IPv4- und UDP-Felder. Aufzeichnen, Deuten und Gegenprüfen liegen damit bei drei verschiedenen Werkzeugen.

4.4.4 Fuzzing und formale Gegenprobe

Die vierte Entscheidung stellt der Testsuite zwei Gegenproben zur Seite, die auf das Gefahrenmodell der KI-gestützten Entwicklung antworten (Abschnitt 3.3). Das über `cargo-fuzz` eingebundene libFuzzer-Fuzzing mutiert Eingaben des echten Parsercodes abdeckungsgeleitet und sucht Abstürze und verletzte Invarianten [37]. Lean beweist ausgewählte Eigenschaften handgeschriebener Modelle unter den Voraussetzungen der jeweiligen Theoreme; den Begriff führt Abschnitt 3.4.1 ein, die gebauten Ziele und den Beweisumfang beziffert Abschnitt 5.6. Fuzzing beweist keine Fehlerfreiheit, und ein Lean-Beweis gilt nicht automatisch für den Rust-Code, der das Modell umsetzt. Dass daneben auch die Norm selbst Prüfgegenstand bleiben muss, zeigte die Schlussphase der Implementierung: Ein nachträglicher RFC-Abgleich fand trotz

grüner Prüfkette noch Konformitätslücken und wurde als eigener Korrekturschritt umgesetzt (Kapitel 5).

5. Entwurf und Implementierung

Die Referenzimplementierung folgt den Vorgaben aus Kapitel 4. Der Weg geht vom Entstehungsprozess (Abschnitt 5.1) über die Komponenten (Abschnitt 5.2) zu Sendeweg (Abschnitt 5.3), Empfangsweg (Abschnitt 5.4) und Optionsverarbeitung (Abschnitt 5.5), danach zu Testarchitektur, Betriebsgrenzen und bewussten Grenzen (Abschnitt 5.6, Abschnitt 5.7, Abschnitt 5.8). Vorversuchsbefunde erscheinen nur, wenn sie den Bau begründen. Die Entscheidungen aus Abschnitt 4.4 und die Mechanik aus Kapitel 2 werden angewandt.

5.1 Vorgehen der Implementierung

Die Umsetzung begann mit dem Plan. Der erste Commit des Repositorys legte den vollständigen Projektplan an, achtzehn Schrittdateien mit je Ziel, Anforderungen, Plan, Aufgaben und Abschlusskriterien, dazu die Modulgerüste.¹ Der Plan bezeichnet sich selbst als schrittweise abzuarbeitenden Masterplan mit einem Menschen in der Schleife; der Anforderungskatalog stand damit vor dem ersten Funktionsschritt.

Ab Schritt 1 lief der Kernzyklus aus Abbildung 3.3: ein eigener Arbeitszweig, die Umsetzung mit Tests im selben Sammelcommit, das lokale Gate mit am Ende 18 Prüfläufen, der Pull-Request mit der kleineren CI-Kette und zwei angeforderten KI-Prüfungen, zuletzt die menschliche Durchsicht des Diffs und ein freigegebener Sammelcommit auf dem Hauptzweig; die Schritte 14, 15 und 17 teilten sich einen solchen Commit. Die Schritte 0 und 0.5 bildeten die frühe Ausnahme und verteilten sich auf mehrere Commits und Pull-Requests. Die Regeln des reifen Zyklus stehen versioniert im Repository selbst, einschließlich des Grundsatzes, dass jede Änderung vor der Übernahme von einem Menschen gelesen und verantwortet wird.

Der Plan blieb dabei beweglich: Ein Machbarkeits-Vorversuch (Schritt 0.5, Abschnitt 5.4) und die unabhängige Wire-Prüfung (Schritt 10.5, Abschnitt 5.6) wurden eingeschoben, Schritt 9 ging in Schritt 8 auf, und die zunächst mitgebaute IPv6-Unterstützung wurde mit Schritt 16 samt der Katalogzeile FR-47 wieder aus dem Umfang entfernt (Abschnitt 5.8). Den Abschluss bildete kein Funktionsschritt, sondern eine Prüfung: Schritt 18 hielt die Befunde eines nachträglichen RFC-Abgleichs als eigene Schrittdatei fest und arbeitete sie gesammelt ab, darunter die Behandlung unbekannter UNSAFE-Optionen vor einem FRAG und den vollständigen Statusab-

¹ Commit `3c17252` vom 30.05.2026; die Detailpläne, der Anforderungskatalog (Abschnitt 4.2) und die Architektur- und Formatreferenzen folgten am selben Tag mit Commit `36cc465`.

gleich des Anforderungskatalogs.²

Dass diese Schleifen nicht Förmerei waren, zeigen belegte Fänge: Die Prüfung gegen den RFC-Primärtext fand vier Fehler in mehreren Vertragsdokumenten des Repositorys, darunter eine falsch wiedergegebene Ausrichtungsregel.³ Die Zweitprüfung im Pull-Request fing den Um-eins-Fehler der Surplus-Ortung, den 23 selbstgeschriebene Tests übersehen hatten (Abschnitt 5.6). Der Schlussaudit deckte trotz grüner Prüfkette weitere Konformitätslücken auf, darunter die Verarbeitung einer unbekannten UNSAFE-Option vor einem FRAG; Schritt 18 behob sie gesammelt. Ebenso klar sind die Grenzen der Belegbarkeit: Die Sammelcommits verschmelzen die innere Reihenfolge eines Schritts, die KI-Prüfungen waren angefordert, aber nicht erzwungen, und für das Lean-Gate ist kein Fang eines echten Produktfehlers belegt; es sicherte den Modellbestand (Abschnitt 5.6). Mit diesem Vorgehen im Rücken zeigt der Rest des Kapitels den fertigen Bau.

5.2 Gesamtarchitektur

Den Plattform- und Funktionsumfang legt Abschnitt 1.4 fest; Abschnitt 4.4 begründet die vier technischen Entscheidungen, die hier angewandt werden. Innerhalb dieses Rahmens entsteht das Artefakt aus Tabelle 4.2: eine Bibliothek, die die Surplus Area baut, liest und prüft, samt zwei schmalen Messprogrammen für die Kampagnen.

Die Bibliothek⁴ besteht aus acht Modulen und zwei Messprogrammen, zusammen rund 8.600 physische Rust-Zeilen⁵: Sechs Module tragen je einen fachlichen Baustein, vom Wire-Format bis zur öffentlichen Schnittstelle, zwei liegen als Querschnitt unter allen. Komponente meint in diesem Kapitel einen solchen Baustein zusammen mit seiner Quelleinheit: Der Modulname bestimmt zugleich die Quelleinheit unter `src/`, ein gleichnamiges Verzeichnis oder eine gleichnamige Rust-Datei.

Die Module, die Programme und ihre Importbeziehungen zeigt Abbildung 5.1:

Aus Abbildung 5.1 geht zweierlei hervor. Erstens zeigen die Kanten ausgewählte tatsächliche Importe und bilden keine strikte Schichtung; so nutzt die Socket-Schicht für eine Längendiagnose einen Helfer der Empfangspipeline. Zweitens liegen die Netz-Systemaufrufe innerhalb der Bibliothek allein in `socket`, und mit ihnen die beiden

² Commit 4c82ab0 vom 02.08.2026 mit 53 geänderten Dateien; die Schrittdatei ist `docs/plan/steps/18-rfc9868-audit-remediation.md`.

³ Commit f85815f mit vier geänderten Vertragsdateien (`CLAUDE.md`, `docs/architecture.md`, `docs/requirements.md`, `docs/wire-format.md`).

⁴ Implementierungs-Repository `udp-transport-options`, Commit f847895; alle Datei- und Zeilenangaben dieses Kapitels beziehen sich auf diesen Stand.

⁵ Gezählt mit `wc -l` über alle `.rs`-Dateien der neun Quelleinheiten unter `src/` (sieben Verzeichnisse sowie die Einzeldateien `error.rs` und `model.rs`), ohne die 40 Zeilen von `src/lib.rs`: genau 8.606 Zeilen.

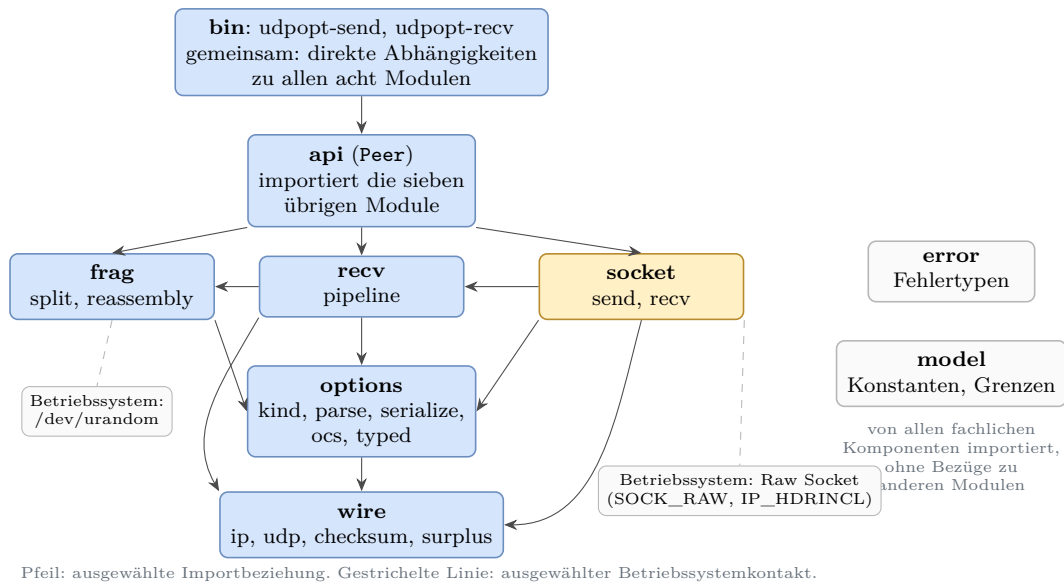


Abbildung 5.1: Komponenten der Bibliothek und ihre Abhängigkeiten

`unsafe`-Blöcke in `src/` (Abschnitt 4.4.1), gekapselt hinter sicheren Schnittstellen. Daneben liest `frag` für die FRAG-Identifikation das Zufallsgerät `/dev/urandom`, `api` und `socket` fragen für Fristen die monotone Uhr ab, und die beiden Messprogramme kommen mit Konsolenausgabe hinzu; `udpopt-send` kann zusätzlich eine JSONL-Manifestdatei fortschreiben. Daraus folgt eine Eigenschaft, die die Testarchitektur in Abschnitt 5.6 trägt: Die reinen Protokollpfade erhalten Zeitpunkte und Identifikation als Parameter, jede Protokollentscheidung ist damit ohne Netz und ohne erhöhte Rechte prüfbar.

Unten liegt `wire`, die reine Bytesicht: IPv4-Kopf, UDP-Kopf, Prüfsummenarithmetik und die Ortung der Surplus Area. Hier wird der IPv4-Kopf geprüft und nur `Protocol 17` angenommen (Umfangsvorgabe aus Abschnitt 4.1), und hier liegt die Ortungsregel, nach der Optionen an jedem gültigen UDP `Length`-Offset beginnen dürfen [2, Sec. 8]: Aus Gesamtlänge, Kopflänge und UDP `Length` bestimmt `wire` Beginn, Pad-Bedarf und Länge der Surplus Area, ohne ein einziges Optionsbyte zu deuten.

Darüber teilen sich drei Komponenten die Optionslogik. `options` trägt den Optionskörper: Optionsverzeichnis, Parser, Serialisierer, die typisierten Optionswerte und die OCS-Rechenregeln (FR-19, FR-21). `recv` enthält mit der Pipeline die Empfangsordnung, die für jedes an sie übergebene Datagramm über Annahme oder Verwerfen entscheidet (FR-20, FR-38). `frag` zerlegt große Nachrichten in Fragmente und setzt sie unter begrenztem Speicher wieder zusammen (FR-34, NFR-06). Die acht *must-support*-Optionen ([2, Sec. 10]) verteilen sich über diese drei Komponenten; die Vertiefung folgt in Abschnitt 5.4 bis Abschnitt 5.7.

Oben bündelt `api` alles im Typ `Peer`, der Gegenstelle als Bibliothekstyp: Eine Anwendung konfiguriert einen `Peer` und erhält Senden und Empfangen als je einen Aufruf, ohne Socket-Verwaltung, Prüfreihefolge oder Optionsbau selbst zu übernehmen; Aufruferpflicht bleiben die erhöhten Rechte für Raw Sockets, die Wahl der Empfangsstrengigkeit und der anwendungsseitige Protokollzustand, etwa die Herkunft eines RES-Tokens; Rechte und Strengigkeit behandelt Abschnitt 5.7. Die zwei Programme `udpopt-send` und `udpopt-recv` sind schmale Messaufsätze für die Kampagnen. Quer zu allem liegen `error` mit den Fehlertypen und `model` mit den Protokollkonstanten sowie den Reassemblierungs- und Schutzgrenzen: von allen fachlichen Komponenten importiert, selbst ohne Abhängigkeiten zu anderen Bibliotheksmodulen. Wie ein Datagramm dieses Gefüge von oben nach unten durchläuft, zeigt der Sendepfad.

5.3 Sendepfad

Beim Senden baut die Bibliothek aus Nutzdaten und gewünschten Optionen ein vollständiges IPv4-Datagramm und übergibt es dem Raw Socket; den Rahmen setzt Abschnitt 1.4, beschrieben wird durchgehend der IPv4-Weg unter Linux. Als roter Faden dient von hier an ein festes Beispieldatagramm, das die Darstellung bis in die Evaluation begleitet: drei Nutzdatenbytes `odd` und genau eine REQ-Option mit dem vier Byte langen Token `c0ffee01`.⁶ Unter den vorbereiteten Prüfscenarien der Bibliothek ist es das kleinste, das alle vier Bauelemente der Surplus Area zugleich enthält: das Pad-Byte, das OCS, eine TLV-Option und die Nullfüllung nach EOL.

Wie aus Nutzdaten und Optionsliste das fertige Datagramm des Beispiels entsteht, zeigt Abbildung 5.2 in vier Schritten:

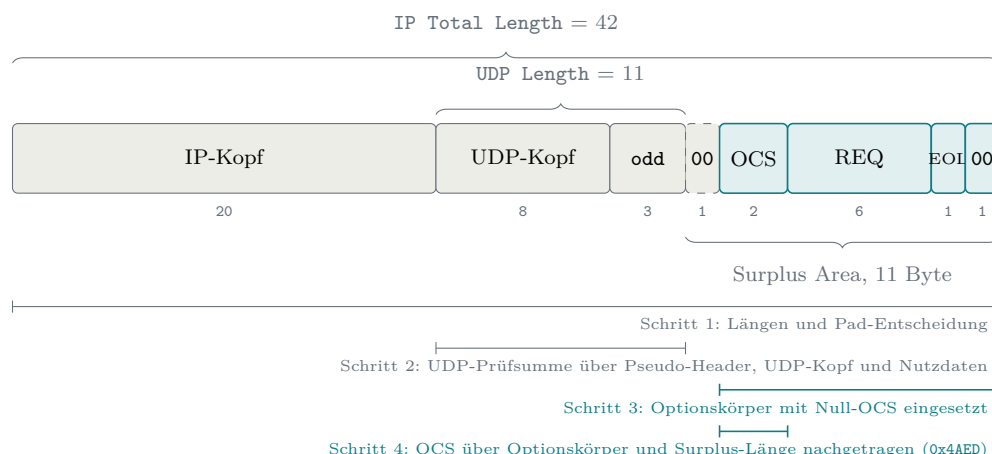


Abbildung 5.2: Entstehung des Beispieldatagramms in vier Schritten

⁶ Prüfscenario `pad-odd` in `examples/wire_probe.rs` (Zeilen 248 bis 258); die erwarteten Bytes hält unabhängig davon das Prüfprogramm `scripts/wire-check.py` fest (Zeilen 248 bis 250 und 272).

Aus Abbildung 5.2 geht hervor, dass die Schrittfolge nicht beliebig ist: Die UDP-Prüfsumme in Schritt 2 deckt Pseudo-Header, UDP-Kopf und Nutzdaten (Abschnitt 2.2.1), das OCS in Schritt 4 den Optionskörper samt Surplus-Länge (Abschnitt 2.4.3); die beiden Bereiche ergänzen sich ohne Überschneidung, und das OCS folgt zuletzt, weil in seine Summe der fertige Optionskörper und die endgültige Surplus-Länge eingehen. Schritt 1, die Längenrechnung, zeigt der folgende Ausschnitt aus der Aufbaufunktion⁷, gekürzt um die Überlauf- und Bereichsprüfungen des Originals:

```
let udp_len = udp::HEADER_LEN + user_data.len();
let natural_start = usize::from(length::UDP_HEADER) + 20 + user_data.len();
let needs_pad = !options_body.is_empty() && natural_start % 2 == 1;
let surplus_len = if options_body.is_empty() {
    0
} else {
    usize::from(needs_pad) + options_body.len()
};
let total_len = 20 + udp_len + surplus_len;
```

Für das Beispiel ergibt die Rechnung: **UDP Length** $8 + 3 = 11$, und die Surplus Area beginnt hinter IP-Kopf, UDP-Kopf und Nutzdaten bei $\text{Byte } 20 + 11 = 31$. Dieser Offset ist ungerade, nach der Paritätsregel aus Abschnitt 2.4.2 fällt also genau ein Pad-Byte an. Der Optionskörper aus dem Serialisierer ist zehn Byte lang (zwei Byte OCS-Platzhalter, sechs Byte REQ, EOL und ein Füllbyte auf gerade Länge; Abschnitt 5.5), die Surplus Area damit $1 + 10 = 11$ Byte und die **IP Total Length** $20 + 11 + 11 = 42$ Byte. Sichtbar wird hier zugleich die Senderseite der Offset-Regel aus [2, Sec. 8]: Optionen dürfen an jedem gültigen **UDP Length**-Offset beginnen, und die Bibliothek wählt stets den natürlichen Start unmittelbar hinter den Nutzdaten.

Geschrieben wird anschließend in einen mit Nullen vorbelegten Puffer der Gesamtlänge: der IP-Kopf, der UDP-Kopf mit der bereits berechneten UDP-Prüfsumme, die Nutzdaten und der Optionskörper an seiner Zielposition. Das Pad-Byte wird dabei nie eigens geschrieben: Es bleibt die Null der Puffervorbelegung und erfüllt so die Pflicht aus [2, Sec. 8], nach der Ausrichtungsbytes vor dem OCS null sein müssen (FR-05). Zum Schluss rechnet die OCS-Routine der Komponente `options` über den eingesetzten Körper und trägt das Ergebnis in den Platzhalter ein.

Bei den Prüfsummen begegnen sich zwei Nullregeln aus Abschnitt 2.2.1 und Abschnitt 2.4.3. Die UDP-Prüfsumme kennzeichnet mit einer gespeicherten Null den Verzicht auf eine Prüfsumme; eine errechnete Null wird deshalb als das im Einerkomplement gleichwertige `0xFFFF` übertragen [1]. Für das OCS verlangt der RFC einen Wert ungleich null, sobald die UDP-Prüfsumme ungleich null ist; die Null bleibt der Kennzeichnung eines unbenutzten OCS vorbehalten und ist nur bei

⁷ `assemble_datagram` in `src/socket/send.rs` (Zeilen 25 bis 92); der Ausschnitt gibt die Zeilen 38 bis 56 wieder.

UDP-Prüfsumme null zulässig [2, Sec. 9]. Eine ausdrückliche Senderegel für eine errechnete OCS-Null enthält RFC 9868 nicht; weil der Nullwert die unbenutzte Form kennzeichnet, bleibt für ein verwendetes OCS im Einerkomplement aber nur die Abbildung auf 0xFFFF. Beide Abbildungen stehen an je genau einer Stelle: für die UDP-Prüfsumme in `wire`, für das OCS in `options`.⁸

Die OCS-Rechnung selbst folgt der Regel aus Abschnitt 2.4.3: In die Einerkomplementsumme geht neben dem Optionskörper die Länge der gesamten Surplus Area als 16-Bit-Summand ein, obwohl sie in keinem Feld des Pakets steht; der Sender bestimmt sie beim Bau aus Pad-Bedarf und Optionskörperlänge, der Empfänger leitet sie aus IP- und UDP-Länge ab, und der Entwurfsgrund, die Kompensation fehlerhaft rechnender Zwischensysteme, ist ebenfalls dort beschrieben [2, Sec. 9]. Am Beispiel: Die 16-Bit-Wörter des Optionskörpers summieren sich mit dem Platzhalter null zu 0xB507, der Längensummand 11 hebt die Summe auf 0xB512, und invertiert entsteht das OCS 0x4AED. Die Probe des Empfängers geht auf: 0xB507, 0x4AED und 0x000B summieren sich zu 0xFFFF.

Zwischen Puffer und Leitung steht der Kernel, denn bei `IP_HDRINCL` liefert die Bibliothek den IP-Kopf selbst an. Nach der Raw-Socket-Dokumentation füllt der Kernel dabei die `IP Total Length` und die IP-Kopfprüfsumme stets selbst aus [17]; der eigene Mitschnitt kann diese allgemeine Aussage nur für die beobachteten Felder bestätigen. Die Gegenseite der Arbeitsteilung zeigt er dafür deutlich: UDP-Kopf und Surplus Area bleiben unangetastet. Im handgebauten Prüfzenario `cksum0-ocs0`, das das UDP-Prüfsummenfeld auf null lässt und den OCS-Platzhalter nie füllt, lag das Nullfeld unverändert auf der Leitung; die von RFC 9868 zugelassene Kombination aus ungenutzter UDP-Prüfsumme und ungenutztem OCS ist aus dem Benutzerraum also exakt konstruierbar [2, Sec. 9].

Derselbe Sendeweg setzt zugleich eine harte Größengrenze: Der `IP_HDRINCL`-Pfad fragmentiert nicht auf IP-Ebene, ein Datagramm oberhalb der MTU weist der Kernel mit dem Fehlercode `EMSGSIZE` („Nachricht zu lang“) ab. Die in Abschnitt 2.1 beschriebene IP-Fragmentierung steht dem Raw-Pfad damit nicht zur Verfügung; das ist ein Befund dieses Sendewegs, keine Vorgabe des RFC. Seine eigene Fragmentierungsoption begründet der RFC unabhängig davon: `FRAG` überträgt Nachrichten oberhalb der Empfangsgrenze `EMTU_R` und kopiert die UDP-Ports in jedes Fragment, sodass die Fragmente zustandslose NAT-Geräte durchqueren können [2, Sec. 11.4]; die Vertiefung folgt in Abschnitt 5.5. Die Bibliothek zieht im Sendepfad die Konsequenz: `Peer::send` prüft gegen eine konfigurierbare Obergrenze der Datagrammlänge (Voreinstellung 1500 Byte) und zerlegt größere Nachrichten bei aktiviertem Modus in `FRAG`-Fragmente, sofern die MRDS- und Segmentgrenzen dies

⁸ `src/wire/udp.rs` (Zeilen 85 bis 88) und `src/options/ocs.rs` (Zeilen 55 bis 58).

zulassen; andernfalls endet der Aufruf mit `SendError::Split`. `SendOptions` wählt die unterstützten Optionen je Sendung; `max_datagram_len` bildet die maximale Fragment- oder Datagrammlänge ab. Es fehlt aber eine Kennzeichnung, ob eine Option für Fragmente oder für die Nachricht vor der Fragmentierung gilt, sowie eine vom Aufrufer gewählte Mindestlänge mit EOL und Nullfüllung. Die tiefe Funktion `build_outgoing_datagrams` nimmt `SendConfig` je Aufruf an. Bei `Peer::send` ist diese Konfiguration dagegen beim Binden gespeichert; der Aufruf erhält nur Nutzdaten und `SendOptions`.⁹ Damit ist die Sende-API-Form aus Abschnitt 15 teilweise abgebildet; die anschließende Erwartung paketweiser Aktivierung erfüllt nur die tiefe, nicht die hohe Schnittstelle vollständig. In den Sendekampagnen kehrt `EMSGSIZE` zudem als erwartete Betriebsgrenze oberhalb von 1500 Byte wieder (Abschnitt 6.3).

Am Ende des Pfads steht der Systemaufruf. Die Socket-Schicht öffnet den Raw Socket mit den Crates `socket2` und `libc` [51, 52] und schaltet `IP_HDRINCL` per `setsockopt` ein; dieser Aufruf ist der erste der beiden in Abschnitt 5.2 angekündigten `unsafe`-Blöcke in `src/` (`src/socket/send.rs`, Zeile 131). Er bleibt hinter einer sicheren Hilfsfunktion gekapselt, und ein `SAFETY`-Kommentar begründet, warum Zeiger und Längen des Aufrufs gültig sind (Abschnitt 4.4.1). Auf der Gegenseite kehrt sich die Reihenfolge um, und die Verantwortung wächst: Der Empfänger kann sich auf keine dieser Zusicherungen verlassen und prüft alles selbst.

5.4 Empfangspfad

Auf der Empfangsseite erhält der Raw Socket seine Kopie des Datagramms, bevor der UDP-Pfad des Kernels Länge, Prüfsumme und Portzuordnung geprüft hat (Abschnitt 2.5, Abbildung 2.10). Aus dieser Lage folgt die Eigenverantwortung für den UDP-Anteil: Die im Raw-Pfad ausgelassenen Kernelprüfungen und die Portauswahl muss die Bibliothek selbst leisten, bevor sie auch nur ein Byte der Surplus Area deutet.

Wie wörtlich das gemeint ist, zeigte ein Vorversuch am Beginn der Implementierung: eine Messreihe mit 97 Fällen über ein virtuelles Netzschnittstellenpaar (MTU 1500) zwischen zwei Netznamensräumen, davon 92 aus einem Kreuzprodukt von Nutzdaten- und Surplus-Größen um die MTU-Grenze, ein Fall weit oberhalb der MTU und vier gezielte Kopfanomalien.¹⁰ 69 Fälle erreichten den Raw-Empfang, die übrigen 28 scheiterten erwartungsgemäß schon beim Senden mit `EMSGSIZE` (Ab-

⁹ `SendOptions` in `src/api/mod.rs` (Zeilen 58 bis 63), `SendConfig` mit `max_datagram_len` (Zeile 33 sowie Zeilen 110 bis 137), `Peer::send` (ab Zeile 298); die Zerlegung übernimmt `build_outgoing_datagrams` (ab Zeile 362).

¹⁰ Fallgenerator `cases()` in `examples/spike_client.rs` und `examples/spike_server.rs` (beide Seiten bauen dieselbe Liste); Protokoll des Vorversuchs in `docs/plan/steps/00b-spike.md`.

schnitt 5.3). Das zentrale Ergebnis für den Empfang sind die vier Anomalien: ein Datagramm mit UDP-Prüfsumme null, eines mit absichtlich falscher Prüfsumme, eine UDP `Length` von 500 bei 48 verfügbaren Bytes und eine UDP `Length` von 4, kürzer als der UDP-Kopf selbst. Alle vier erreichten den Raw Socket; der Versuch prüfte dabei nur die Ankunft und protokollierte Längenfelder und Surplus-Bytes, ein Byte-für-Byte-Vergleich mit dem Sendepuffer gehörte nicht zum Aufbau. Daneben bestätigte die Messreihe die Arbeitsteilung aus Abschnitt 5.3: Von den 69 empfangenen Fällen trugen 42 im Sendepuffer ein absichtlich falsches `Total Length`-Feld, und der Kernel schrieb jeden dieser Werte vor der Übertragung auf die tatsächliche Pufferlänge um. Aus dem Vorversuch folgt die Grundentscheidung des Empfangspfads: Die Bibliothek prüft Längenkonsistenz, UDP-Prüfsumme, OCS und erst dann die Optionen selbst, in der Empfangsordnung des RFC [2, Sec. 10 und 14]; die vorgelagerte Prüfung des IP-Kopfs kommt als eigene zusätzliche Kontrolle dieser Bibliothek hinzu.

Die vier Prüfungen bis zur Surplus-Ortung zeigt Listing 5.1 in gekürzter Form; ausgelassene Fehlerdetails sind mit `..` markiert:

```

1 let (ip, udp_at) = IpRepr::parse(ip_datagram)?;
2 let udp = UdpHeader::parse(&datagram[udp_at..])?;
3 if usize::from(udp.length) > ip.transport_payload_len() {
4     return Err(RecvError::UdpLengthExceedsIpPayload { .. });
5 }
6 if udp.checksum != 0 {
7     let expected = UdpHeader { checksum: 0, ..udp }
8         .compute_checksum(&ip, user_data);
9     if expected != udp.checksum {
10         return Err(RecvError::UdpChecksumMismatch { .. });
11     }
12 }
13 let Some(layout) = locate_surplus(&ip, &udp) else {
14     return deliver_without_options(false, OcsStatus::Absent);
15 };

```

Listing 5.1: Prüfreihefolge der Empfangspipeline bis zur Surplus-Ortung (gekürzt)

Zeile für Zeile entspricht das der Vorgabe: `IpRepr::parse` nimmt nur Datagramme mit `Protocol 17` an (Umfangsvorgabe) und weist dabei auch eine falsche IP-Kopfprüfsumme ab; `UdpHeader::parse` verlangt eine UDP `Length` von mindestens 8; der Längenvergleich verwirft Datagramme, deren UDP `Length` über die IP-Nutzlast hinausweist, wie im Anomaliefall 500 bei 48; und die UDP-Prüfsumme wird genau dann geprüft, wenn ihr Feld nicht null ist, denn eine gespeicherte Null ist keine Prüfsumme (Abschnitt 5.3). Jeder Fehler bis hierher verwirft das ganze Datagramm als Fehlerwert der Pipeline. Damit ist die in Abschnitt 2.5 angekündigte Reihenfolge (Länge und Längenkonsistenz, dann Prüfsumme, erst danach Surplus-Ortung und OCS) im Quelltext eingelöst.¹¹

¹¹ Einstieg `process_datagram` in `src/recv/pipeline.rs` (Zeilen 201 bis 203); die gezeigte Prüfreihefolge steht in den Zeilen 211 bis 266.

Hinter dieser Eingangsprüfung verzweigt die Pipeline in ihre Ergebnisarten; den Lauf des reinen Verarbeitungskerns `process_datagram`, gekürzt um die Sonderpfade, zeigt Abbildung 5.3:

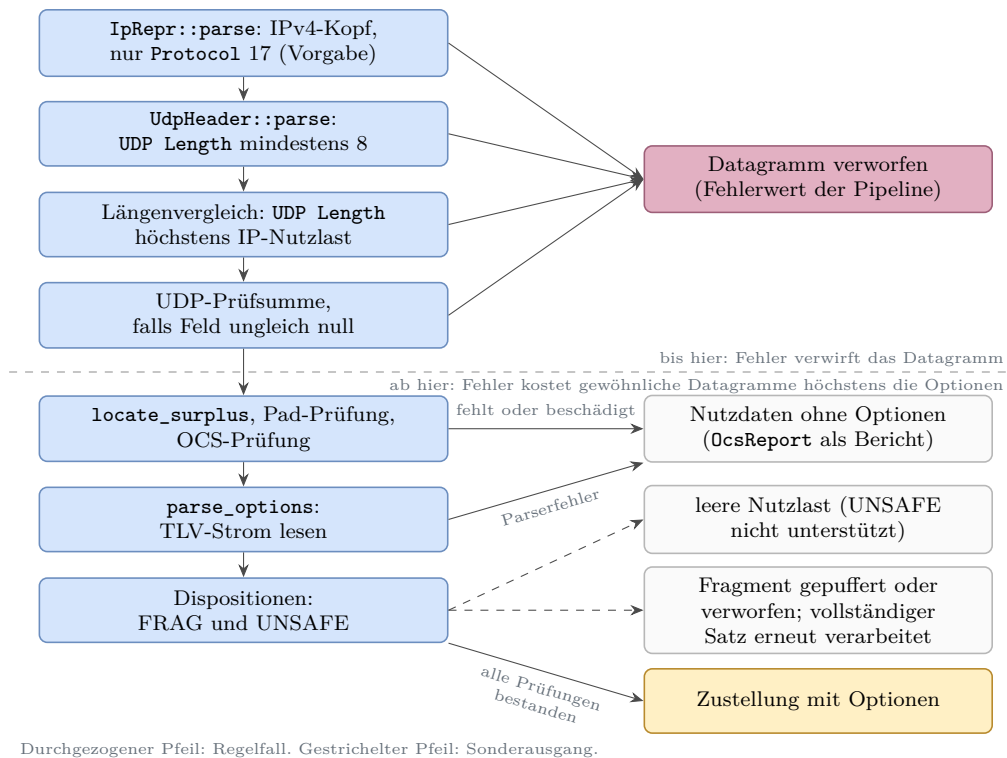


Abbildung 5.3: Empfangspipeline der Bibliothek mit ihren Ergebnisarten

Aus Abbildung 5.3 geht die Grenze der gewöhnlichen Empfangsverarbeitung hervor: Ein Fehler bis einschließlich der UDP-Prüfsumme kostet das ganze Datagramm, ein Fehler danach höchstens die Optionen; die gestrichelten Sonderausgänge für UNSAFE und FRAG vertieft Abschnitt 5.5; der Regelausgang schließt eine leere sichtbare Optionsliste ein, denn FRAG, NOP und EOL erscheinen nicht als sichtbare Optionen. Anders als Abbildung 2.8, die die Normentscheidung des RFC zeigt, steht hier die Umsetzung. Das Beispieldatagramm aus Abschnitt 5.3 nimmt den geraden Weg und liefert die drei Nutzdatenbytes samt REQ-Option.

Die Surplus-Ortung selbst trägt eine Feinheit des Normtexts: „Optionen vorhanden“ verlangt mehr als einen Überschuss größer null, denn RFC 9868 bindet die Deutung der Surplus Area daran, dass das ausgerichtete OCS samt einem etwaigen Pad-Byte hineinpasst [2, Sec. 8]. Die Ortungsfunktion der Komponente `wire` setzt das um (Zeilenumbrüche angepasst):¹²

```

pub fn locate_surplus(ip: &IpRepr, udp: &UdpHeader) -> Option<SurplusLayout> {
    let surplus = ip.transport_payload_len()
        .checked_sub(usize::from(udp.length))?;
    if surplus == 0 {

```

¹² `locate_surplus` in `src/wire/surplus.rs` (Zeilen 52 bis 67).

```

        return None;
    }
    let natural_start = ip.header_len() + usize::from(udp.length);
    let needs_pad = natural_start % 2 == 1;
    if surplus < usize::from(needs_pad) + usize::from(OCS) {
        return None;
    }
    Some(SurplusLayout { starts_at: natural_start, needs_pad, len: surplus })
}

```

Ein Überschuss von einem Byte (oder von zwei Bytes bei ungeradem Start) trägt danach keine Optionen. Ihn deshalb auch nicht als Fehler zu behandeln, ist die aus dieser Formulierung gezogene Entwurfsentscheidung der Bibliothek: Die Funktion liefert `None`, die Pipeline stellt die Nutzdaten unverändert zu, und der Bericht vermerkt mit `Absent`, dass kein nutzbarer Optionsbereich vorlag. Die Mindestgrößen dahinter hat Abschnitt 2.4.2 hergeleitet; hier entscheiden sie über den Weg im Code.

Was bei einem beschädigten Optionsbereich geschieht, regeln die Empfängerpflichten satzgenau. Weil Optionen senderseitig an jedem gültigen UDP `Length`-Offset beginnen dürfen (Abschnitt 5.3), muss der Empfänger jede solche Lage verarbeiten; verlangt sind Nullbytes vor dem OCS, und steht dort etwas anderes, sind alle Optionen zu ignorieren und die Surplus Area still zu verwerfen. Dieselbe Disposition, also derselbe Verarbeitungsausgang, gilt bei fehlgeschlagener OCS-Prüfung und bei Längenwerten, die über die Surplus Area hinausweisen (in der durch das Erratum konsolidierten Fassung) [2, Sec. 8, 9 und 10][15]. Verworfen wird dabei die Surplus Area, nicht das Datagramm: Nutzdaten mit bestandener oder zulässig ungenutzter UDP-Prüfsumme werden weiter zugestellt. In der Pipeline führt jeder dieser Wege, ebenso wie jeder andere Parserfehler im TLV-Strom, über dieselbe Hilfsfunktion `deliver_without_options`, die die leere Optionsliste liefert und den OCS-Status getrennt mitführt; der konkrete TLV-Parserfehler wird nur protokolliert. Diese Grundregel gilt für gewöhnliche Datagramme; eine nicht unterstützte UNSAFE-Option führt stattdessen zur Zustellung mit leerer Nutzlast, ein fragmentlokaler Fehler nach bereits erkanntem gültigem FRAG zum Verwerfen des Fragments (Abschnitt 5.5).

Ob der Empfänger strenger sein darf, regelt die Voreinstellung des RFC: Standardmäßig verhält sich ein optionsfähiger Empfänger wie ein Altempfänger und stellt auch bei fehlgeschlagener OCS-Prüfung zu; strengeres Verhalten muss die Anwendung ausdrücklich anfordern (Abschnitt 2.4.5). Die Bibliothek bildet das in der Empfangsrichtlinie `ReceivePolicy` ab, mit drei Stellschrauben: Pflichtoptionen (nur für meldefähige Optionsarten), das Ausfiltern aller optionstragenden Datagramme und eine OCS-Pflicht, die sowohl ein gültiges als auch ein zulässig ungenutztes OCS erfüllt.¹³ Alle drei sind ab Werk aus. Ein Betreiber, der die OCS-Pflicht anfordert,

¹³ `ReceivePolicy` in `src/api/mod.rs` (Zeilen 139 bis 200).

gewinnt die Zusicherung, dass Optionen nur mit bestandenem oder zulässig ungenutztem OCS zugestellt werden, und verliert dafür Datagramme mit unterwegs beschädigter Surplus Area; das Ausfiltern aller optionstragenden Datagramme verzichtet darüber hinaus auch auf gültige Optionen. Eine Grenze bleibt: Das Prinzip des RFC nennt neben der OCS- auch die Authentifizierungs- und Entschlüsselungsprüfung; AUTH und UENC sind nicht implementiert (Tabelle 4.2), dieser Teil des Prinzips liegt außerhalb des Umfangs.

Sichtbar werden diese Entscheidungen über zwei Berichtstypen. Der `OcsReport` führt Quelle und `OcsStatus`; der Status meldet in sechs Ausprägungen, was aus der OCS-Prüfung wurde: kein nutzbarer Optionsbereich (`Absent`), gültig (`Valid`), zulässig ungenutzt (`Unused`), fehlgeschlagen (`Failed`), unzulässige Null (`InvalidZero`) und nicht beobachtet (`Unobserved`). Je sichtbarer TLV-Option meldet daneben ein `OptionReport` die Optionsart, die Quelle und den Ausgang: Erfolg, Fehlschlag oder bewusstes Übergehen; FRAG, NOP und EOL bleiben intern.¹⁴ Diese Sicht ist bewusst unvollständig, und drei Grenzen sind festzuhalten: Ein Pad-Fehler und eine fehlgeschlagene OCS-Rechnung fallen in derselben Ausprägung `Failed` zusammen, ein Parserfehler im TLV-Strom erzeugt keinen eigenen Optionsbericht, und `Peer::recv` bildet alle Nicht-Socket-Fehler sowie gepufferte, verworfene und gefilterte Ausgänge auf „kein Datagramm“ ab. Ob diese Sicht für die Messkampagnen ausreicht, bewertet Kapitel 6.

Im Messbetrieb zeigte sich eine Schwäche des Empfangswegs. Der Raw-Empfänger sieht im jeweiligen Netznamensraum den gesamten UDP-Verkehr, unabhängig vom Zielport. In einer frühen Fassung beendete das erste fremde Paket den Empfangsaufwurf still; eine laufende Kampagne endete damit ohne Befund, und als Notbehelf diente der Neustart des Empfängers. Die Behebung verlegt die Filterung in eine Schleife unter einer Gesamtfrist: Fremde Ports, eigene Kopien und unlesbare Köpfe beenden den Aufruf nicht mehr, und jeder Leseversuch erhält nur die Restzeit.¹⁵ Auf der tiefen Ebene bedeutet `RawReceiver::recv = None` damit Fristablauf ohne passenden Treffer. Die hohe Ebene `Peer::recv` verwendet `None` zusätzlich für Dekodierfehler sowie `Buffered`, `Dropped` und `Filtered`; öffentlich sind diese Ursachen nicht unterscheidbar.

In dieser Schleife steht auch der zweite und letzte `unsafe`-Block in `src/`: Nach jedem Leseerfolg fasst `std::slice::from_raw_parts` die vom Betriebssystem gefüllten Bytes als Slice zusammen (`src/socket/recv.rs`, Zeile 169), und der SAFETY-Kommentar hält fest, dass die Socket-Schicht genau die ersten n Bytes initialisiert

¹⁴ `OcsReport`, `OcsStatus` und `OptionReport` in `src/recv/pipeline.rs` (Zeilen 128 bis 173).

¹⁵ `src/socket/recv.rs` (Zeilen 113 bis 175) und `Peer::recv` in `src/api/mod.rs` (Zeilen 322 bis 332); Behebung in den Commits `f3bf430` und `8f8c11b`, beide Vorfahren von `f847895`.

hat. Damit ist die Zählung aus Abschnitt 5.2 eingelöst: genau zwei **unsafe**-Blöcke im Produktionsbaum **src/**, einer je Richtung. Was hinter der Surplus-Ortung mit dem Optionsstrom geschieht, vertieft nun Abschnitt 5.5.

5.5 Optionsverarbeitung

Die Komponente **options** trägt beide Richtungen des Optionsteils: Der Serialisierer baut aus typisierten Optionswerten den Optionskörper, den der Sendepfad in die Surplus Area kopiert (Abschnitt 5.3), und der Parser liest beim Empfang den TLV-Strom hinter dem OCS (Abschnitt 5.4). Das Format selbst hat Abschnitt 2.4.4 eingeführt; hier steht, wie der Bau es umsetzt und welche Politik er darüberlegt.

Der Rahmen ist eine bewusste Anlehnung an TCP: Die Optionen nutzen eine TLV-Syntax nach dem Vorbild der TCP-Optionen, und die zwei Ein-Byte-Optionen **EOL** (Kind 0) und **NOP** (Kind 1) übernehmen zusätzlich die Kind-Werte von TCP; beide haben eine implizite Länge von eins, und nach einem frühen **EOL** folgen Nullbytes bis zum Ende der Surplus Area [2, Sec. 8 und 10][9].

Auf der Sendeseite endet der gewöhnliche Optionsbau in derselben Schlussfunktion des Serialisierers; nur die Optionskörper der einzelnen FRAG-Fragmente erzeugt **frag/split.rs** getrennt (Abschnitt 5.7). Die Schlussfunktion zeigt die ganze Ordnung des Optionskörpers auf einmal:¹⁶

```
pub fn finish(self) -> Result<Vec<u8>, SerializeError> {
    let mut options = self.validated_options()?;
    options.sort_by_key(|option| canonical_rank(option.kind.to_byte()));

    let body_len = serialized_body_len(&options)?;
    patch_frag_start(&mut options, body_len)?;
    let mut out = Vec::with_capacity(body_len);
    out.extend_from_slice(&[0, 0]);

    for option in &options {
        if out.len() % 2 == 1 {
            out.push(kind::NOP);
        }
        encode_option(&mut out, option)?;
    }

    out.push(kind::EOL);
    if out.len() % 2 == 1 {
        out.push(0);
    }
    debug_assert_eq!(out.len(), body_len);
    Ok(out)
}
```

¹⁶ **OptionsBuilder::finish** in **src/options/serialize.rs** (Zeilen 65 bis 87, unverändert); die kanonische Reihenfolge bestimmt **canonical_rank** (Zeilen 173 bis 183).

Der Körper beginnt mit dem Null-Platzhalter für das OCS, damit die Socket-Schicht später nur noch nachpatchen muss (Abschnitt 5.3). Danach folgen die Optionen in kanonischer Reihenfolge, also der festen Sendereihenfolge der Bibliothek: FRAG zuerst, dann APC, MDS, MRDS, REQ, RES, unbekannte Arten nach Kind-Wert (`patch_frag_start` trägt dabei in einer FRAG-Option nach, wo ihre Fragmentdaten beginnen). Beginnt eine Option an ungerader Stelle, schiebt der Serialisierer ein NOP davor; den Schluss bilden EOL und höchstens ein Nullbyte auf gerade Länge, und vor dem EOL selbst wird nie ausgerichtet, wie es der RFC empfiehlt [2, Sec. 11.1]. Für das Beispieldatagramm entsteht so genau der Körper aus Abschnitt 5.3: Platzhalter, die sechs REQ-Bytes, EOL, ein Füllbyte. Der Parser der Gegenseite liefert daraus genau eine Option, Kind 6 mit Länge 6 und dem Token `c0ffee01`; damit ist das Beispiel, das Abschnitt 5.3 gebaut und Abschnitt 5.4 geprüft hat, als Parserergebnis abgeschlossen.

Der Parser behandelt die zwei Ein-Byte-Optionen in eigenen Zweigen: NOP wird überlesen und gezählt, EOL beendet die Liste endgültig; was danach kommt, betrachtet der Parser nicht mehr.¹⁷ Genau dort liegt ein bewusst nicht genutzter Spielraum: Der RFC erlaubt dem Empfänger, die Nullfüllung nach EOL zu prüfen, verlangt es aber nicht; wer prüft und etwas anderes als Nullen findet, verwirft die Optionsliste nach der Grundregel und stellt die Nutzdaten zu [2, Sec. 11.1]. Die Implementierung verzichtet auf diese freiwillige Prüfung.

Oberhalb des Formats liegt Sendepolitik, und sie ist bewusst geschichtet. Der Serialisierer erzwingt nur die harten Bauregeln: gültige Kind-Bereiche, die festen Wertlängen der Pflichtoptionen, die Obergrenzen für Wertlänge und Gesamtlänge des Optionskörpers, die Darstellbarkeit von `Frag.Start` und höchstens ein FRAG je Körper.¹⁸ Die Empfehlung des RFC, dass gewöhnliche Optionen nicht mehrfach auftreten sollen, trägt dagegen die öffentliche Schnittstelle: Der hohe Sendepfad weist Duplikate genau der fünf meldefähigen Arten APC, MDS, MRDS, REQ und RES zurück, während die tiefe Serialisierer-Ebene Duplikate absichtlich zulässt, damit auch regelwidrige Optionsfolgen für Prüfzwecke entstehen können. Die Normseite dazu ist zweigeteilt: Mehrfache gewöhnliche Optionen sind eine Soll-Regel mit klarer Empfängerfolge (nur die erste Instanz zählt), mehrfaches FRAG dagegen macht den Optionsbereich empfängerseitig zum Fehlerfall [2, Sec. 10]. Die Sperre im Serialisierer ist damit eigene Sendepolitik und keine wörtliche RFC-Pflicht.

Eine zweite eigene Politik betrifft das erweiterte Längenformat. Es ist für Optionen ab 255 Bytes Gesamtlänge vorgesehen; der Sender muss unterhalb dieser Grenze die

¹⁷ `src/options/parse.rs` (Zeilen 62 bis 89).

¹⁸ `validated_options` samt Schichtungskommentar in `src/options/serialize.rs` (Zeilen 89 bis 108); die Duplikatprüfung des hohen Sendepfads in `src/api/mod.rs` (Zeilen 592 bis 595 und 613 bis 630); die Menge der meldefähigen Arten (Zeilen 704 bis 709) gehört zur Empfangsrichtlinie.

Kurzform verwenden und soll stets das kleinste Format wählen; für eine strukturell vollständige, aber nicht kanonische erweiterte Gesamtlänge (4 bis 254) legt der RFC keine eigene Empfängerdisposition fest, während Längen unter dem Mindestwert der jeweiligen Art ein RFC-Fehlerfall bleiben [2, Sec. 10]. Der Parser entscheidet streng: Eine erweiterte Länge unter 255 gilt als ungültige Länge und verwirft alle Optionen, genau wie ein echter Überlauf über die Surplus Area hinaus (FR-10). Der Quelltext kennzeichnet diese Strenge ausdrücklich als lokale Politik, nicht als RFC-Pflicht.¹⁹

Im Datenmodell trennt eine feste Grenze SAFE von UNSAFE: Kind-Werte 0 bis 191 gelten als SAFE, weil ein Altempfänger sie folgenlos übergehen kann, Kind-Werte 192 bis 255 als UNSAFE, weil sie die Deutung der Nutzdaten verändern können (Abschnitt 2.4.5).²⁰ Entsprechend unterschiedlich fallen die Dispositionen aus: Eine unbekannte SAFE-Option wird still übergangen. Eine unbekannte UNSAFE-Option außerhalb eines Fragmentkontexts führt zur Zustellung mit leerer Nutzlast: Verworfen werden die Nutzdaten, nicht das Paket, denn ein Empfänger, der die Option nicht kennt, würde die Nutzdaten falsch deuten. Hat die Pipeline vor der unbekannten UNSAFE-Option bereits ein gültiges FRAG erkannt, verwirft sie den zugehörigen Reassemblierungssatz; trägt erst das wieder zusammengesetzte Datagramm eine solche Option, wird auch dieses mit leerer Nutzlast zugestellt [2, Sec. 12]. Selbst erzeugen kann die Bibliothek UNSAFE-Optionen nicht: Der Serialisierer weist jeden Kind-Wert ab 192 zurück.

Acht Optionen bilden den Pflichtkern: EOL, NOP, APC, FRAG, MDS, MRDS, REQ und RES muss jeder Endpunkt, der UDP Options unterstützt, erkennen und bei entsprechender Konfiguration auch erzeugen können; TIME gehört nicht dazu [2, Sec. 10]. Die Bibliothek führt genau diese acht als benannte Arten ihres Optionsverzeichnis (`src/options/kind.rs`); jede andere Art läuft als `Other` mit rohem Kind-Wert durch das Datenmodell. FRAG ist unter den acht die aufwendigste und teilt sich in zwei Hälften: `frag/split.rs` zerlegt eine Nachricht in Fragmente, `frag/reassembly.rs` setzt sie unter begrenztem Speicher wieder zusammen; die Grenzen dieser Puffer behandelt Abschnitt 5.7.

REQ und RES zeigen schließlich den Rahmenwerk-Charakter des Standards: RFC 9868 definiert Format und Grundsemantik des Frage-Antwort-Paars; die Pfad-MTU-Ermittlung DPLPMTUD für UDP Options, die es nutzt, steht als Anwendungsfall in einem eigenen Dokument [3]. Die Bibliothek folgt diesem Zuschnitt: `Res` ist reine Datenhaltung, die Antwort auf ein empfangenes REQ löst die Anwendung aus, und die Herkunft des RES-Tokens bleibt ausdrücklich Aufruferpflicht

¹⁹ `src/options/parse.rs` (Zeilen 95 bis 110, Kommentar in den Zeilen 101 bis 104).

²⁰ `UNSAFE_MIN` in `src/model.rs` (Zeilen 33 bis 37); die Dispositionen in `src/recv/pipeline.rs` (Zeilen 317 bis 334).

(Abschnitt 5.7).²¹ Ob der Bau die Kennungen erfüllt, zeigt die Prüfanlage im nächsten Abschnitt.

5.6 Testarchitektur

Den Sollprozess der Prüfung hat Abschnitt 3.4 festgelegt; dieser Abschnitt zeigt die gebaute Testarchitektur mit ihren Zahlen am eingefrorenen Stand. Sie besteht aus sechs Ebenen: Attributtests, Property-Tests, Fuzz-Ziele, Integrationstests, Lean-Modelle und die Wire-Prüfung mit getrennten Prüfern. Wo diese Ebenen im Repository liegen und worauf sie sich stützen, zeigt Abbildung 5.4:

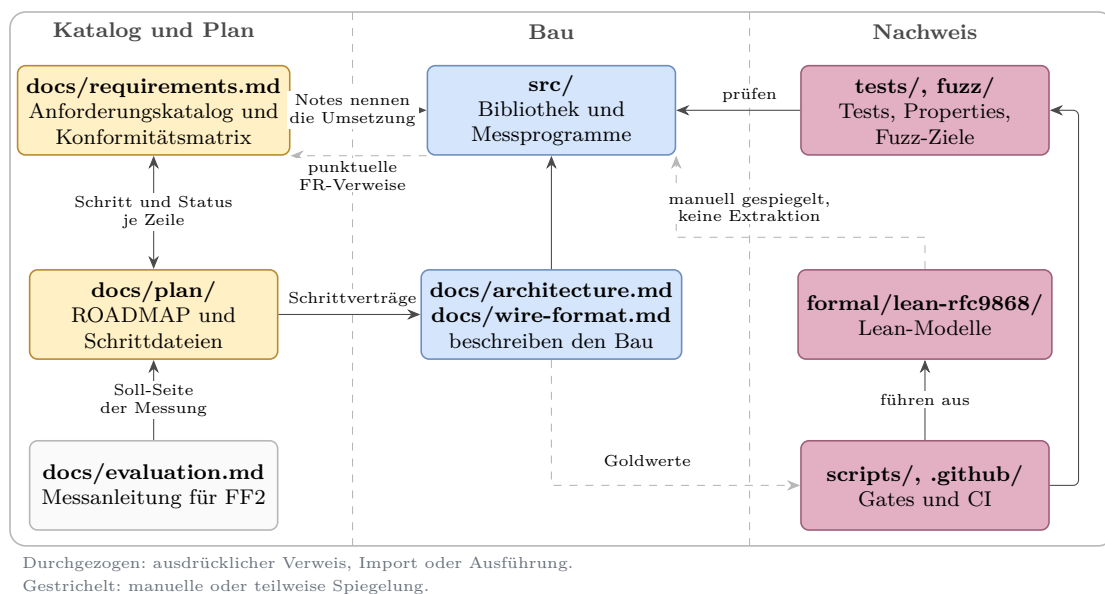


Abbildung 5.4: Entwicklungs- und Nachweisartefakte des Implementierungs-Repositorys

Aus Abbildung 5.4 geht die Arbeitsteilung der Artefakte hervor: Links führt der Katalog die Anforderungen und der Plan die Schritte, in der Mitte steht der Bau mit seinen beschreibenden Referenzen, rechts prüfen Tests, Modelle und Gates; nur die gestrichelten Kanten tragen keine erzwungene Kopplung, und genau dort setzt die Bewertung in Kapitel 6 an.

Die ersten beiden Ebenen laufen bei jedem Prüflauf auf dem Entwicklungsrechner. 212 #[test]-Attributzeilen verteilen sich auf 156 in `src/`, 55 in `tests/` und eine im Wire-Beispiel unter `examples/`; gezählt sind Attributzeilen, nicht Prüffälle, denn aus einer Zeile können im Lauf viele Fälle entstehen. Das gilt vor allem für die zehn Property-Dateien unter `tests/`: Der Host-Prüflauf erzeugt je Eigenschaft 1.024 Zufallsfälle; ohne diese Vorgabe gilt der Standardwert 256 der Bibliothek `proptest`,

²¹ `src/options/typed.rs` (Zeilen 164 bis 177).

etwa in der Kreuzprüfung auf dem Zielsystem.²²

Die dritte Ebene sucht Abstürze: neun Fuzz-Ziele, je eines für Reassemblierung, Zerlegung, OCS, Serialisierer, TLV-Strom, typisierte Werte, Empfangspipeline, Datagrammaufbau und Wire-Schicht. Der Arbeitskorpus entsteht während des Laufs; eingeecheckt sind kuratierte Startwerte (20 Dateien für sechs der neun Ziele) und die Gegenbeispiele früherer Funde als gewöhnliche Regressionstests (`tests/fuzz_regressions.rs`, 94 Zeilen).

Die vierte Ebene prüft das Zusammenspiel mit dem Betriebssystem über Loopback und Raw Socket. Sieben Tests tragen `#[ignore]`: Sechs verlangen erhöhte Rechte und laufen in der Prüfmaschine, der siebte ist ein Hilfsprozess, den ein Rauschtest über eine Umgebungsvariable startet. Die sudo-Lane der Prüfskripte ist dabei eine Ausführungsart derselben Testdateien, keine eigene Testmenge.

Die fünfte Ebene modelliert die Wire-Regeln in Lean: elf Fachmodule mit zusammen 172 Theoremen, von der Prüfsummenarithmetik bis zur Empfangsordnung. Ein Prüfskript baut die Modelle und auditiert jedes Theorem auf Kernel-Ebene: Zugelassen sind nur die drei Grundaxiome von Lean; ein `sorry` oder ein `native_decide` fiele als zusätzliches Axiom auf.²³ Bewiesen sind damit Modelle, nicht das Rust-Programm; diese Grenze nimmt Abschnitt 5.8 auf.

Die sechste Ebene begegnet einer Schwäche, die alle bisherigen teilen: Sender, Empfänger und Prüfungen stammen aus derselben Feder. Ein beidseitig gleicher Formatfehler überstünde jeden Roundtrip-Test; Property-Tests und Fuzz-Ziele prüfen zwar getrennt formulierte Invarianten, können aber Annahmefehler ihres eigenen Orakels teilen. Die Wire-Prüfung bricht diese Selbstbezüglichkeit mit drei getrennten Prüfkomponenten: `tcpdump` zeichnet hinter dem Kernel auf, ein eigenständiges Python-Prüfprogramm ohne gemeinsamen Rust-Code rechnet die für die hinterlegten Szenarien relevanten IPv4-, UDP-, Surplus-, OCS-, TLV-, APC- und FRAG-Werte mit eigener RFC-1071-Faltung und eigener CRC32C-Tabelle nach (696 Zeilen), und `tshark` liest die IPv4- und UDP-Felder als fremder Dissektor [12]. Dass die Kette greift, zeigt eine gezielte Einzelprobe: Ein einzelnes absichtlich verändertes Surplus-Byte lässt das Prüfprogramm dreifach fehlschlagen. Eine systematische Mutationsanalyse des Rust-Quelltexts ist das ausdrücklich nicht.

Wie nötig auch für das Prüfprogramm selbst geeignete Prüfdaten sind, zeigte ein Fund aus seiner ersten Fassung: Die Nachrechnung des ausgerichteten OCS gruppierte die 16-Bit-Wörter um ein Byte verschoben, und der Referenzfall blieb trotzdem grün,

²² `PROPTTEST_CASES` in `scripts/pre-pr.sh` (Zeilen 20 und 50); die Kreuzprüfung ruft `cargo test` ohne diese Variable auf (`scripts/vm-ubuntu-server.sh`, Zeilen 130 und 141).

²³ `scripts/lean-gate.sh` (Zählung Zeilen 24 bis 27, Audit Zeilen 31 bis 40); die Aggregatdatei `Rfc9868.lean` zählt nicht mit.

denn die damalige REQ-Option mit dem Token `deadbeef` faltet zu `0xA3A3`, einem Byte-Palindrom, dem die Vertauschung nichts anhat. Die Lehre ist doppelt: Genau gegen solche Verschiebungen richtet der RFC das OCS an der 2-Byte-Grenze aus (Abschnitt 2.4.2), und auch eine unabhängige Prüfinstanz braucht Testdaten, die ihre eigenen Fehlerklassen nicht maskieren. Die heutige REQ-Option mit dem Token `c0ffee01` faltet deshalb zu dem nicht palindromischen Wert `0xB507`.²⁴

Was jede Ebene leistet und was nicht, stellt Tabelle 5.1 einander gegenüber:

Tabelle 5.1: Reichweite der Prüfmittel

Prüfmittel	findet	findet nicht
Attributtests (212 Zeilen)	festgelegte Einzelfälle, Regressionen	Fehler außerhalb der gewählten Fälle
Property-Tests (10 Dateien)	verletzte Invarianten unter Zufallseingaben	beidseitig geteilte Fehlannahmen
Fuzz-Ziele (9)	Abstürze und Panics auf rohen Bytefolgen	Fehldeutungen ohne verletzte hinterlegte Invariante
Integrationstests (7 mit <code>ignore</code>)	Zusammenspiel mit Kernel und Rechten	Verhalten fremder Netzpfade
Lean-Modelle (172 Theoreme)	Widersprüche und Lücken in den modellierten Regeln	Abweichungen zwischen Modell und Rust
Wire-Prüfung (3 getrennte Prüfer)	beidseitig geteilte Formatfehler auf dem Draht	Fehler jenseits der hinterlegten Szenarien

Aus Tabelle 5.1 geht hervor, dass keine Prüfebene die anderen ersetzt. Das zeigte der Um-eins-Fehler der Surplus-Ortung, den 23 Attributtests übersahen und erst die Zweitprüfung fand. Danach wurden Property-Tests und Fuzz-Ziele je Parsefläche verpflichtend. Schritt 18 fand trotz grüner Prüfkette weitere Konformitätslücken (Abschnitt 5.1).

Den Schluss der Klammer, die Abschnitt 5.2 geöffnet hat, bildet die Rückverfolgbarkeit. Abschnitt 5.2 bis Abschnitt 5.5 haben die tragenden Komponenten mit den Anforderungskennungen des Implementierungs-Repositorys verknüpft; die Datei `docs/requirements.md` ordnet umgekehrt jeder FR- und NFR-Zeile den umzusetzen Schritt des Projektplans und ihren Status zu (Abschnitt 4.2), und punktuelle Rückverweise im Quelltext nennen die Kennungen an tragenden Stellen, etwa FR-49 an der Längenprüfung. Diese Rückverfolgbarkeit ist nicht flächendeckend: Nicht jede Quelltextstelle nennt ihre Kennung, und der Katalogabgleich lief periodisch, vollständig erst mit Schritt 18 (Abschnitt 3.2); was eine Zeile wirklich stützt, entscheidet deshalb je Nachweis erst Kapitel 6. Mit dieser Zuordnung ist das Messinstrument beisammen; es bleibt zu klären, unter welchen Rechten und Grenzen es betrieben wird.

²⁴ Begründung der Token-Wahl im Prüfprogramm `scripts/wire-check.py` (Zeilen 244 bis 248).

5.7 Betriebs- und Sicherheitsgrenzen

Der Betrieb beginnt mit einem Privileg: Raw Sockets verlangen unter Linux die Fähigkeit `CAP_NET_RAW`, also das gezielt vergebbare Recht auf rohe Netzwerkzugriffe, oder gleich Root-Rechte [17, 53]. Wer die Bibliothek einsetzt, gibt dem Prozess damit weitreichenden Zugriff auf rohe IPv4-Datagramme, nicht nur auf die eigenen dieser Arbeit: Linux füllt bei `IP_HDRINCL` nur `IP Total Length` und die IP-Kopfprüfsumme stets selbst aus (Abschnitt 5.3), Quelladresse und Paketkennung dagegen nur, wenn sie null sind; ein solcher Prozess kann also insbesondere fremde Quelladressen vorgeben [17]. Die Bibliothek kapselt diesen Zugriff hinter ihrer Schnittstelle, aufheben kann sie ihn nicht; die Rechtevergabe bleibt eine Betriebsentscheidung des Aufrufers (`src/socket/mod.rs`, Zeilen 1 bis 6).

Drei Entwurfsprinzipien des RFC aus Abschnitt 2.4.5 prägen den Betrieb der Empfangsseite. Das erste betrifft die Richtung: `Peer::recv` ruft keinen Sendepfad auf, die RES-Option ist im Datenmodell reine Datenhaltung, und dass ein RES-Token wirklich aus einem empfangenen REQ stammt, bleibt ausdrückliche Aufruferpflicht (Abschnitt 5.5). Diese strikte Trennung verkleinert die Fläche für Reflexionsmissbrauch, ein allgemeiner Schutz ist sie nicht; die Sicherheitsbetrachtungen des RFC nennen REQ als die einzige Pflichtoption, die ohne eigene Sendung der Anwendung eine Antwort der Oberschicht auslösen kann [2, Sec. 25.2]. Die Verbindung dieser Beobachtung mit der Pfadtrennung des Baus ist eine eigene Ableitung dieser Arbeit.

Das zweite Prinzip betrifft den Zustand: UDP bleibt zustandslos, nötiger Zustand gehört in die Anwendung oder eine Bibliothek in ihrem Auftrag, und die Reassemblierung ist die einzige, ausdrücklich begrenzte Ausnahme. Im Bau ist der `ReassemblyCache` der einzige fachliche Empfangszustand über Paketgrenzen hinweg; sein Schlüssel besteht aus dem UDP-Vierertupel und der FRAG-Identifikation, also genau dem Identitätskriterium, mit dem der RFC ein ursprüngliches Datagramm eindeutig bestimmt. Eine Cache-Instanz je Socket-Paar fordert der RFC dagegen nicht; er empfiehlt nur, die Speichergrenzen der Reassemblierung nicht über mehrere Sockets hinweg als gemeinsame Ressource zu berechnen [2, Sec. 11.4]. Die dokumentierte Bindung jeder Instanz an genau ein Socket-Paar ist deshalb der strengere eigene Aufrufervertrag dieser Implementierung, denn die Mengengrenze gilt je Cache. Der Typ erzwingt ihn nicht, und auch die `Peer`-Ebene löst ihn nur teilweise ein: Jeder `Peer` hält genau einen Cache, filtert eingehende Datagramme aber allein über die beiden UDP-Ports, sodass Verkehr verschiedener Adresspaare mit gleichen Ports denselben Cache und dessen Mengengrenze teilt.²⁵ Sendeseitig existiert daneben nur der fortlaufende Identifikationszähler der Zerlegung, der monoton zählt und

²⁵ `src/frag/reassembly.rs` (Zeilen 16 bis 25 und 241 bis 253); `Peer` mit eigenem Cache in `src/api/mod.rs` (Zeilen 263 bis 290).

bei Erschöpfung anhält, statt Werte zu wiederholen (`src/frag/split.rs`, Zeilen 66 bis 94). Die globalen Warnzähler dienen allein der Diagnose; die Zähler im Reassemblierungszustand erzwingen dagegen die konfigurierten Grenzen.

Das dritte Prinzip legt Grenzen in die Konfiguration statt in das Format: Der Parser kennt keine Obergrenze für die Zahl der Optionen und prüft jede Option nur lokal auf Längenfehler. Mehr als sieben aufeinanderfolgende `NOP`-Optionen lösen eine Warnung aus, beenden die Verarbeitung aber nicht; Warnungen erscheinen je Kategorie beim ersten und danach bei jedem 64. Vorkommnis und werden nur sichtbar, wenn die einbettende Anwendung einen Logger installiert. Mengengrenzen trägt die Empfangskonfiguration `ReassemblyLimits` mit vier Feldern: wiederhergestellte Datagrammgröße einschließlich UDP-Kopf, Fragmentzahl je Datagramm, gleichzeitig unvollständige Datagramme je Cache und Verfallsfrist (FR-34, NFR-06). Die ersten beiden Voreinstellungen sind wörtliche `MUST`-Mindestwerte des RFC: 2.926 Byte als kleinste für IPv4 zu unterstützende Wiederherstellungsgröße (das entspricht genau zwei Fragmenten in Ethernet-Größe) und zwei Fragmente, unabhängig von der IP-Version. Die voreingestellte Verfallsfrist von zwei Minuten übernimmt eine `SHOULD`-Obergrenze für den Standardwert. Die vierte Zahl, 64 unvollständige Datagramme je Cache, ist dagegen eine eigene Schutzentscheidung: Eine Grenze für anhängige Fragmente empfiehlt der RFC nur Implementierungen, die die Fragmentierung als mögliche Gefährdung ansehen, und er nennt keinen Zahlenwert [2, Sec. 11.4, 11.6 und 25.4].²⁶ Auch hier gilt die Zweistufigkeit der Schnittstelle: Die tiefe Ebene erlaubt eigene Grenzen (`ReassemblyCache::with_limits`), wobei die wirksame Verfallsfrist bei zwei Minuten gekappt bleibt, und die hohe `Peer`-Ebene verwendet die Voreinstellungen. Die Verfallsfrist wirkt dabei nicht selbsttätig: Abgelaufener Zustand wird erst bei der nächsten Einfügung oder durch einen ausdrücklichen Bereinigungsaufwurf (`gc`) entfernt, ohne beides bleibt er gespeichert. `udpopt-recv` stellt die vier Empfangsgrenzen über Kommandozeilenargumente ein, `udpopt-send` trennt davon die Sendelänge und die angenommenen MRDS-Grenzen der Gegenstelle. Die konfigurierte Sendelänge ist eine Vorgabe des Aufrufers und keine Ermittlung der Pfad-MTU: Überschreitet ein fertiges Datagramm die MTU der Strecke, liefert Linux `EMSGSIZE`, und die Bibliothek reicht das als allgemeinen E/A-Fehler weiter (Abschnitt 5.3).

Eine letzte Betriebsgrenze zieht die Umfangsvorgabe dieser Arbeit: Angenommen wird ausschließlich `Protocol 17`; jedes andere Protokollfeld weist die Wire-Schicht mit einem eigenen Fehlerwert ab. Shim-Ketten wie IPsec oder IPComp werden damit nicht verfolgt, obwohl deren Kopflängen von der Obergrenze der UDP `Length`

²⁶ `ReassemblyLimits` mit Voreinstellungen in `src/frag/reassembly.rs` (Zeilen 216 bis 239); die Konstanten samt Herkunft in `src/model.rs` (Zeilen 73 bis 80).

zusätzlich abziehen wären; Abschnitt 7 des RFC beschreibt diese Reduktion, trägt an dieser Stelle aber keine mit Schlüsselwörtern markierte Pflicht [2, Sec. 7]. Diese Zuschnittsentscheidung hat Abschnitt 4.1 dokumentiert.

5.8 Bewusste Grenzen

Die erste Grenze ist die Plattform. Der Bau belegt den Benutzerraumweg unter Linux mit IPv4 und Raw Sockets, wie Abschnitt 1.4 ihn festlegt; über Kernel-Implementierungen, andere Betriebssysteme oder IPv6 sagt er nichts aus. Der macOS-Messpunkt aus Tabelle 4.3 bleibt Beobachtungspunkt der Messinfrastruktur.

Die zweite Grenze ist das Netz. Alle Prüfungen dieses Kapitels laufen auf dem eigenen Host oder im kontrollierten Prüfaufbau; ob die Surplus Area reale Pfade mit fremden Zwischensystemen übersteht, ist damit nicht gezeigt und auch nicht Gegenstand dieses Kapitels. Genau diese Frage untersucht Kapitel 6 mit dem hier gebauten Instrument.

Die dritte Grenze ist die Beweiskraft. Die Lean-Modelle beweisen Aussagen über Modelle der Wire-Regeln, nicht über das laufende Rust-Programm; die Verbindung zwischen beiden bleibt Prüfsache der Tests (Abschnitt 4.4.4). Fuzzing wiederum findet Fehler, beweist aber keine Abwesenheit von Fehlern. Beide Aussagen begrenzen, was „geprüft“ in dieser Arbeit bedeutet.

Die vierte Grenze ist der bewusst begrenzte Anforderungsumfang. Im 67-zeiligen Arbeitsindex des Anforderungskatalogs sind sechs Zeilen teilweise umgesetzt: die Trennung des Reassemblierungsspeichers zwischen Socket-Kontexten (Abschnitt 5.7), zwei Empfangsregeln für geforderte oder fehlende Optionen je Quellebene, der Geltungsbereich und die Mindestlänge der Sende-API sowie zusammengefasste Empfehlungen zur Ressourcenbegrenzung und zur Referenzreihenfolge empfangener Optionen. Dazu treten abgewählte Zeilen wie die freiwillige Nullprüfung nach EOL (FR-18; Abschnitt 5.5) und der Verzicht auf einen Deckel für die Optionszahl (NFR-04) sowie die aufrufergesteuerte Cache-Bereinigung und die fehlende Pfad-MTU-Ermittlung (Abschnitt 5.7). Das sind Implementierungs- und Entwurfsgrenzen, keine belegten Grenzen von Linux, IPv4 oder Raw Sockets; ob daraus Teilbewertungen werden, entscheidet Kapitel 6.

Damit ist das Messinstrument samt seinen Grenzen beschrieben. Kapitel 6 setzt es auf die Forschungsfragen an und verwendet das Paket mit dem Token `c0ffee01` als Wire-Referenz.

6. Evaluation und Diskussion

Die Bibliothek aus Kapitel 5 wird an den Kategorien aus Abschnitt 4.2 und am Pfadmodell aus Abschnitt 4.3 geprüft. Nach dem Messkonzept (Abschnitt 6.1) folgen die Pfadergebnisse (Abschnitt 6.2), der Soll-Ist-Abgleich (Abschnitt 6.3), die KI-Reflexion (Abschnitt 6.4) und die Grenzen (Abschnitt 6.5).

6.1 Test- und Messkonzept

Für die Ergebnisse gelten die in Kapitel 4 definierten Begriffe unverändert. Ein Optionsdatagramm kommt je Pfad, Richtung und Messzeitpunkt **erhalten**, **verändert** oder **entfernt** an, oder es wird **verworfen** (Abschnitt 4.3); Anforderungen werden als **vollständig**, **teilweise** oder **nicht erfüllbar** bewertet, die Sonderklasse **nicht anwendbar** bleibt sichtbar, wird aber nicht bewertet (Abschnitt 4.2). Dabei dürfen Kategorien leer bleiben: Findet die Bewertung keine grundsätzlich unerfüllbare Anforderung, ist das ein Ergebnis und kein Mangel des Maßstabs.

Die Messläufe selbst tragen unterschiedliche Beweiskraft, und genau diese Abstufung legt das Testkonzept vorab fest. Eine **Kampagne** umfasst beidseitige Mitschnitte, ein Sendemanifest, einen Auswerterlauf und ein versiegeltes Archiv; eine **vorregistrierte Messreihe** folgt einem vorab festgelegten Zellplan, verzichtet aber auf einen Teil dieses Vollpakets; ein **Pilot** ist eine Funktionsprobe ohne Mitschnitt oder Manifest; eine **Vorstufe** prüft eine Richtung mit einem Datagramm je Klasse. Alle Läufe dieser Arbeit ordnet Tabelle 6.1 ein:

Aus Tabelle 6.1 geht hervor, dass die tragenden Aussagen der Ergebnisabschnitte auf Kampagnen ruhen, während Piloten und Messreihen Lücken schließen und Mechanismen isolieren. Pilotwerte werden nicht mit Kampagnensummen verrechnet; jede Zahl erbt die Einstufung ihres Laufs. Der Katalog umfasst 45 Kennungen. Die Archive belegen auf jedem der sechs vollständigen Hostpaare 18 Treiberszenarien. Diese Treiber decken die Katalogthemen in unterschiedlicher Tiefe ab, rechtfertigen aber keine pauschale Zählung von 43 vollständig kataloggetreu gefahrenen Szenarien: Bei S-13 fehlt die vorgesehene APC-Längenvariante, der als S-21 bezeichnete Lauf prüft eine NOP-Flut statt der drei katalogisierten Strukturverstöße, und der TIME-Lauf prüft Parserverhalten statt der in S-23 vorgesehenen Echo- und RTT-Semantik. S-24 ist empfängerseitig durch S-24r belegt und senderseitig eine dokumentierte Grenze; S-51 wurde begründet gestrichen. Ohne vorher festgelegte Äquivalenzregel lässt sich keine korrigierte Vollständigkeitsquote bilden.

Tabelle 6.1: Messläufe der Arbeit nach Pfad, Stand und Einstufung

Lauf	Pfad	Stand	Umfang	Stufe	Archiv
Lokale Lanes	4 Namespaces	f847895	5 Szenarien	Referenz	Impl.
Extern 10.08.	achim → mcs	7b11140	IPv4/IPv6, Tunnel	Kampagne	extern
Bidir 11.08.	achim ↔ mcs	7b11140	9/9 normalisiert, 0/6 Rückweg	Kampagne	bidir
Piloten 12./13.08.	achim, Mac, 1blue, mcs	b12ba8e	1blue 6/6; achim/Mac 0/13, 0/10	Pilot	piloten
Vorstufe 14.08.	mcs → 1blue	Archiv	11 Szenarien, $n = 1$	Vorstufe	piloten
P0/P1/P2 14./15.08.	1blue ↔ mcs	Archiv	P0 >10k; P1/P2-Suiten	Kampagne	1blue
Helsinki 15.08.	mcs, hel, 1blue	d7187eb	18 Szenarien je Paar	Kampagne	hel
Gate-Zellen 15.08.	drei EU-Paare	d7187eb	G/C/K je Richtung	Messreihe	hel
Hotspot 15.08.	achim ↔ mcs, 1blue	Archiv	$n = 30$, ATOM $n = 10$	Messreihe	hotspots
AWS-Suiten 16.08.	aws, mcs, hel, 1blue	d7187eb	18 Szenarien, 3 Paare	Kampagne	aws
GCP/AWS 16.08.	gcp, aws, aws2	d7187eb	Piloten; Zellen $n = 20$	Pilot/Zelle	aws-us
AP 16.08.	6 Regionen, mcs, 1blue	d7187eb	Zellen $n = 20$	Messreihe	aws-ap

Die Spalte Stand nennt den für einen Lauf dokumentierten Stand der Messprogramme. Die drei Pilot-READMEs ordnen den Stand **b12ba8e** den dort genannten SHA-256-Werten zu. Die Binärdateien, zeitgenössische Prüfsummenprotokolle und ein Buildprotokoll sind im Korpus der elf Archive jedoch nicht enthalten; die Zuordnung lässt sich daraus nicht unabhängig prüfen. Der Commit war bei dieser Prüfung kein Vorfahr des Hauptzweigs und wurde im vollständig bezeichneten Korpus von keiner benannten Ref erreicht. Davon getrennt beziehen sich alle Aussagen über Quelltext und Tests auf **f847895**. Die AWS-Suiten nutzen zusätzlich den Treiberstand **9d6c5bd**. Jedes der elf versiegelten Archive besitzt eine passende äußere SHA-256-Prüfsumme. Die inneren Manifeste sind nicht einheitlich portabel: Einige Archive besitzen kein Gesamtmanifest, zwei Manifeste verweisen zusätzlich auf eine fehlende temporäre Datei, und das GCP-Manifest enthält alte absolute Pfade. Das begrenzt die portable Innenprüfung, ist aber kein Hinweis auf beschädigte Archivdaten.¹

Die Werkzeugkette folgt Abschnitt 4.4.3: **tcpdump** zeichnet auf, der eigene Prüfer deutet die Surplus Area, **tshark** liefert die fremde Gegenprobe der IPv4- und UDP-Felder. Der Messbetrieb ergänzt dazu vier Betriebsregeln, die aus Fehlversuchen auf der Loopback-Schnittstelle stammen: Ausgehende Pakete werden dort nicht

¹ Die Archive und ihre Sollwerte liegen im Verzeichnis **thesis/evidence/** des Arbeitsrepositoriums; die Kurzbezeichnungen der Tabelle stehen für die dort abgelegten Dateien. Ein Hashwert belegt die Unverändertheit eines Artefakts, nicht die Wahrheit seines Inhalts (Kapitel 3).

über Richtungsfilter erfasst, Mitschnittdateien wechseln nach dem Rechteabstieg von `tcpdump` den Eigentümer, die Schnittstelle meldet sich mit einem künstlichen Ethernet-Rahmen, und ein vom Filter gezähltes Paket ist noch kein geschriebenes Paket. Aus diesem Grund beweist vor jedem Lauf ein Aufwärmpaket die Mitschnittbereitschaft. Die Terminierung des Mitschnitts unterscheidet sich dabei je Lane: Die lokale Wire-Lane wartet vor dem Signal an `tcpdump` auf drei unveränderte Dateigrößen, die Namespace-Lane nutzt eine feste Wartezeit, und die öffentlichen Kampagnen senden `SIGINT`, warten eine Sekunde und prüfen anschließend, dass `tcpdump` keinen Kernelverlust meldet. Für die Deutung der Ergebnisse wichtig ist zudem die Arbeitsteilung im Sendeweg: Auf dem `IP_HDRINCL`-Pfad setzt der Kernel die IPv4-Gesamtlänge und die Kopfprüfsumme, und da die Sendeseite der Bibliothek die IPv4-Identifikation null lässt, kann der Kernel auch dieses Feld füllen. UDP-Kopf und Surplus Area bleiben unverändert; deshalb sind Grenzfälle wie ein Datagramm mit UDP-Prüfsumme null und OCS null weiterhin exakt konstruierbar.

Den Prüfumfang der Realpfadkampagnen legt der Szenarienkatalog fest: 18 Treiberszenarien je Hostpaar (elf P0, vier P1, drei P2). Hinzu kommen 21 etikettierte Options- und FRAG-Zellen des P2-Zellplans (14 `p2opt`, sieben `p2frag`) sowie S-50 als eigene vorregistrierte Cache-Reihe mit zwölf Läufen. Der Zellplan hält je Zelle fest, ob die Erwartung aus dem RFC, aus dem Implementierungsstand oder aus einer Pfadannahme stammt.² Die lokalen Referenzläufe verwenden dagegen fünf eigene Szenarien in vier Namespace-Topologien und sieben Verdikt-Klassen. Sie liefern einen kontrollierten Nullpunkt für ausgewählte Wire-, Options- und FRAG-Fälle, bilden den Realpfadkatalog aber nicht vollständig nach.

Die externen Kampagnen vergleichen zeitnah gesendete, gleich große Kontroll- und Optionsdatagramme auf demselben gerichteten Pfad. Ein externer Lauf ist nur auswertbar, wenn die Senderaufzeichnung den Versand und die erwartete Surplus Area belegt. Die maschinelle Auswertung vergleicht Sender- und Empfängerzeichnung nach der je Kampagne dokumentierten Zuordnung: Zielport und Szenario, Sequenznummern, bei leeren Nutzdaten die Klassen- und Längeninformation sowie bei FRAG die Kennung des Fragmentsatzes. Abweichende Paketzahlen werden gesondert gemeldet und machen einen Lauf nur dann ungültig, wenn die Auswertung ein bestimmtes Sollergebnis erzwingt. Auch die Behandlung IP-fragmentierter Datagramme ist auswerterspezifisch: Der Referenzauswerter bricht bei jedem IPv4-Fragment mit einem Fehler ab, weil er Fragmente nicht zusammensetzt; die Kampagnenauswerter überspringen nur Folgefragmente.

² Im Implementierungsrepositorium liegen der Treiber `scripts/pair-campaign.sh`, der Zellplan `scripts/p2-cellplan.json` und die Auswerter `scripts/p0-eval.py` bis `scripts/p2-eval.py`; die lokalen Topologien und Verdikt-Klassen definiert `docs/evaluation.md`.

Ob die Messpfade während der Kampagnen stabil blieben, prüft eine eigene Auswertung der Lebensdauerfelder aller Mitschnitte:

Richtung	Ankunfts-TTL (Pakete)	Router*
mcs → 1blue	TTL 53: 7345	11
1blue → mcs	TTL 53: 7198 TTL 54: 147	11 / 10
mcs → hel	TTL 51: 7206	13
hel → mcs	TTL 51: 7206	13
hel → 1blue	TTL 52: 7206	12
1blue → hel	TTL 54: 7136 TTL 53: 70	10 / 11

Gesendet: alle 43618 erfassten Egress-Pakete einheitlich TTL 64; * abgeleitet unter der Annahme von genau einem Dekrement je Weiterleitung

Abbildung 6.1: Ankunfts-TTL je Richtung über die Kampagnen-Mitschnitte der europäischen Messpfade

Aus Abbildung 6.1 ist nicht für jede Richtung eine einheitliche Ankunfts-TTL abzulesen. Auf dem Rückweg von 1blue nach mcs kommen 7198 Pakete mit TTL 53 und 147 mit TTL 54 an; die zweite Gruppe umfasst 137 Pakete desselben Flusses in einem zusammenhängenden Zeitfenster sowie zehn Einzelpakete mit eigenen Flusskennungen. Das belegt zeitweises Rerouting des Messflusses, und die Einzelpakete passen zusätzlich zu flussabhängiger Wegewahl. Von 1blue nach Helsinki kommen 7136 Pakete mit TTL 54 und 70 mit TTL 53 an; dort liegen 40 der 70 Pakete auf Kontrollen mit jeweils eigener Flusskennung, was eine flussabhängige Aufteilung stützt, aber nur für die Messfenster gilt. Beide Effekte kehren in Abschnitt 6.2 als Wegevielfalt des Rückpfads wieder. Keine der Gruppen liefert einen eigenständigen Beleg für eine klassenspezifische Behandlung oder für einen allgemein stabilen Pfad. Die abgeleitete Routerzahl beruht dabei auf der Annahme, dass jede Weiterleitung den Wert um genau eins senkt (Abschnitt 2.5). Dass je Richtung weniger Pakete ankommen als abgesendet wurden, ist kein Messverlust, sondern Ergebnis: Die Differenz besteht aus den Negativzellen, deren Verwerfen die Ergebnisabschnitte gerade nachweisen. Für die späteren Klassenbefunde sind deshalb die jeweiligen Kontrollpakete und Messfenster maßgeblich.

Zwei Verzichte gehören zum Konzept. Das Szenario S-24 (unterschiedliche Optionssätze je Fragment) wird senderseitig nicht erzeugt, weil die Bibliothek je Datagramm einen Optionssatz führt; die Empfängerpflicht ist mit der Ersatzzelle S-24r real belegt. Die netem-Störlane S-51 entfiel aus vier Gründen: Erstens ist jeder Mecha-

nismus, den sie stochastisch anstoßen würde, bereits deterministisch mit exakten Sollwerten belegt (S-41 bis S-44, S-49, S-50); zweitens hätte ihre Verlustzelle den Prüfgegenstand verfehlt, weil unfragmentierter Verkehr keinen RFC-9868-Codepfad berührt; drittens ist die Verlustzuordnung am realen Objekt stärker demonstriert als unter künstlicher Störung; viertens stand das Betriebsrisiko eines Root-Qdisc auf dem einzigen Messserver, dessen Verwaltungszugang über dieselbe Schnittstelle läuft, ohne eigenen Messpunkt hinter der Störstufe in keinem Verhältnis zum Nutzen. Der Kampagnentreiber verweigert das Szenario seither ausdrücklich. Als Folge bleibt das Ende-zu-Ende-Verhalten unter stochastisch gestörtem Pfad ungemessen; die Ergebnisse gelten für saubere, kalibrierte Pfade, und Abschnitt 6.3 weist diese Grenze bei den betroffenen Aussagen aus.

Mit diesen Läufen ist das Messprogramm abgeschlossen; weitere Kampagnen sind nicht vorgesehen. Zwei Grenzen begleiten deshalb alle Ergebniskapitel: Als Zugangsnetz wurde ausschließlich ein Mobilfunk-Hotspot-Pfad untersucht, und ohne Messpunkt im Betreibernetz endet jede Ursachenzuordnung an einem Intervall zwischen zwei eigenen Messpunkten. Der folgende Abschnitt wendet das Konzept auf die realen Pfade an.

6.2 Beobachtbarkeit und Middlebox-Verträglichkeit auf realen Pfaden

FF2 fragt, in welchem Umfang die Surplus Area auf realen Pfaden bis zur Gegenstelle erhalten bleibt (Abschnitt 1.3). Die Ergebnisse folgen den fünf Pfadstufen aus Abschnitt 4.3 in aufsteigender Netzferne: zunächst die kontrollierten Stufen Loopback, virtualisierte Topologien und Tunnel, ferner die öffentlichen Pfade in Europa, abschließend die interkontinentalen Pfade. Quer zu den Stufen wird jede beobachtete Abweichung nur dem Attributionsintervall zwischen den eigenen Messpunkten zugeschrieben (Abschnitt 4.3); die dabei entstandene Typologie der Mechanismenklassen bündelt Abschnitt 6.2.4 am Ende des Abschnitts. Keiner der untersuchten Pfade vervollständigt FF2: Jeder Befund gilt für das beobachtete Paar, die Richtung und das Messfenster.

6.2.1 Kontrollierte Stufen: Loopback, eigene Topologien, Tunnel

Die ersten drei Pfadstufen liefern den Nullpunkt, an dem jeder Realpfadbefund gespiegelt wird. Ihre Umgebungen zeigt Abbildung 6.2:

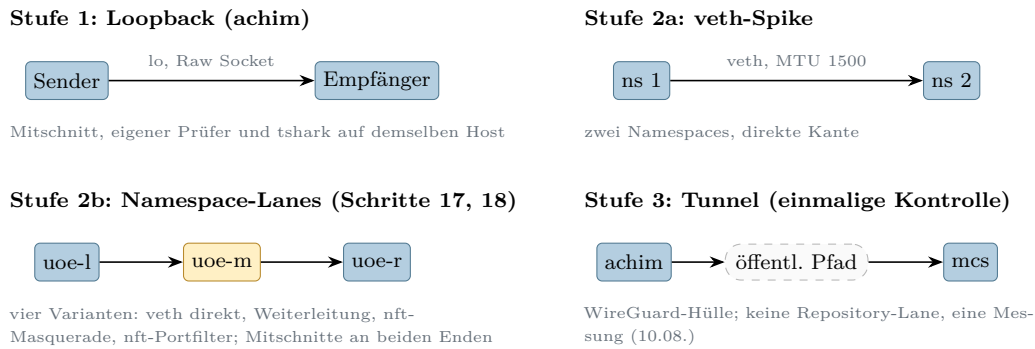
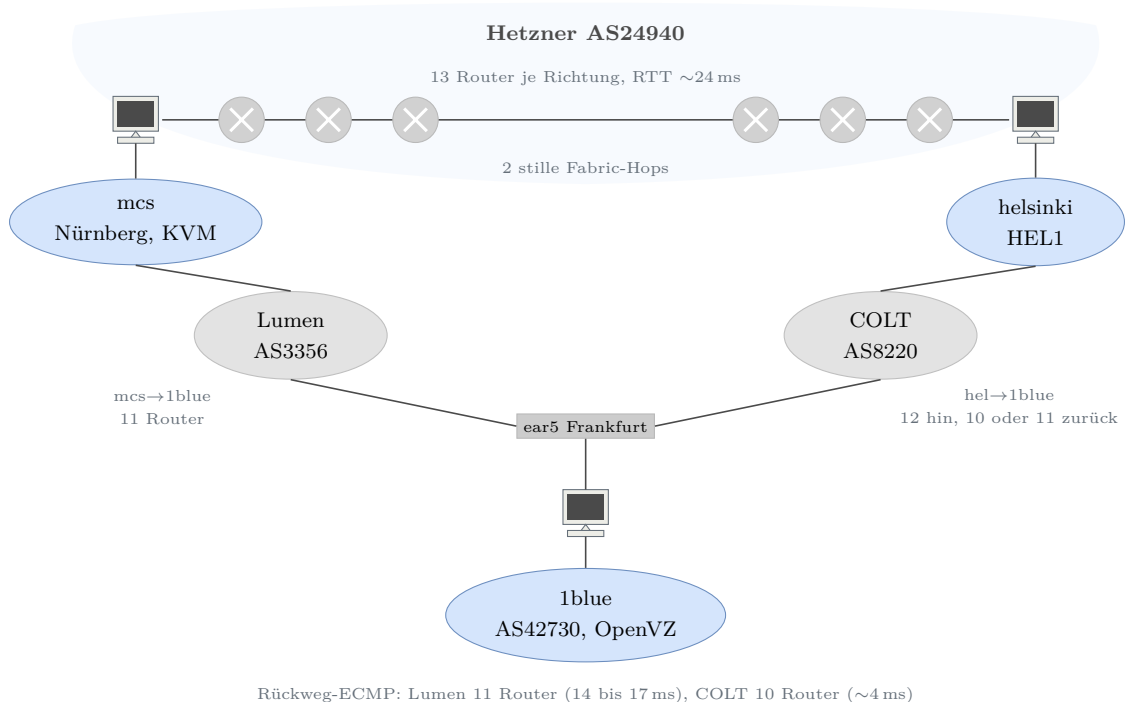


Abbildung 6.2: Kontrollierte Messumgebungen der Pfadstufen 1 bis 3

Aus Abbildung 6.2 geht hervor, dass in den Stufen 1 und 2 jede Kante entweder bekannt oder absichtlich konfiguriert ist: Ein abweichender Befund wäre dort dem Endpunkt oder der ausdrücklich eingerichteten Lane zuzuordnen, nicht einem fremden Akteur. Die fünf lokalen Szenarien belegen die Zustellung ausgewählter gültiger Options- und FRAG-Formen sowie den vollständigen Verlust in der Filtertopologie. Die Negativzellen mit beschädigtem Pad, beschädigter OCS oder fehlerhafter TLV-Kodierung stammen dagegen aus den Realpfadkampagnen und werden durch Parser- und Pipeline-Tests ergänzt. Die Tunnelstufe steuert eine einzelne Kontrolle bei: Durch die WireGuard-Kapselung erreichte das innere Datagramm die Gegenstelle mit unveränderten 26 Surplus-Bytes und gültiger OCS; wie in Abschnitt 2.5 festgehalten, belegt das die Zustellung nach der Entkapselung, nicht den nativen Transport.

6.2.2 Öffentliche Pfade in Europa

Die vierte Pfadstufe umfasst zwei sehr verschiedene Umgebungen: das Cloud-Dreieck aus den Endpunkten mcs, helsinki und 1blue sowie den Mobilfunk-Zugangspfad des Entwicklungsrechners. Den Aufbau des Dreiecks zeigt Abbildung 6.3:



Rückweg-ECMP: Lumen 11 Router (14 bis 17 ms), COLT 10 Router (~4 ms)

Abbildung 6.3: Europäisches Messdreieck mit zwei Endpunkt- und zwei beobachteten Transit-Netzen

Aus Abbildung 6.3 geht hervor, dass die drei Messstrecken zwei Endpunkt-Netze (AS24940, AS42730) und zwei beobachtete Transit-Netze (Lumen, COLT) berühren; die abgeleiteten Routerzahlen stehen unter der Annahme aus Abschnitt 2.5, dass jede Weiterleitung den TTL-Wert um genau eins senkt, und zwei Weiterleitungen existieren nur als TTL-Differenz, weil sie nie antworten. Auf diesem Dreieck wurde zuerst die Grundlast geprüft: Die Verlustbasislinie ergab auf jedem der drei Hostpaare 3000 von 3000, zusammen 9000 von 9000, zugestellte Pakete in Sandwich-Tripeln gleicher Gesamtlänge (95-Prozent-Schranke der klassenweisen Verlustrate unter 0,6 Prozent je Richtung), der MTU-Sweep trug bis exakt 1500 Byte Gesamtlänge vollständig, und über zehntausend P0-Pakete blieben ohne unerklärten Verlust. Ferner überstand das goldene Prüfzenarien-Set aus Kapitel 5 den Pfad von mcs nach 1blue unverändert: elf Szenarien mit 15 Datagrammen und 0 bis 308 Byte Surplus, an beiden Enden aufgezeichnet und beidseitig mit elf von elf Szenarien bestanden; als Vorstufe mit einer Richtung und einem Datagramm je Klasse ist das ein Pfadtransparenz-, kein Zustellungsnachweis. Zusammen mit dem Piloten vom 13.08. (je Richtung sechs von sechs Optionsdatagramme intakt) liefert der OpenVZ-Endpunkt zugleich einen eigenen Virtualisierungs-Datenpunkt: Auch dieser Netzstapel normalisiert die Surplus Area nicht. Die vollständigen Suiten liefen anschließend fehlerfrei auf beiden Helsinki-Paaren (je 18 Szenarien und Richtung).

Dennoch ist das Dreieck kein neutrales Medium, und zwar in zwei Punkten:

der Wegevielfalt des Rückpfads und einem Prüfsummen-Gate. Die Wegevielfalt dokumentiert Abbildung 6.4:

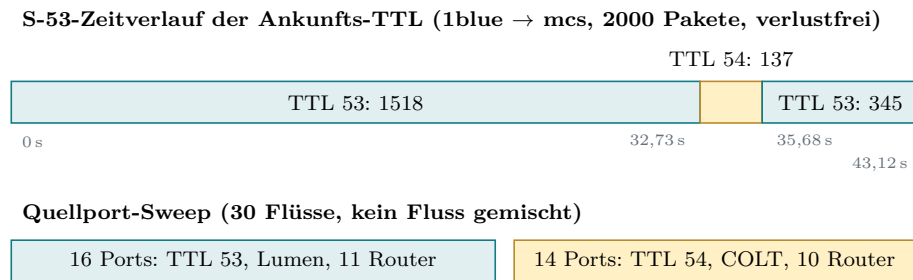


Abbildung 6.4: Reroute-Fenster im Szenario S-53 und Aufteilung des Quellportraums auf die ECMP-Zweige

Aus Abbildung 6.4 geht hervor, dass derselbe Fluss für 2,93 Sekunden verlustfrei auf eine um einen Router kürzere Variante wechselte und zurück, und dass die 1blue-Aggregation den Portraum nahezu hälftig auf einen Lumen- und einen COLT-Zweig verteilt; der Port-Sweep ist dabei eine Stichprobe des Hash-Raums, keine Lastmessung. Außerhalb des Fensters lag der feste Messfluss durchgehend auf der längeren Lumen-Variante. Diese Beobachtung hat zur direkten Konsequenz, dass ein Pfad seine Struktur auch ohne jedes Verlustsignal ändern kann; jede Pfadaussage dieser Arbeit gilt deshalb nur für ihr Messfenster.

Der zweite Befund ist ein Zustellmuster, das einer fehlerhaften Prüfung der UDP-Prüfsumme über die gesamte IP-Nutzlast entspricht, im Folgenden **Prüfsummen-Gate**. Es wurde zunächst in einer lokalen, nicht versiegelten Voranalyse erkannt und danach durch die versiegelten Kreuztests auf weiteren Paaren repliziert. In der Voranalyse von 248 einzeln nachgerechneten Paketen stimmte die Zustellung ohne Abweichung mit folgender Bedingung überein: Das UDP-Prüfsummenfeld war null oder die Einerkomplementsumme über Pseudo-Header und gesamte IP-Nutzlast ging auf, wobei die Pseudo-Header-Länge aus der IP-Gesamtlänge stammte statt aus dem UDP-Längenfeld. Der entscheidende Kreuztest bestand aus zwei Zellen: Zelle G trägt ein Pad-Byte **ff** bei rechnerisch gültiger OCS; sie ist wegen des Null-Pad-Gebots aus Abschnitt 2.4.2 regelwidrig gebaut, ihre Legacy-Faltung geht nicht auf, und sie verschwindet vollständig. Zelle C kompensiert denselben Pad-Fehler durch eine gegenkorrigierte OCS **5b53**; sie ist ebenso regelwidrig, ihre Faltung geht auf, und sie kommt vollständig an. In dieser Stichprobe folgt die Zustellung damit der Legacy-Summe und nicht der OCS. Algebraisch schließt sich das: Bei RFC-korrektur OCS ist die Legacy-Summe gleich dem Pad-Byte, denn der Längenzuschlag des Pseudo-Headers hebt sich exakt gegen den Längensummanden der OCS-Definition auf (Abschnitt 2.4.3). Damit ist zugleich die entscheidende Anschlussfrage beantwortet, ob regelkonform gebaute Datagramme ein solches Gate passieren: Ja. Ein Datagramm

mit Null-Pad und korrekter OCS besteht ein Gate dieses Typs zwingend, und genau das erklärt die verlustfreie P0-Grundlast; die Messung bestätigt das in Abschnitt 9 genannte Entwurfsziel der OCS, Datagramme durch Zwischensysteme zu tragen, die den Surplus fälschlich in die UDP-Prüfsumme einbeziehen [2, Sec. 9]. Zugleich folgt eine Prüfbarkeitsgrenze für Abschnitt 6.3: Die Empfängerpflichten für verfälschte Pads und Prüfsummen sind auf solchen Pfaden nur messbar, wenn das Datagramm pfadneutral gehalten wird, entweder durch die kompensierte OCS oder durch eine genullte UDP-Prüfsumme; beide Umgehungen wurden mit je 40 Paketen validiert.³

Wo dieses Gate sitzt, klärt die systematische Wiederholung des Kreuztests auf allen Paaren, ergänzt um einen reinen Amazon-Vergleichspfad; Abbildung 6.5 stellt alle Richtungen zusammen:

Richtung	G: Pad ff, OCS gültig	C: OCS 5b53 kompensiert	Kontrolle
EU-Paare			
1blue → mcs	× 0/20	✓ 20/20	✓ 20/20
mcs → 1blue	× 0/20	✓ 20/20	✓ 20/20
mcs → hel	× 0/20	✓ 20/20	✓ 10/10
hel → mcs	× 0/20	✓ 20/20	✓ 10/10
hel → 1blue	× 0/20	✓ 20/20	✓ 10/10
1blue → hel	× 0/20	✓ 20/20	✓ 10/10
AWS-Paare			
aws → mcs	× 0/20	✓ 20/20	✓ 10/10
mcs → aws	× 0/20	✓ 20/20	✓ 10/10
aws → hel	× 0/20	✓ 20/20	✓ 10/10
hel → aws	× 0/20	✓ 20/20	✓ 10/10
aws → 1blue	× 0/20	✓ 20/20	✓ 10/10
1blue → aws	✓ 20/20	✓ 20/20	✓ 10/10
Amazon-Backbone			
aws → aws2	✓ 20/20	✓ 20/20	✓ 10/10
aws2 → aws	✓ 20/20	✓ 20/20	✓ 10/10

Zugestellt (✓) heißt: am fernen Ende aufgezeichnet beziehungsweise ausgeliefert; die zugestellten G-Datagramme verwirft der Empfänger anschließend regelkonform (Pad ungleich null), die Nutzdaten stellt er zu

Abbildung 6.5: Prüfsummen-Gate-Kreuzzellen je Messrichtung

Aus Abbildung 6.5 geht hervor, dass die regelwidrige, aber faltungsgültige Zel-

³ Die 248-Paket-Reihe und ihr SHA-256-Manifest liegen nur im lokalen, von Git ignorierten Arbeitsverzeichnis. Sie sind nicht Bestandteil der elf versiegelten Evidenzarchive und dienen deshalb der Hypothesenbildung. Die übertragenen Pfadaussagen stützen sich auf die späteren versiegelten Kreuztests.

le C jede der 14 Richtungen vollständig passiert, während die faltungsungültige Zelle G in elf Richtungen vollständig verschwindet und in drei Richtungen vollständig ankommt. Das Muster der Überlebensrichtungen trägt die Verortung: Der Amazon-Backbone lässt G beidseitig durch, ebenso der Weg von 1blue zu aws; die Ausfälle konzentrieren sich auf alle Richtungen mit Hetzner-Beteiligung und auf den 1blue-Eingang. Als sparsamstes Modell bleibt eine kanten- oder empfangs-seitige Legacy-Prüfsummenvalidierung bei Hetzner in beiden Richtungen und bei 1blue nur eingehend, keine bei Amazon; das ist eine Schlussfolgerung aus Endpunkt-Mitschnitten, denn zwischen Provider-Kante und letztem Transit-Abschnitt kann ohne Messpunkt im Netz nicht unterschieden werden. Der Asien-Durchlauf replizierte beide Kantenbefunde aus sechs weiteren Transiten und ergänzte drei weitere G-Überlebensrichtungen von 1blue nach Mumbai, Singapur und Osaka. In der ersten Überlebensrichtung wurde zudem erstmals das Empfängerbild einer zugestellten G-Zelle beobachtet: Der Empfänger verwirft die Optionen wegen des Pad-Bytes regelkonform und stellt die Nutzdaten zu. Diese Beobachtung hat zur direkten Konsequenz, dass ein regelwidriges Pad ein Datagramm auf vielen realen Pfaden unzustellbar macht, bevor irgendein Empfänger urteilen kann; die Null-Pad-Regel ist damit keine Formalie, sondern Zustellbedingung.

Die zweite europäische Umgebung ist der Zugangspfad über einen Mobilfunk-Hotspot; Abbildung 6.6 zeigt beide Betriebsarten:

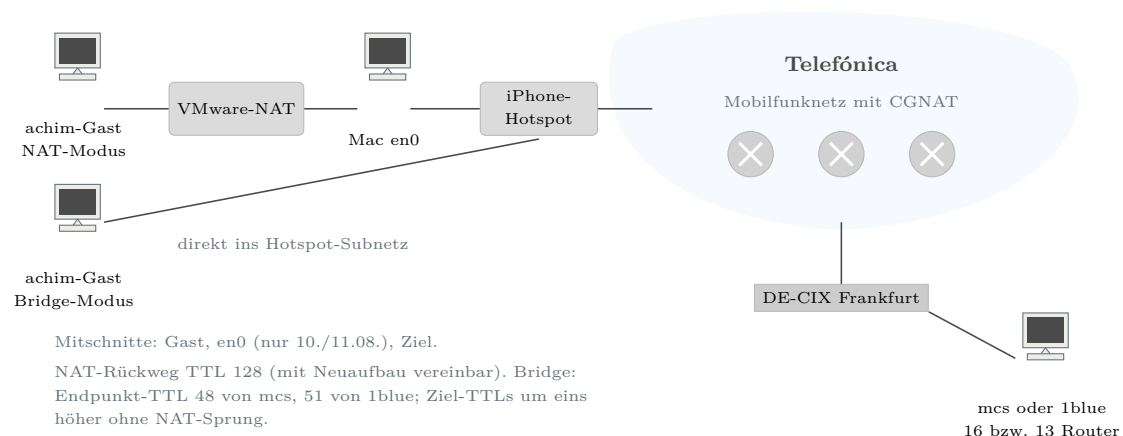


Abbildung 6.6: Zugangspfad über den Mobilfunk-Hotspot in den Betriebsarten NAT und Bridge

Auf diesem Pfad scheiterte der erste Realpfadversuch dieser Arbeit zweifach, und beide Mechanismen ließen sich später trennen. Im NAT-Modus kamen abgehende Optionsdatagramme zwar an, aber verändert: neun von neun ohne ihre Surplus Area, gegen 18 von 18 intakte Kontrollen (Fisher exakt, einseitig, $p = 2,13 \cdot 10^{-7}$). Rückantworten mit TTL 128 und neu geschriebenen IPv4-Kennungen sind mit einem

Neuaufbau aus dem Verbindungszustand der NAT vereinbar (Abbildung 6.6). Die Messpunkte trennen `vmnet-natd` jedoch nicht von der macOS-VMnet-API und legen das interne Längenmodell nicht offen. Direkt belegt ist allein die Normalisierung auf IPv4-Kopflänge plus UDP-Länge, bei unveränderten Nutzdaten und entfernter Surplus Area. Dieses beobachtete Drahtbild definiert die Mechanismenklasse des **Normalisierers** mit der Ergebnisklasse entfernt; der interne Mechanismus des Geräts bleibt unbewiesen. Sein deutlichster Fall betrifft atomare FRAG-Datagramme, deren gesamter Inhalt in der Surplus Area liegt: Sie kommen als leere 28-Byte-Hüllen an und werden zugestellt, zehn von zehn; die Anwendung erhält leere statt fehlender Datagramme, ohne jedes Verlustsignal. In der Rückrichtung verschwanden Optionsdatagramme dagegen vollständig: null von sechs gegen 15 von 15 gleich große Kontrollen (Fisher exakt, einseitig, $p = 1,84 \cdot 10^{-5}$); offen blieb zunächst, welches Merkmal der Klasse den Ausschlag gibt.

Diese Frage beantwortete die vorregistrierte Wiederholung mit Kontrollzellen zu je 30 Paketen: Kontrollen ohne Surplus Area passieren in beiden Größen (58 und 44 Byte) vollständig, während gültige Optionen, inhaltsloser Zufallsrumpf mit korrekter OCS und die Variante mit genullter UDP-Prüfsumme gleichermaßen vollständig verschwinden (je null von 30; jeder Vergleich Fisher exakt, einseitig, $p = 8,5 \cdot 10^{-18}$). Damit diskriminiert der Verwerfer allein die **Präsenz** einer Surplus Area, also die Bedingung UDP-Länge kleiner als IP-Nutzlast: nicht den Optionsinhalt, nicht die Prüfsummenlage, nicht die Größe; ein Prüfsummen-Gate ist doppelt ausgeschlossen, weil auch die genullte Prüfsumme stirbt. Dieses Wirkungsbild definiert die Mechanismenklasse des **Präsenz-Verwerfers**. Die Triangulation mit getauschtem fernen Ende (1blue statt mcs, anderes Endpunkt-Netz, anderer Transit, identisches Zellenbild) verortet den Mechanismus in das gemeinsame Segment aus Hotspot-Anschluss und Telefónica-Mobilfunknetz. Der Bridge-Betrieb schließlich nahm die VMware-NAT aus dem Pfad: Erstmals verließen Surplus-Datagramme den Gast unversehrt, und nun verwarf der Pfad sie auch im Uplink klassenrein (null von 30 je Zelle, atomare Zelle null von zehn, an beiden Zielen; die um genau eins erhöhten Ziel-TTLs belegen den entfernten NAT-Sprung). Der Zugangspfad verwirft die Klasse also in beiden Richtungen, und der NAT-Modus hatte den Uplink-Verwerfer zuvor verdeckt, weil die Normalisierung jedes Datagramm surplus-frei machte, bevor es den Carrier erreichte: Ein früher Eingriff kann einen späteren verdecken, hier als Messergebnis. Was ohne Messpunkt im Telefon oder Betreibernetz offen bleibt, ist die Trennung zwischen Hotspot-NAT und Mobilfunknetz; jede engere Zuschreibung wäre eine Behauptung ohne Beleg.

6.2.3 Interkontinentale Pfade

Die fünfte Pfadstufe erweitert die Messbasis um zwei Hyperscaler-Fabrics und lange Transitketten; Abbildung 6.7 ordnet die Endpunkte:

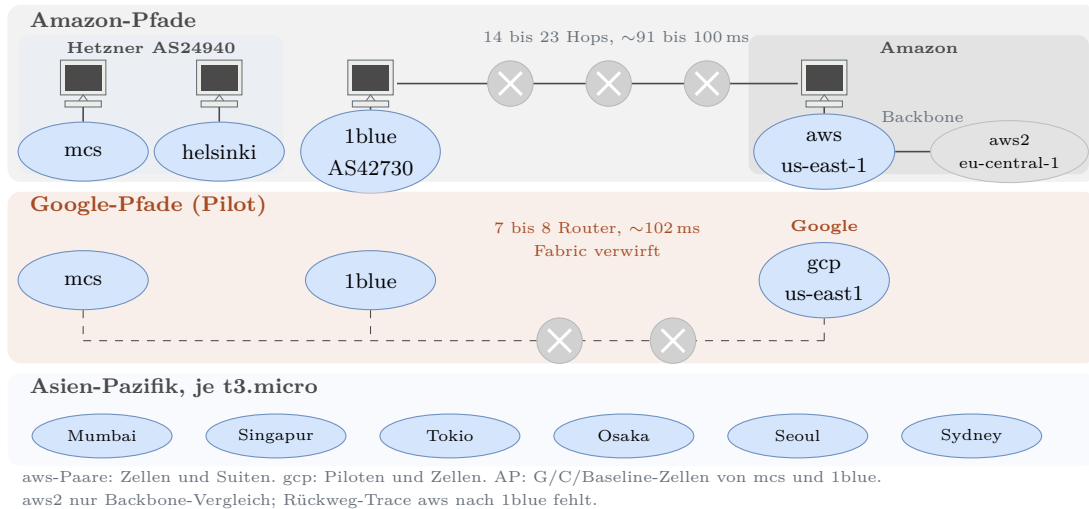


Abbildung 6.7: Interkontinentale Messendpunkte mit Belegstufe und Rundlaufzeiten

Die beiden Fabrics verhalten sich entgegengesetzt. Auf den Amazon-Pfaden aus Nitro-Fabrik, Adressumsetzung am Internet-Gateway und Transatlantik-Transit kamen in den sechs Pilot-Lanes 18 von 18 Baselines und 36 von 36 Optionsdatagramme unversehrt an; die anschließende Diskriminator-Matrix mit sechs Zellformen bestätigte das mit 240 von 240 Paketen. Die erfolgreiche UDP-Prüfsummenprüfung nach der Adressumsetzung belegt deren Anpassung, während die übereinstimmenden Optionsbytes und die gültige OCS getrennt davon stützen, dass die Surplus Area unverändert blieb. Auch die vollständigen Suiten erreichten auf allen drei aws-Paaren die erwarteten funktionalen Zustellmengen einschließlich der MTU-Kante bei exakt 1500 Byte. Damit ist zweierlei belegt: Der Präsenz-Verwerfer ist keine Eigenschaft von Hyperscaler-Fabrics an sich, und der EU-US-Transit verwirft die Klasse nicht; dies sind die ersten interkontinentalen Pfade dieser Arbeit, auf denen RFC 9868 in beiden Richtungen funktioniert. Ein Lauf des Szenarios S-50 auf dem Pfad von aws nach 1blue lag mit einem Terminalfragment außerhalb des vorregistrierten 30-Sekunden-Fensters (Lauf 3, rückwärts). In den sechs Läufen dieser Richtung kamen insgesamt 966 von 966 Fragmenten an, und im betroffenen Lauf wurden alle 64 erwarteten Sätze vor dem 60-Sekunden-Cachetimeout zugestellt; der funktionale Cachebefund blieb also erhalten, der Lauf war dennoch nicht abweichungsfrei. Die Google-Fabric dagegen verwarf die Klasse bidirektional und inhaltsblind: Baselines 40 von 40, alle fünf Surplus-Zellformen null von 200, keine leeren Hüllen; auch die Zelle, die jedes Legacy-Gate passieren würde, stirbt. Die Endpunktmitschnitte grenzen den Verlust auf das Intervall nach der sendenden und vor der empfangenden Gast-

Schnittstelle ein, und der gleiche Befund gegen mcs und 1blue macht ein gemeinsames Google-seitiges Element zum sparsamsten Modell; die konkrete Komponente ist ohne Zwischenmesspunkt nicht nachgewiesen. Ein **Präsenz-Verwerfer** tritt hier als Eigenschaft einer Cloud-Fabric auf, providerspezifisch statt klassentypisch.

Auf den Hinwegen nach Asien-Pazifik zeigte sich dieselbe Mechanismenklasse ein drittes Mal, nun im Transit und quellabhängig; Abbildung 6.8 fasst die Lage zusammen:

Quelle	Mumbai	Singapur	Tokio	Osaka	Seoul	Sydney
mcs	✓	✓	×	✓	×	✓
hel (Pilot)	·	·	×	·	×	·
1blue	✓	✓	×	✓	×	×
aws-us (Pilot)	·	·	✓	✓	✓	✓

✓ Surplus-Klasse zugestellt; × Surplus-Klasse verworfen (Baselines kamen in allen Lanes vollzählig an);
 · nicht gemessen; dünner Rahmen: Pilotbeleg ohne Mitschnitt ($n = 6$ je Lane), sonst Zellenbeleg mit Mitschnitten ($n = 20$ je Zelle und Richtung)

Abbildung 6.8: Schicksal der Surplus-Klasse auf den Hinwegen nach Asien-Pazifik je Quelle

Aus Abbildung 6.8 geht hervor, dass die Opfermenge von der Quelle abhängt. In den drei Eskalationsmatrizen kamen von mcs nach Tokio und Seoul sowie von 1blue nach Sydney jeweils alle Baselines an, aber keine der fünf Surplus-Zellen; die jeweilige Gegenrichtung blieb mit 120 von 120 Paketen sauber, zusammen also mit 360 von 360 Paketen. Die Helsinki-Piloten nach Tokio und Seoul lieferten je drei Baselines, aber keines der je sechs Optionsdatagramme, während die Amazon-Piloten nach Tokio, Seoul, Sydney und Osaka je drei Baselines und sechs Optionsdatagramme vollständig zustellten. Zwei Kontrollen stützen die Deutung: Osaka teilt mit Tokio die sichtbare AS-Kette und bleibt dennoch transparent, sodass die sichtbare Kette allein den Unterschied nicht erklärt, das Attributionsintervall mit Endpunktmitschnitten aber auch nicht weiter zu trennen ist; und die Sydney-Anomalie blieb im zeitversetzten Gegenteil quellabhängig. Interkontinentale Erreichbarkeit der Surplus Area ist damit für die beobachteten Messfenster eine Eigenschaft des konkreten Paares aus Quelle und Ziel, nicht des Ziels allein.

6.2.4 Mechanismenklassen und Folgen

Aus den Vorversuchen und Messungen sind drei wiederkehrende Mechanismenklassen entstanden. Abbildung 6.9 zerlegt einen Messpfad aus Sicht der beiden Messpunkte in Kanten und ordnet die Klassen den beobachteten Attributionsintervallen zu

(Abschnitt 4.3); die Zuordnung ist kein Gerätenachweis, und keine Klasse ist an die gezeigten Kanten gebunden, denn Kanten können Verkehr quellabhängig behandeln:

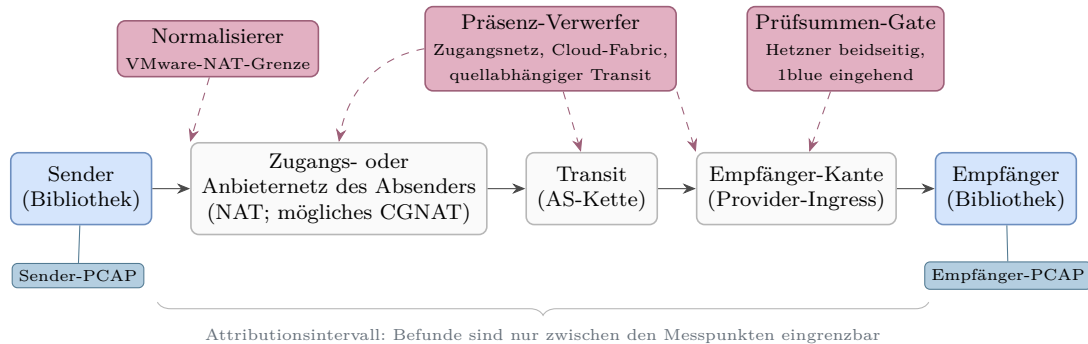


Abbildung 6.9: Kantenmodell eines Messpfads mit den beobachteten Attributionsintervallen der drei Mechanismenklassen

Abschnitt 25.6 des RFC beschreibt sowohl das Entfernen von Optionen durch UDP-Vermittler als auch das Verwerfen optionsbehafteter Datagramme auf bestimmten Pfaden [2, Sec. 25.6]. Dazu passen der **Normalisierer** an der Grenze aus VMware-NAT und VMnet mit der Ergebnisklasse entfernt, das **Prüfsummen-Gate** in den Randintervallen der beiden Hetzner-Endpunkte und am 1blue-Eingang mit der Ergebnisklasse verworfen für alle faltungsungültigen Datagramme sowie der **Präsenz-Verwerfer** in drei Habitaten, nämlich Zugangsnetz, Cloud-Fabric und quellabhängigem Transit, mit der Ergebnisklasse verworfen für die gesamte Klasse. Die Amazon-seitigen Intervalle und der reine Amazon-Backbone zeigten kein Prüfsummen-Gate. Das macht die vollständigen Pfade zu Hetzner und von Amazon zu 1blue nicht gatefrei; dort begrenzen die Endpunktmessungen das Gate auf die jeweils entfernte Seite oder den letzten Transitabschnitt. Dazu tritt der Maskierungseffekt als Wechselwirkung: Ein Normalisierer vor einem Präsenz-Verwerfer entfernt dessen Auslöser und macht den nachgelagerten Verwerfer dadurch unsichtbar. Was das für ein einzelnes Datagramm bedeutet, bündelt Abbildung 6.10 am Beispieldatagramm aus Kapitel 5 und seinen zwei härtesten Gegenstücken:

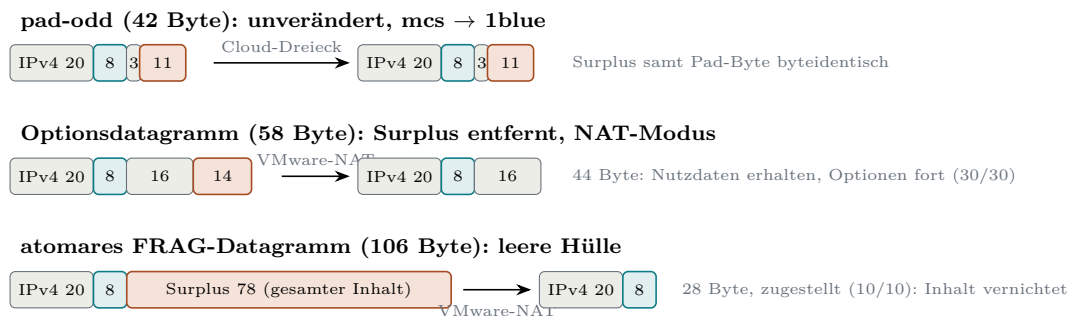


Abbildung 6.10: Drei beobachtete Schicksale von Beispieldatagrammen auf realen Pfaden

Aus Abbildung 6.10 geht hervor, dass zwischen Erhalt und Verlust eine dritte, schwer erkennbare Form liegt: das zugestellte leere Datagramm, bei dem der Inhalt ohne Verlustsignal verschwindet. Regelkonforme Optionsdatagramme blieben im europäischen Cloud-Dreieck und auf den untersuchten Amazon-Pfaden erhalten; sie scheiterten dagegen am Mobilfunkzugang, im Google-Messintervall und auf einzelnen Transitwegen, und ein regelwidriges Pad scheiterte zusätzlich an einem Prüfsummen-Gate. Ob die Anforderungen des Standards unter diesen Bedingungen erfüllbar bleiben, bewertet der folgende Soll-Ist-Abgleich.

6.3 Soll-Ist-Abgleich mit RFC 9868

FF1 fragt, welche Anforderungen von RFC 9868 sich unter Linux und IPv4 im Benutzerraum vollständig, teilweise oder nicht erfüllen lassen (Abschnitt 1.3). Bewertungsgrundlage sind die 67 Zeilen der Konformitätsmatrix aus Abschnitt 4.2. Nach dem in dieser Arbeit gewählten Verfahren werden normative Anforderungen einschließlich unmarkierter normativer Sätze bewertet (Kapitel 4); nicht gewählte MAY-Funktionen und Aussagen außerhalb des gewählten Umfangs bleiben sichtbar, zählen aber weder als erfüllt noch als technisch unmöglich (Kapitel 3). Die FR- und NFR-Kennungen dienen als Nachweisindex und nicht als zweiter Nenner; 58 der 67 Matrixzeilen tragen im eingefrorenen Repository-Arbeitsindex mindestens eine solche Kennung. Je bewerteter Zeile folgt der Nachweis einer festen Reihenfolge: zuerst die Notiz der Matrixzeile, dann die verknüpfte Anforderungskennung, dann Quelltext und Test, zuletzt, wo vorhanden, das Realpfadszenario. Damit ist auch die aus Kapitel 5 übergebene Frage zu beantworten, ob die zusammengefassten Empfangsberichte für die Messkampagnen ausreichen: allein nicht. Weil ein Pad-Fehler und eine fehlgeschlagene OCS-Rechnung im selben Status zusammenfallen und verworfene Ausgänge als „kein Datagramm“ erscheinen, stützt sich jeder Realpfadnachweis auf das Kampagnenpaket aus Mitschnitten, Sendemanifest und Auswerterlauf; ein Ausbleiben (etwa der Reassemblierungs-Timeout, der keine eigene Verwerfmeldung erzeugt) ist nur über diese Negativevidenz messbar.

Die am Primärtext korrigierte Einzelzuordnung aller 67 Zeilen zeigt Tabelle A.1 in Anhang A. Tabelle 6.2 fasst die Summen zusammen.

Tabelle 6.2: Summen der Konformitätsmatrix

Größe	Anzahl
Matrixzeilen insgesamt	67
anwendbar oder gewählt	57
technisch vollständig erfüllbar	57
technisch teilweise oder nicht erfüllbar	0
vollständig umgesetzt	50
teilweise umgesetzt	7
nicht anwendbar, nicht gewählt oder Leitlinie	10

Von den 67 Zeilen sind 57 anwendbar oder für die Referenzimplementierung gewählt. Die Einzelprüfung findet für diese 57 Zeilen keine technische Grenze des gewählten Rahmens; Spalte T enthält deshalb 57-mal V und zehnmal n.a., die Kategorien „teilweise“ und „nicht erfüllbar“ bleiben bei der technischen Erfüllbarkeit leer. Der Implementierungsstand ist davon zu trennen: 50 Zeilen sind vollständig und sieben teilweise umgesetzt, zehn Zeilen sind nicht anwendbar, nicht gewählt oder reine Leitlinie. Die sieben Teilumsetzungen sind die Zeilen 199, 216, 218, 219, 221, 222 und 225; keine davon belegt eine Grenze von Linux, IPv4 oder Raw Sockets. Die zehn nicht anwendbaren oder leitenden Zeilen sind 188, 206, 207, 208, 209, 211, 223, 226, 227 und 228.

Eine leere Kategorie „nicht erfüllbar“ besagt nur, dass im gewählten Prüfraster kein grundsätzlicher technischer Hinderungsgrund gefunden wurde: Die harten Grenzen des Benutzerraum-Zugangs treffen den Betrieb des Sendewegs, aber keine einzelne Normaussage der Matrix. Sie belegt weder volle RFC-Konformität noch Fehlerfreiheit und gilt nur für Linux, IPv4, Benutzerraum und den gewählten Funktionsumfang.

Davon getrennt bleibt der Implementierungsnachweis des geprüften Protokollkerns belastbar: Ortung und Ausrichtung der Surplus Area, die OCS-Arithmetik samt Nullregeln, die TLV-Rahmung mit ihren Längen- und Ordnungspflichten, die acht must-support-Optionen und die FRAG-Verarbeitung einschließlich Reassemblierung. Er stützt sich auf die sechs Ebenen der Testarchitektur aus Abschnitt 5.6.

Die Realpfadprüfung aus Abschnitt 6.2 ergänzt 15 Empfängerklassen. Die Negativkampagne auf der Empfangsseite erfüllte in allen 15 Klassen die im Auswerter hinterlegten Zellkriterien, darunter das Verwerfen bei Pad- und OCS-Fehlern, das stille Übergehen unbekannter SAFE-Optionen und die Nutzdatenkürzung bei einer UNSAFE-Option. Diese Klassen enthalten auch Kontrollen und bilden nicht alle be-

rührten RFC-Anforderungen ab; insbesondere verarbeitet S-21 die NOP-Leiter trotz Überschreiten der lokalen Meldeschwelle weiter. Die Robustheitswelle ergänzte EOL-Füllbytes, NOP-Leitern, erweiterte Längenfelder, ein atomares FRAG-Einzelfragment, parallele Fragmentsätze und ungleiche Optionssätze je Fragment. Auf jedem der sechs vollständigen Hostpaare liefen dieselben 18 Treiberszenarien, nämlich auf dem Referenzpaar 1blue $\leftrightarrow$ mcs, auf beiden Helsinki-Paaren und transatlantisch auf drei aws-Paaren. Die Katalogtreue einzelner Kennungen ist davon getrennt zu bewerten (Abschnitt 6.1).

Sieben Matrixzeilen sind **teilweise** umgesetzt: 199, 216, 218, 219, 221, 222 und 225. Zeile 199 korrigiert die damalige Repository-Selbstauskunft **Covered yes**: Die tiefe Schnittstelle verlangt einen Cache je Adress-und-Port-Paar als dokumentierte Aufruferpflicht, die mitgelieferte Komfortebene bindet ihren Empfänger jedoch nur an die Ports, sodass Datagramme verschiedener Adresspaare dieselbe Mengengrenze teilen (Abschnitt 5.7). Bei den Zeilen 216 und 221 liefert die öffentliche API für ignorierte oder fehlgeschlagene Optionen Kind und Status, aber nicht immer die empfangenen Parameter. Die Empfangs-API prüft Pflichtoptionen zudem gebündelt je Datagramm oder Fragmentsatz und bietet weder getrennte Regeln je Fragment noch eine Ausschlussliste einzelner Optionsarten (Zeilen 218 und 219). Der Sende-API fehlen getrennte Optionslisten je Fragment und eine vom Aufrufer gewählte Mindestlänge (Zeile 222). Zeile 225 fasst teilweise umgesetzte Schutzgrenzen zusammen. Das sind Implementierungs- und Entwurfsentscheidungen, keine technischen Grenzen von Linux, IPv4 oder Raw Sockets.

Die NOP-Regel (Zeile 189) gilt nach der Primärtextprüfung dagegen als vollständig erfüllt. Abschnitt 11.2 verlangt vom Empfänger nur, dauerhaft auftretende Läufe von mehr als sieben aufeinanderfolgenden NOP zumindest gelegentlich zu melden, und kein Beenden der Verarbeitung; die Bibliothek erkennt und meldet solche Läufe. Die bedingte Empfehlung zur Ressourcenbegrenzung steht in Abschnitt 25.2 und ist dort als Zeile 225 geführt, sodass dieselbe Lücke nicht zweimal zählt [2, Sec. 11.2 und 25.2].

Außerhalb der Matrix treten die erwarteten Betriebsgrenzen hinzu: Der Sendeweg trägt bis exakt 1500 Byte Gesamtlänge und scheitert darüber am Kernel. Je Richtung wurden elf Längensufen bis 1500 Byte geprüft, mit je drei Baseline- und drei Optionsdatagrammen, also 66 von 66 Zustellungen; in den fünf Längensufen ab 1504 Byte scheiterten je Richtung alle 30 Sendeversuche mit **EMSGSIZE**. Dieses Bild wiederholte sich auf allen sechs Hostpaaren einschließlich der transatlantischen. Eine Pfad-MTU-Ermittlung fehlt, und die Cache-Bereinigung bleibt aufrufergesteuert. Die Bibliothek setzt den geprüften Protokollkern damit um, bildet aber nicht alle bewerteten API-Anforderungen vollständig ab.

Nicht anwendbar oder leitend sind zehn Zeilen: 188, 206, 207, 208, 209, 211, 223, 226, 227 und 228. Dazu gehören die nicht gewählte Empfangsprüfung der Bytes nach EOL, die nicht implementierten Optionen TIME und EXP, die reservierten AUTH- und UNSAFE-Arten, die Entwurfs- und Namensregeln für neue Optionsarten, die Multicast-Betrachtungen sowie die API-Leitlinie zu Optionsreihenfolge und Fragmentgrenzen; letztere bleibt Leitlinie, weil der Primärtext sie ausdrücklich als nicht beabsichtigte Nutzersteuerung formuliert [2, Sec. 15]. Zeile 226 betrifft die bedingte Empfehlung zur empfangsseitigen Referenzordnung aus Abschnitt 25.2: Das vorangehende **MAY** wurde nicht gewählt, und die kanonische Sendereihenfolge erfüllt keine empfangsseitige Ordnung. Die unmarkierte Sende-API-Form ist dagegen anwendbar und teilweise umgesetzt (Zeile 222). Eine formgültige TIME-Option lief auf dem Realpfad als unbekannte SAFE-Option still mit; das belegt das stille Übergehen einer unbekannten SAFE-Option, nicht die Umsetzung von TIME, und ändert die Umfangseinstufung nicht.

Vier **Differenzen zwischen Spezifikation, Modell, Programm und Laufzeit** verdienen eigene Sätze. Erstens ist die Kombination aus UDP-Prüfsumme null und OCS null aus dem Benutzerraum exakt konstruierbar und wurde auf der Leitung nachgewiesen; in der Matrix aus UDP-Prüfsumme und OCS wird diese Zelle regelkonform verarbeitet. Das geprüfte Datagramm trug weder eine APC noch den Integritätsschutz einer anderen Schicht, sodass seine UDP-Nutzdaten und seine Surplus Area ungeschützt blieben. Eine APC kann allgemein die UDP-Nutzdaten zusätzlich schützen, aber weder die Surplus Area noch UDP-Kopf und Pseudo-Header [2, Sec. 11.3]; weiteren Schutz liefert nur eine andere Schicht. Zweitens löst Erratum 8834 den Widerspruch zwischen den Abschnitten 10 und 14 zugunsten des Sec.-10-Verfahrens auf (Kapitel 3); die Laufzeit folgt dem nachweislich, denn im realen Überlängenfall wurde ein gültiges REQ vor der defekten Option nicht gemeldet, sondern der gesamte Optionsbereich verworfen. Drittens behandelt der Parser ein erweitert kodierte Längenfeld unter 255 strenger, als der RFC dem Empfänger vorschreibt; das ist als lokale Politik dokumentiert und in Tabelle A.1 keine Abweichung, sondern eine ausgewiesene Verschärfung. Viertens beweisen die Lean-Theoreme Aussagen über Modelle der Wire-Regeln, nicht über das laufende Programm; die Brücke bleibt Prüfsache der Tests (Abschnitt 5.6), weshalb diese Arbeit keine formale Konformitätsaussage erhebt.

Vier **Abweichungen** traten im Prüf- und Messbetrieb auf. Erstens beendete sich der Raw-Empfänger auf öffentlichen Hosts still beim ersten fremden UDP-Paket; Ursache war ein als Ende gewerteter Filterausgang, und die Korrektur besteht aus dem Weiterlesen nach gefiltertem Rauschen und einer Gesamtfrist.⁴ Die externe

⁴ Commits `f3bf430` und `8f8c11b` im Implementierungsrepositorium.

Kampagne vom 10.08. und die bidirektionale Kampagne vom 11.08. nutzten noch 7b11140, also einen Stand vor beiden Korrekturen. In der bidirektionalen Kampagne hielt eine äußere Neustartschleife den Empfänger verfügbar; die Ankunftszeiten beruhen deshalb auf den PCAP-Dateien, während die lückenhaften JSONL-Dateien dort nur die Decodierung belegen. Die vollständigen P0-, P1- und P2-Kampagnen ab dem 14.08. liefen mit beiden Korrekturen. Zweitens überstand der Um-eins-Fehler der Surplus-Ortung 23 Einzelfalltests und fiel erst der Zweitprüfung im Pull-Request auf; als Korrektur wurden Property-Tests und Fuzz-Ziele je Parsefläche verpflichtend. Drittens fand der RFC-Abgleich des Abschlussschritts trotz grüner Prüfkette weitere Konformitätslücken, darunter die Behandlung unbekannter UNSAFE-Optionen vor einem FRAG; die Korrektur erfolgte gesammelt und real nachgeprüft. Viertens verfehlte ein AWS-Lauf des Szenarios S-50 das vorregistrierte Zeitkriterium, obwohl der Kampagnentreiber Status 0 meldete, denn die Auswerter für P0, P1 und P2 überführen semantische Abweichungen nicht in einen Fehlerstatus. Zusammen zeigen die vier Fälle, dass eine grüne Prüfkette weder Konformität noch die vollständige Erfüllung aller Zellkriterien garantiert (Abschnitt 5.1).

Damit kehrt die Bewertung zu der in Kapitel 2 offen gelassenen Frage zurück, was aus dem Verlust einzelner Fragmente wird, den auch FRAG nicht behebt. Die Messungen beantworten sie in vier Schritten. Ein fehlendes Fragment lässt seinen Satz liegen, ohne Nachbarn zu stören: In der langen Serie aus 300 Zwei-Fragment-Sätzen mit gezielt zurückgehaltenem Terminalfragment kamen exakt die geplanten 270 Sätze an, die 30 beschädigten verhungerten still, und die Quote blieb über die Serie drittelgenau konstant (570 von 570 Fragmenten auf dem Draht, keine Drift). Der Reassemblierungs-Timeout wirkt real, ist aber nur über Negativevidenz messbar, denn das Auslaufen erzeugt keine eigene Verwerfmeldung. Die Eindeutigkeit der Identification ist eine markierte Senderpflicht aus Abschnitt 11.4 [2, Sec. 11.4], und der Versuch mit gleicher Kennung verletzt sie absichtlich: Bei komplementären, nicht überlappenden Fragmenten setzte der Empfänger ein nie gesendetes Datagramm zusammen und lieferte es kommentarlos aus. Das ist keine Abweichung der Implementierung vom RFC, sondern eine Robustheitsgrenze gegenüber nichtkonformen Senderkennungen, denn der RFC schreibt dem Empfänger keine zusätzliche Erkennung solcher Kollisionen vor. Die APC bietet dafür eine optionale protokollinterne Erkennung beschädigter Nutzdaten; schlägt sie fehl, wird der Fehlschlag gemeldet und die Nutzlast standardmäßig dennoch zugestellt.

Fragmentierung und Soft-State lassen sich in vier getrennte Grenzen zerlegen, die Tabelle 6.3 Norm, Umsetzung und Beobachtung direkt zuordnet.

Aus Tabelle 6.3 geht hervor, dass der RFC weder die Kapazität noch das Überlaufverfahren festlegt [2, Sec. 11.4, 25.4]. Die 16 nicht zugestellten Sätze je S-50-Lauf

Tabelle 6.3: Fragmentierung und Soft-State: Norm, Umsetzung und Messbeleg

Grenze	RFC 9868	Implementierung	Beobachtung
Menge und Schutz	Reassemblierungsspeicher begrenzen; kein Zahlenwert (Sec. 11.4, 25.4)	64 unvollständige Sätze je Cache; Bestandsverkehr bleibt erhalten	S-50: je Lauf 64 von 80 Sätzen, 16 ohne Zustellung; 12 von 12 Läufen wie vorhergesagt; Kontrollen und atomares Fragment zugestellt
Zeit und Bereinigung	Standardtimeout höchstens zwei Minuten; kein ICMP (Sec. 11.4)	Grenze bei 120 Sekunden; Ablauf wird bei Einfügung oder durch <code>gc</code> wirksam; kein Hintergrundlauf	S-50 nutzte 60 Sekunden; in einem Lauf lag ein Terminal außerhalb von 30 Sekunden und wurde noch zugestellt
Trennung	Grenzen nicht socketübergreifend berechnen (Sec. 11.4)	ein Cache je <code>Peer</code> ; der Raw-Empfänger filtert nur Ports, daher können verschiedene Adresspaare dieselbe Grenze teilen	Teilumsetzung der Matrixzeile 199; keine Messung konkurrierender Adresspaare
Auslieferung	einzelne Fragmente nie liefern; bei Fehlschlag kein Leerdatagramm (Sec. 11.4)	Buffered oder Dropped ; Berichte erst nach vollständiger Reassemblierung	S-49: 270 von 300 Sätzen geliefert, 30 unvollständige Sätze ohne Nutzerrahmen

folgen daher der gewählten Abweisungspolitik: Bei vollem Cache wird ein Fragment, das einen neuen Satz eröffnen würde, nicht aufgenommen. Nach Freigaben können spätere Terminalfragmente neue unvollständige Zustände anlegen; zugestellt wird keiner der 16 Sätze. Anwendungen müssen Ausbleiben, Kennungskollisionen und Cacheüberlauf deshalb selbst behandeln. APC und Optionsberichte liefern nur die beschriebenen Signale; sie ersetzen keine Ende-zu-Ende-Behandlung.

Zwei Geltungsgrenzen rahmen dieses Ergebnis. Alle Realpfadaussagen gelten für saubere, kalibrierte Pfade; das Verhalten unter stochastisch gestörtem Pfad wurde nach dem begründeten Verzicht aus Abschnitt 6.1 nur mechanismenweise geprüft (Verlust, Timeout, Duplikat, Umordnung, Quote, Cachegrenze), nicht als Mischung. Und die Empfängerpflichten für verfälschte Pads und Prüfsummen sind auf Pfaden mit Prüfsummen-Gate nur über die beschriebenen pfadneutralen Konstruktionen messbar; für den Pad-Fall ist das sogar beweisbar, weil ein regelwidriges Pad die Legacy-Faltung zwingend bricht. Damit ist der derzeitige Zuordnungsstand für FF1 zusammengefasst; die Einzelwerte stehen in Tabelle A.1. Wie diese Zuordnung zustande kam, und zwar unter durchgehender Beteiligung von Sprachmodellen, beschreibt der folgende Abschnitt.

6.4 Reflexion der KI-gestützten Arbeitsweise

Dieser Abschnitt ergänzt die Offenlegung aus Kapitel 3 um beschreibende Kennzahlen der Zusammenarbeit. Er ist rein deskriptiv: Er berichtet belegte Fälle mit ihren Ausgängen und erhebt keine Wirksamkeitsaussage, denn eine Prüfung ohne Sprachmodelle als Vergleichsgruppe existiert nicht. Die Rollen sind die aus Kapitel 3: das umsetzende Werkzeug für Entwürfe und Quelltext, der getrennte Zweitprüfer mit dem Auftrag, Fehler und Gegenbeispiele zu suchen, und der Autor, der entscheidet und verantwortet. Grundlage ist ein Befundregister aus zwölf Fällen, jeder mit öffentlichem Anker im Implementierungsrepositorium oder in den versiegelten Archiven;⁵ das Register ist eine geprüfte Auswahl aus 27 Pull-Requests und Messläufen, kein Vollbestand. Vier Fallgruppen tragen das Bild:

- **Übernommene Funde vor dem Merge.** Im Wire-Modell-Review wurden zwei Funde übernommen (eine erzwungene Längeninvariante der Prüfsummenrechnung und eine geprüfte Längen-Verengung, Fix im Pull-Request dokumentiert); beim Aufbau des Lean-Gates drei (ein erweitertes Theorem-Audit sowie zwei gegen Veränderung gepinnte Werkzeugbezüge). Der wertvollste Einzelfund gelang allein dem Zweitprüfer: der durch ein Byte-Palindrom maskierte Ausrichtungsfehler im unabhängigen Prüfprogramm der Wire-Lane (Abschnitt 5.6); ebenso fing allein die Zweitprüfung den Um-eins-Fehler der Surplus-Ortung, den 23 eigene Einzelfalltests überstanden hatten.
- **Abgelehnte Befunde.** Nicht jeder Befund wurde übernommen: Im Lean-Gate-Review lehnte der Autor einen Vorschlag mit dokumentierter Begründung ab (der bemängelte Bestand lag außerhalb des Änderungsumfangs), und im TLV-Parser-Review blieben Prüferbefunde insgesamt ohne Übernahme, weil sie den beabsichtigten Umfang betrafen und keinen Faktenirrtum zeigten. Die Ablehnungen sind in den Verläufen nachlesbar; sie gehören zum Bild derselben Arbeitsweise.
- **Prüfung nach dem Merge.** Die Reihenfolge hielt nicht immer: PR 27 wurde vier Minuten vor der Zweitprüfer-Rückmeldung zusammengeführt. Ein bereits zuvor eröffneter, unabhängiger Fix für die Speichergrenze der Fuzz-Lane wurde fünf Minuten nach PR 27 zusammengeführt; er war keine Reaktion auf dessen Befund. Die vom Zweitprüfer beanstandete Gesamtfrist des Empfangspfads folgte mit Commit `8f8c11b` drei Tage später. Auch der Schlussaudit fand trotz grüner Prüfkette weitere Konformitätslücken; Schritt 18 behob sie gesammelt (Abschnitt 5.1).

⁵ Register als `thesis/daten/ki-befundregister.csv` im Repositorium der Arbeit; die Pull-Request-Verläufe sind öffentlich, Existenz, Titel, Merge-Zeitpunkte und Kommentarfolgen wurden am 26.08.2026 nachgeprüft.

- **Gemeinsam übersehene Fehler.** Der Zählfehler der bidirektionalen Kampagne betraf 15 statt 18 FULL-Kontrollpakete in Richtung A; er passierte drei Prüfende und wurde erst nachträglich als Erratum im versiegelten Archiv berichtigt. Ein Faktencheck dieses Textes widerlegte zudem eine eigene Kapitelaussage am Quelltext (der nur teilweise eingelöste Cachevertrag aus Abschnitt 6.3), und die Nachprüfung der Grundlagen führte zu zwei ausdrücklich als lokale Politik dokumentierten Entscheidungen samt einer geschlossenen Testlücke. Außerhalb des Quelltexts ließ ein Abrufwerkzeug den entscheidenden Satz eines geprüften Abstracts aus, sodass die Erstbewertung falsch war und nach einem Autorenhinweis berichtigt wurde (Kapitel 3).

Als beschreibende Kennzahlen des Registers ergibt das: Zehn der zwölf Fälle endeten mit Übernahme, einer teils mit Übernahme und teils mit begründeter Ablehnung, einer vollständig mit Ablehnung; sechs Fälle lagen vor dem Merge, fünf danach, einer außerhalb des Repositoriums. Drei Funde stammen allein vom Zweitprüfer, die Übernahmenserie eines Falls allein vom führenden Werkzeug. Das Register kennzeichnet nur den Kampagnen-Zählfehler ausdrücklich als zunächst von allen drei Beteiligten übersehen. Für die frühen Projektphasen fehlen vollständige Modell- und Konfigurationsangaben; diese Fälle tragen den Vermerk der lückenhaften Provenienz und werden nicht nachträglich vervollständigt (Kapitel 3). Ebenso bleibt festgehalten, dass die KI-Prüfungen angefordert, aber nicht technisch erzwungen waren und Sammelcommits die innere Reihenfolge eines Schritts verschmelzen.

Das Schlussbild ist eine gemischte Bilanz, und genau sie trägt die Glaubwürdigkeit der Methode: Die Arbeitsweise erzeugte nachweisbare Korrekturen bis hinein in Prüfprogramm und Kapiteltext, sie produzierte Fehlbefunde, die der Autor mit Begründung ablehnte, und sie übersah Fehler, die erst eine andere Ebene fand. Mehr als diese Beschreibung gibt der Aufbau nicht her, und mehr behauptet diese Arbeit nicht.

6.5 Diskussion und Grenzen der Aussagekraft

Technische Umsetzbarkeit am Endpunkt und Durchlässigkeit des Pfads sind getrennte Eigenschaften (Tabelle A.1). Die Pfadmessungen zeigen zugleich, dass selbst regelkonform erzeugte Optionsdatagramme nicht auf jedem Pfad zugestellt werden: Ob eine Surplus Area ankommt, entscheiden Normalisierer, Prüfsummen-Gates und Präsenz-Verwerfer entlang des konkreten Pfads. Erreichbarkeit ist deshalb paarbezogen, also eine Eigenschaft aus Quelle und Ziel und nicht des Ziels allein, und jeder Befund gilt je Paar, Richtung und Messfenster. Umgekehrt stützen dieselben Gates den Entwurf der OCS: Ihre Prüfsummenneutralität lässt ein regelkonformes Datagramm an einem

Gate dieses Typs passieren, während ein regelwidriges Pad die Zustellung verhindern kann.

Für die Betroffenen folgt daraus zweierlei. Anwendungen müssen auf vergleichbaren Pfaden mit klassenbasiertem Totalverlust und mit stiller Inhaltsentfernung rechnen, denn ein empfangenes leeres Datagramm zeigt nicht an, dass sein Inhalt unterwegs entfernt wurde; Ende-zu-Ende-Bestätigung, die Auswertung der Empfangsberichte und im Zweifel eine Kapselung, die die Klasse vor dem Pfad verbirgt, mindern dieses Risiko. Wer Netze betreibt, sollte beide Richtungen getrennt prüfen, weil eine Adressumsetzung einen dahinterliegenden Verwerfer maskieren kann. Beide Folgerungen ziehen die Messbefunde dieser Arbeit; Betreiberpflichten von RFC 9868 sind sie nicht.

Diesen Aussagen stehen klare Grenzen gegenüber. Jeder Pfadbefund gilt für das beobachtete Paar, die Richtung und das Messfenster; eine Aussage über „das Internet“ trifft diese Arbeit nicht, und als Zugangsnetz wurde nur ein Mobilfunk-Hotspot-Pfad untersucht. Ohne Messpunkt im Betreibernetz endet jede Ursachenzuordnung an einem Intervall zwischen zwei eigenen Messpunkten; Formulierungen wie Hetzner-Kante oder NAT-Vermittler sind sparsame Modelle, keine Gerätenachweise. Die Realpfade waren kalibriert, und die Störmechanismen wurden nach dem begründeten Verzicht einzeln statt gemischt geprüft. Auf der Prüfseite beweisen die Lean-Theoreme Modelle statt des Programms, findet Fuzzing Fehler statt Fehlerfreiheit, und die Folge der Modellausgaben früher Projektphasen ist nur lückenhaft überliefert; eine Wirksamkeitsaussage zur KI-gestützten Arbeitsweise folgt daraus nicht (Abschnitt 6.4).

Auf Implementierungsseite bleiben die sieben Teilumsetzungen, darunter die auf der Komfortebene nur teilweise eingelöste Paarbindung des Reassemblierungs-Caches und die für ignorierte oder fehlgeschlagene Optionen nicht bereitgestellten Parameter. Hinzu tritt eine Lücke der Prüfkette: Die Auswerter für P0, P1 und P2 überführen semantische Abweichungen nicht in einen Fehlerstatus. P1 und P2 sammeln solche Befunde bereits, beenden den Lauf aber unabhängig von deren Zahl mit Status 0, und P0 gibt negative Prüfergebnisse aus, ohne den Rückgabewert zu ändern; das ist kein Einzelfall, sondern zeigte sich unter anderem bei S-14, S-36 und S-50. Ein Status 0 belegt daher nur den Abschluss des Auswerterlaufs, nicht die Erfüllung aller Zellkriterien. Mit diesen Ergebnissen und Grenzen ist die Evaluation vollständig; die wörtliche Beantwortung beider Forschungsfragen und die daraus folgenden Empfehlungen übernimmt Kapitel 7 (Abschnitt 7.1).

7. Fazit und Ausblick

7.1 Beantwortung der Forschungsfragen

FF1 fragt, welche Anforderungen von RFC 9868 sich unter Linux und IPv4 im Benutzerraum vollständig, teilweise oder nicht erfüllen lassen. Die Zuordnung im geprüften Umfang steht in Tabelle 7.1. Die sieben Teilumsetzungen sind Implementierungs- und Entwurfsgrenzen, keine nachgewiesenen Plattformgrenzen. Der Index selbst mischt markierte BCP-14-Anforderungen, unmarkierte normative API-Formen, Beschreibungen, Leitlinien und zusammengesetzte Aussagen; er ist ein Arbeitsindex und kein Verzeichnis aller Pflichten des Standards. Das Ergebnis beantwortet FF1 deshalb auf genau diesem erklärten Prüfraster und beweist weder die Satzgenauigkeit der Zerlegung noch vollständige Konformität mit allen Aussagen von RFC 9868.

FF2 fragt, in welchem Umfang die Surplus Area auf realen Pfaden bis zur Gegenstelle erhalten bleibt. Im europäischen Messdreieck und auf den drei transatlantischen Hostpaaren zwischen den AWS-US-Endpunkten und den europäischen Gegenstellen blieb sie in den jeweiligen Messfenstern erhalten. Die VMware-NAT wirkte als **Normalisierer** und entfernte sie. Der gebridgte Mobilfunk-Hotspot-Pfad und die Google-Fabric wirkten als **Präsenz-Verwerfer** und verwarfen die gesamte Klasse in beiden Richtungen; auf einzelnen Hinwegen zu AWS-Regionen in Asien-Pazifik trat derselbe Totalverlust quellabhängig im Transit auf. Die beobachteten **Prüfsummen-Gates** verwarfen Datagramme, deren Einerkomplementsumme über Pseudo-Header und gesamte IP-Nutzlast nicht aufging; regelkonforme Datagramme mit Null-Pad und gültiger OCS passierten sie. Jede dieser Aussagen gilt nur für das beobachtete Paar, die Richtung und das Messfenster.

Tabelle 7.1: Synthese der Forschungsfragen im geprüften Umfang

FF	Ergebnis	Beobachtete Grenze	Geltungsbereich
FF1	57 technisch vollständig erfüllbar; 0 teilweise; 0 nicht erfüllbar	Referenzimplementierung: 50 vollständig, 7 teilweise; 10 nicht anwendbar oder Leitlinie	67-zeiliger Arbeitsindex; Linux, IPv4, Benutzerraum, Raw Sockets
FF2	Surplus Area je nach Pfad erhalten, entfernt oder als Klasse verworfen	Normalisierer, Prüfsummen-Gates und Präsenz-Verwerfer	beobachtetes Paar, Richtung und Messfenster

Aus Tabelle 7.1 geht hervor, dass beide Ergebnisse unabhängig voneinander ausfallen. Die Leitfrage, wie sich UDP um Transportoptionen erweitern lässt, ohne seine grundlegenden Eigenschaften aufzugeben, ist damit zweigeteilt zu beantworten. Auf der Entwurfsseite bleibt der UDP-Header unverändert, die Optionen liegen in der Surplus Area, und ein Empfänger liefert die Nutzdaten auch dann aus, wenn eine SAFE-Option fehlschlägt; Nachrichtenorientierung, Verbindungslosigkeit und der Verzicht auf Sequenznummern, Fenster und Zeitgeber bleiben erhalten (Abschnitt 2.2.2). Die einzige belegte Ausnahme ist die Reassemblierung von FRAG-Datagrammen: Sie legt am Empfänger einen zeitlich und mengenmäßig begrenzten Zustand an, und die Messungen zeigen dessen Verhalten an der Mengengrenze und am Zeitablauf (Tabelle 6.3). Auf der Pfadseite zeigen die Messungen, dass die Abwärtskompatibilität des Wire-Formats keine allgemeine Zustellbarkeit garantiert. Technische Umsetzbarkeit am Endpunkt und Durchlässigkeit des Pfads sind unabhängige Eigenschaften.

7.2 Ausblick

Für eine Weiterentwicklung der Bibliothek liegen die sieben Teilumsetzungen nahe: der fehlende Geltungsbereich für Optionen vor oder nach der Fragmentierung, eine vom Aufrufer wählbare Mindestlänge der Sende-API, quellspezifische Empfangsregeln, die Bereitstellung der empfangenen Parameter auch für ignorierte oder fehlgeschlagene Optionen und eine klar dokumentierte Trennung des Reassemblierungsspeichers zwischen Socket-Kontexten. Die hohe Sende-API könnte zusätzlich die Leitlinie zur paketweisen Fragmentierungssteuerung abbilden. Eine Prüfkette, die semantische Abweichungen ihrer Auswerter in einen Fehlerstatus überführt, würde den Rückgabewert wieder aussagekräftig machen.

Für künftige Untersuchungen bleiben die Grenzen dieser Arbeit die naheliegenden Ansatzpunkte: eine unabhängige Implementierung, weitere Zugangsnetze, IPv6, andere Messzeitpunkte und das Zusammenspiel mit RFC 9869 [3] könnten Interoperabilität und zeitliche Stabilität der hier berichteten Befunde einordnen. Wer auf der lokalen 248-Paket-Voranalyse aufbauen will, sollte sie zuvor versiegeln; portable innere Manifeste würden die äußeren Archivprüfsummen sinnvoll ergänzen.

7.3 Schlusswort

Die Arbeit belegt, dass die Umsetzung der UDP-Optionsmechanik im Benutzerraum und ihre Zustellbarkeit auf realen Pfaden getrennte Fragen sind. Eine funktionierende Referenzimplementierung beseitigt die pfadabhängigen Normalisierer und Verwerfer nicht. Die getrennte Bewertung verhindert, dass ein grüner Implementierungstest als

Netzbeleg oder eine erfolgreiche Pfadmessung als Konformitätsbeleg missverstanden wird.

A. Konformitätsmatrix

Die folgende Tabelle enthält die am Primärtext geprüfte Einzelzuordnung aller 67 Zeilen der Konformitätsmatrix. Die Summen und ihre Auswertung stehen in Abschnitt 6.3; Tabelle 6.2 fasst sie zusammen.

Die Tabelle trennt Anwendbarkeit (Spalte A), Implementierungsstand (Spalte I) und technische Erfüllbarkeit (Spalte T). In Spalte A steht J für anwendbar und N für nicht anwendbar. In Spalte I steht V für vollständig und P für teilweise umgesetzt; nicht anwendbare Zeilen erhalten N. In Spalte T steht V für vollständig, P für teilweise und X für technisch nicht erfüllbar; n.a. kennzeichnet Zeilen ohne technisches Urteil. Die Belegspalte kürzt die Module ab: W steht für `src/wire`, O für `src/options`, R für `src/recv`, F für `src/frag`, A für die Schnittstelle unter `src/api` und K für die Kampagnenartefakte; jede Modulangabe schließt die zugehörigen Tests ein. Code und eigene Tests gelten dabei nicht als voneinander unabhängige Belegfamilien.¹

Tabelle A.1: Prüfung der 67 Matrixzeilen gegen RFC 9868 und den Implementierungsstand

Z.	Sec.	Normtext, gekürzt	Stufe	A	I	T	Beleg
162	10	Ungültige UDP-Länge verwerfen und melden	MUST	J	V	V	R; FR-49
163	7	Surplus Area hinter dem Ende gemäß UDP-Länge	Festlegung	J	V	V	W; FR-04
164	8	Ganze Surplus Area nutzen; OCS an erster Zweiergrenze	Festlegung	J	V	V	W, O; FR-04/19
165	8	Pad vor OCS null, sonst Optionen verwerfen	MUST	J	V	V	W, R; FR-05; S-04
166	9	OCS über Bereich und 16-Bit-Längensummand	Festlegung	J	V	V	O; FR-19; Wire-Prüfung
167	9	OCS nicht null, wenn UDP-Prüfsumme nicht null	MUST	J	V	V	O; FR-21; S-06
168	9	Nichtnull-OCS als Voreinstellung	MUST	J	V	V	O, A; FR-21
169	9	Bei OCS-Fehler Optionen und Surplus verwerfen	MUST	J	V	V	R; FR-20; S-05
170	9/14	Gültige Nutzdaten trotz OCS-Fehler standardmäßig liefern	MUST	J	V	V	R; FR-38; S-06

¹ Die Einzelzuordnung liegt als `thesis/daten/konformitaet-kategorien.csv` im Repositorium der Arbeit; ein Prüfskript erzeugt Tabelle und Datei aus demselben Bestand, in dem die drei Urteile je Zeile getrennt gespeichert sind, und prüft die Summen.

Z. Sec.	Normtext, gekürzt	Stufe	A I T Beleg
171 10	TLV-Rahmung und erweitertes Längenformat	Festlegung	J V V O; FR-07/09
172 10	NOP und EOL ohne Längenfeld	Festlegung	J V V O; FR-08
173 10	Länge unter Formatminimum verwirft alle Optionen	MUST	J V V O, R; FR-10; S-11a
174 10	Unterlauf oder Überlauf macht den Bereich ungültig	MUST	J V V O, R; FR-10/50; S-11b
175 10	Bekannte Option unter Mindestlänge verwirft alle Optionen	MUST	J V V O, R; FR-11/12
176 10	Ab 255 erweitern; kleinste Kodierung bevorzugen	MUST/ SHOULD	J V V O; FR-09/14
177 10/14	Optionen in Drahtreihenfolge verarbeiten	MUST	J V V O, R; FR-13
178 10	Acht Pflichtoptionen erkennen und bei Konfiguration erzeugen	MUST	J V V O; FR-06/14/22 bis 31
179 10	Unbekannte SAFE-Optionen still übergehen	MUST	J V V O, R; FR-11/12; S-10
180 10	Defektes FRAG wie nicht unterstützte UNSAFE-Option	MUST	J V V R, F; FR-27
181 10	Bestimmte Optionen nur einmal; erste Instanz gilt	SHOULD/ MUST	J V V O, R; FR-15
182 10	NOP darf sich zur Ausrichtung wiederholen	MAY	J V V O; FR-16
183 10	Mehrere FRAG machen den Optionsbereich ungültig	MUST	J V V R, F; FR-29
184 10	Bei UNSAFE Nutzdaten leer und Inhalt im FRAG	MUST	J V V R, F; FR-28/39
185 10	Erste unbekannte UNSAFE beendet die Verarbeitung	MUST	J V V R; FR-39; S-20
186 10	Pflichtoptionen außer NOP/EOL vor anderen SAFE senden	MUST/ MAY	J V V O, A; FR-14
187 11.1	Bei Restplatz EOL zuletzt; danach null senden	MUST	J V V O; FR-17; S-08
188 11.1	Optionale Nullprüfung nach EOL samt Verwerfregel	MAY/ MUST	N N n.a. FR-18; nicht gewählt
189 11.2	Höchstens sieben aufeinanderfolgende NOP; dauerhafte Läufe melden	SHOULD	J V V O, R; FR-16; NFR-05
190 11.3	APC als CRC32C; Fehlschlag angezeigt zustellen	Festlegung/ SHOULD	J V V O, R; FR-22/23

Z. Sec.	Normtext, gekürzt	Stufe	A	I	T	Beleg
191 11.3	Unbekannte APC-Länge wie fehlgeschlagene APC	MUST	J	V	V	O, R; FR-12/23
192 11.4	FRAG-Länge 10 oder 12 abhängig von RDOS	Festlegung	J	V	V	F; FR-26; S-19
193 11.4	Bei FRAG sind UDP-Nutzdaten leer	MUST	J	V	V	R, F; FR-28
194 11.4	Mindestens zwei Fragmente, jedes in 1500 Byte, reassemblieren	MUST	J	V	V	F; FR-35; S-frag
195 11.4	Kennung über den Timeout eindeutig; Erzeugung empfohlen	MUST/ SHOULD	J	V	V	F; FR-31; S-46
196 11.4	Überlappung abbrechen und ohne ICMP verwerfen	MUST	J	V	V	F; FR-32/48; S-45
197 11.4	Exakte Duplikate dürfen verworfen werden	MAY	J	V	V	F; FR-32; S-45 Z0
198 11.4	Standardtimeout höchstens zwei Minuten; kein ICMP	SHOULD/ MUST	J	V	V	F; FR-33; S-42
199 11.4	Speicher begrenzen und nicht socketübergreifend teilen	SHOULD	J	P	V	F, A; FR-34; NFR-06; eigene Korrektur von f847895
200 11.4	Einzelne Fragmente nie an den Nutzer geben	MUST	J	V	V	R, F; FR-31
201 11.4	Fehlgeschlagene Reassemblierung ohne Leerdatagramm	SHOULD	J	V	V	R, F; FR-33; S-49
202 11.5	MDS verarbeiten; MDS begrenzt das Senden nicht	MUST	J	V	V	O, A; FR-24
203 11.6	MRDS-Mindestwerte und Voreinstellungen für IPv4	MUST	J	V	V	O, F; FR-35
204 11.7	REQ/RES ohne Autoantwort; RES-Token aus empfangenem REQ	Festlegung/ MUST	J	V	V	O, A; FR-25; Vertrag
205 11.7	Automatische REQ/RES-Schicht standardmäßig aus	MUST	J	V	V	A; keine solche Schicht
206 11.8	TIME optional; verwendete Zeitwerte nie null	MAY/ MUST	N	N	n.a.	nicht gewählt; S-23 Parser
207 11.9	AUTH ist reserviert	reserviert	N	N	n.a.	nicht definiert
208 11.10	EXP bei Wahl mit Mindestlänge vier	MUST	N	N	n.a.	nicht gewählt
209 12	UCMP und UENC reserviert; UEXP als UNSAFE-Experimentoption	reserviert	N	N	n.a.	nicht definiert

Z. Sec.	Normtext, gekürzt	Stufe	A I T Beleg
210 12	UNSAFE nur in Fragmenten; unbekannte Art verwirft Daten	MUST	J V V R, F; FR-39
211 13	Entwurfsregeln für neue Optionsarten	MUST	N N n.a. keine neue Optionsart
212 14	Empfangsfolge Prüfsumme, OCS, Optionen, Zustellung	Festlegung	J V V R; FR-36
213 14	Vorhandene Pflichtoptionen verarbeiten	MUST	J V V R; FR-06/36
214 14	Andere Optionsarten dürfen ignoriert werden	MAY	J V V R, A; gewählt
215 14	Nutzdaten bei SAFE unabhängig vom Ergebnis liefern	MUST	J V V R; FR-38
216 14	FRAG/NOP/EOL intern; übrige Status bereitstellen	MUST	J P V R, A; FR-37; Parameter nicht immer verfügbar
217 14	Optionsüberlauf nach Abschnitt 10 behandeln	MUST	J V V O, R; FR-50; Erratum 8834
218 15	Empfangs-API: je Paket und Fragment gefordert oder aus	Normform	J P V A; FR-44
219 15	Fehlende Pflichtoption still verwerfen und melden	MUST/ SHOULD	J P V A; FR-44
220 15	Schalter für alle Optionsdatagramme; Standard verarbeiten	Normform/ MUST	J V V A; FR-44
221 15	Optionen und Status für Nutzer verfügbar	Normform/ MUST	J P V R, A; FR-37; Parameter nicht immer verfügbar
222 15	Sende-API: Auswahl, Geltung, Mindestlänge, Maximalfragment	Normform	J P V A; SendConfig/Options
223 15/25	Keine direkte Ordnungs- oder Fragmentgrenzensteuerung	Leitlinie	N N n.a. A; kanonisiert
224 16/19	Transit unverändert (nicht FF1); SAFE-Fehler ignorieren und Nutzdaten liefern	MUST	J V V R, A; FR-38 für Endpunkt; Transit: K, nicht als I bewertet
225 25	Bei DoS-Bedenken Grenzen für Optionen, NOP und Fragmente	bed. SHOULD	J P V R, F; NFR-04/05/06/08
226 25.2	Bei Kanalbedenken Optionen in unabhängiger Referenzordnung	bed. SHOULD	N N n.a. nicht gewählt; kanonische Sendereihenfolge genügt nicht
227 23	Multicast und Broadcast gesondert betrachten	Leitlinie	N N n.a. nur Unicast

Z. Sec.	Normtext, gekürzt	Stufe	A I T Beleg
228 26	Namensregeln für neue SAFE- und UNSAFE-Arten	MUST	N N n.a. keine neue Art
Implementierung: 50 V, 7 P, 10 N. Technisch: 57 V, 0 P, 0 nicht erfüllbar; 10 n.a.			

Literaturverzeichnis

- [1] J. Postel. *User Datagram Protocol*. RFC 768. Internet Engineering Task Force, Aug. 1980. DOI: [10.17487/RFC0768](https://doi.org/10.17487/RFC0768). URL: <https://www.rfc-editor.org/info/rfc768>.
- [2] J. Touch und C. Heard. *Transport Options for UDP*. RFC 9868. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9868](https://doi.org/10.17487/RFC9868). URL: <https://www.rfc-editor.org/info/rfc9868>.
- [3] G. Fairhurst und T. Jones. *Datagram Packetization Layer Path MTU Discovery (DPLPMTUD) for UDP Options*. RFC 9869. Internet Engineering Task Force, Okt. 2025. DOI: [10.17487/RFC9869](https://doi.org/10.17487/RFC9869). URL: <https://www.rfc-editor.org/info/rfc9869>.
- [4] A. Stephan und L. Wüstrich. „The Path of a Packet Through the Linux Kernel“. In: *Seminar Innovative Internet Technologies and Mobile Communications (IITM), WS 23*. Technische Universität München, Chair of Network Architectures und Services, 2024, S. 89–93. DOI: [10.2313/NET-2024-04-1_16](https://doi.org/10.2313/NET-2024-04-1_16). URL: https://www.net.in.tum.de/fileadmin/TUM/NET/NET-2024-04-1/NET-2024-04-1_16.pdf (besucht am 13.08.2026).
- [5] J. Postel. *Internet Protocol*. RFC 791. Internet Engineering Task Force, Sep. 1981. DOI: [10.17487/RFC0791](https://doi.org/10.17487/RFC0791). URL: <https://www.rfc-editor.org/info/rfc791>.
- [6] S. Deering und R. Hinden. *Internet Protocol, Version 6 (IPv6) Specification*. RFC 8200. Internet Engineering Task Force, Juli 2017. DOI: [10.17487/RFC8200](https://doi.org/10.17487/RFC8200). URL: <https://www.rfc-editor.org/info/rfc8200>.
- [7] F. Gont, R. Atkinson und C. Pignataro. *Recommendations on Filtering of IPv4 Packets Containing IPv4 Options*. RFC 7126. Internet Engineering Task Force, Feb. 2014. DOI: [10.17487/RFC7126](https://doi.org/10.17487/RFC7126). URL: <https://www.rfc-editor.org/info/rfc7126>.
- [8] L. Eggert, G. Fairhurst und G. Shepherd. *UDP Usage Guidelines*. RFC 8085. Internet Engineering Task Force, März 2017. DOI: [10.17487/RFC8085](https://doi.org/10.17487/RFC8085). URL: <https://www.rfc-editor.org/info/rfc8085>.
- [9] W. Eddy. *Transmission Control Protocol (TCP)*. RFC 9293. Internet Engineering Task Force, Aug. 2022. DOI: [10.17487/RFC9293](https://doi.org/10.17487/RFC9293). URL: <https://www.rfc-editor.org/info/rfc9293>.
- [10] V. Cerf, Y. Dalal und C. Sunshine. *Specification of Internet Transmission Control Program*. RFC 675. Internet Engineering Task Force, Dez. 1974. DOI: [10.17487/RFC0675](https://doi.org/10.17487/RFC0675). URL: <https://www.rfc-editor.org/info/rfc675>.

- [11] J. F. Kurose und K. W. Ross. *Computer Networking: A Top-Down Approach*. 8. Aufl. Pearson, 2021.
- [12] R. Braden, D. Borman und C. Partridge. *Computing the Internet Checksum*. RFC 1071. Internet Engineering Task Force, Sep. 1988. DOI: [10.17487/RFC1071](https://doi.org/10.17487/RFC1071). URL: <https://www.rfc-editor.org/info/rfc1071>.
- [13] C. Huitema. *Teredo: Tunneling IPv6 over UDP through Network Address Translations (NATs)*. RFC 4380. Internet Engineering Task Force, Feb. 2006. DOI: [10.17487/RFC4380](https://doi.org/10.17487/RFC4380). URL: <https://www.rfc-editor.org/info/rfc4380>.
- [14] D. Thaler. *Teredo Extensions*. RFC 6081. Internet Engineering Task Force, Jan. 2011. DOI: [10.17487/RFC6081](https://doi.org/10.17487/RFC6081). URL: <https://www.rfc-editor.org/info/rfc6081>.
- [15] RFC Editor. *RFC Erratum 8834 for RFC 9868*. Technical; Verified; gemeldet am 17.03.2026. RFC Editor. 23. März 2026. URL: <https://www.rfc-editor.org/errata/eid8834> (besucht am 26.08.2026).
- [16] Linux Kernel Community. *Linux 5.10: net/ipv4/udp.c*. Version 5.10. 13. Dez. 2020. URL: <https://elixir.bootlin.com/linux/v5.10/source/net/ipv4/udp.c> (besucht am 19.08.2026).
- [17] Linux man-pages project. *raw(7): IPv4 raw sockets*. Version 6.18. 8. Feb. 2026. URL: <https://git.kernel.org/pub/scm/docs/man-pages/man-pages.git/plain/man/man7/raw.7?h=man-pages-6.18> (besucht am 17.08.2026).
- [18] J. Jumper, R. Evans, A. Pritzel u. a. „Highly accurate protein structure prediction with AlphaFold“. In: *Nature* 596 (2021), S. 583–589. DOI: [10.1038/s41586-021-03819-2](https://doi.org/10.1038/s41586-021-03819-2).
- [19] A. Davies, P. Veličković, L. Buesing u. a. „Advancing mathematics by guiding human intuition with AI“. In: *Nature* 600 (2021), S. 70–74. DOI: [10.1038/s41586-021-04086-x](https://doi.org/10.1038/s41586-021-04086-x).
- [20] B. Romera-Paredes, M. Barekatin, A. Novikov u. a. „Mathematical discoveries from program search with large language models“. In: *Nature* 625 (2024), S. 468–475. DOI: [10.1038/s41586-023-06924-6](https://doi.org/10.1038/s41586-023-06924-6).
- [21] T. Hubert, R. Mehta, L. Sartran u. a. „Olympiad-level formal mathematical reasoning with reinforcement learning“. In: *Nature* 651.8106 (2026). Online veröffentlicht am 12.11.2025, S. 607–613. DOI: [10.1038/s41586-025-09833-y](https://doi.org/10.1038/s41586-025-09833-y).
- [22] D. E. Knuth. *Claude’s Cycles*. Vom 28.02.2026, überarbeitet am 14.04.2026. 2026. URL: <https://cs.stanford.edu/~knuth/papers/claude-cycles.pdf> (besucht am 13.08.2026).
- [23] Anthropic. *Learning more about Claude’s mathematical capabilities*. Vom 10.08.2026. 2026. URL: <https://www.anthropic.com/research/riemann-zeta> (besucht am 13.08.2026).

- [24] OpenAI. *An OpenAI model has disproved a central conjecture in discrete geometry*. Vom 20.05.2026. 2026. URL: <https://openai.com/index/model-d-isproves-discrete-geometry-conjecture/> (besucht am 13.08.2026).
- [25] M. D. Schwartz. *Resummation of the C-Parameter Sudakov Shoulder Using Effective Field Theory*. arXiv:2601.02484, eingereicht am 05.01.2026. 2026. URL: <https://arxiv.org/abs/2601.02484> (besucht am 13.08.2026).
- [26] *Rewriting Bun in Rust*. Oven (Bun). 2026. URL: <https://bun.com/blog/bun-in-rust> (besucht am 13.08.2026).
- [27] *Rewrite Bun in Rust*. oven-sh/bun, Pull Request 30412. Gemergt am 14.05.2026; 1.009.257 hinzugefügte Zeilen. Metadaten per GitHub-API verifiziert. GitHub. 2026. URL: <https://github.com/oven-sh/bun/pull/30412> (besucht am 13.08.2026).
- [28] *PathString::slice dangling reference UB - add Miri to CI (oven-sh/bun, Issue 30719)*. Gemeldet am 14.05.2026, nach dem Merge des Rust-Ports. GitHub. 2026. URL: <https://github.com/oven-sh/bun/issues/30719> (besucht am 13.08.2026).
- [29] S. Bradner. *Key words for use in RFCs to Indicate Requirement Levels*. RFC 2119. Internet Engineering Task Force, März 1997. DOI: [10.17487/RFC2119](https://doi.org/10.17487/RFC2119). URL: <https://www.rfc-editor.org/info/rfc2119>.
- [30] B. Leiba. *Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*. RFC 8174. Internet Engineering Task Force, Mai 2017. DOI: [10.17487/RFC8174](https://doi.org/10.17487/RFC8174). URL: <https://www.rfc-editor.org/info/rfc8174>.
- [31] RFC Editor. *RFC Erratum 8708 for RFC 9868*. Editorial; Held for Document Update; gemeldet am 17.01.2026. RFC Editor. 20. Jan. 2026. URL: <https://www.rfc-editor.org/errata/eid8708> (besucht am 26.08.2026).
- [32] A. T. Kalai, O. Nachum, S. S. Vempala und E. Zhang. *Why Language Models Hallucinate*. arXiv:2509.04664. 2025. URL: <https://arxiv.org/abs/2509.04664> (besucht am 13.08.2026).
- [33] E. Perez u. a. *Discovering Language Model Behaviors with Model-Written Evaluations*. arXiv:2212.09251. 2022. URL: <https://arxiv.org/abs/2212.09251> (besucht am 13.08.2026).
- [34] M. Cheng, C. Lee, P. Khadpe u. a. „Sycophantic AI decreases prosocial intentions and promotes dependence“. In: *Science* 391.6792 (2026). Artikelkennung eaec8352. DOI: [10.1126/science.aec8352](https://doi.org/10.1126/science.aec8352).
- [35] proptest-rs-Projekt. *Proptest: Hypothesis-like property testing for Rust*. GitHub. 2026. URL: <https://github.com/proptest-rs/proptest> (besucht am 23.08.2026).
- [36] rust-fuzz-Projekt. *cargo-fuzz: Command line helpers for fuzzing*. GitHub. 2026. URL: <https://github.com/rust-fuzz/cargo-fuzz> (besucht am 23.08.2026).

- [37] LLVM Project. *libFuzzer: A Library for Coverage-Guided Fuzz Testing*. Undatierte, fortlaufend gepflegte Dokumentation. LLVM Project. URL: <https://llvm.org/docs/LibFuzzer.html> (besucht am 20.08.2026).
- [38] L. de Moura und S. Ullrich. „The Lean 4 Theorem Prover and Programming Language“. In: *Automated Deduction (CADE 28)*. Bd. 12699. Lecture Notes in Computer Science. Springer, 2021, S. 625–635. DOI: [10.1007/978-3-030-79876-5_37](https://doi.org/10.1007/978-3-030-79876-5_37).
- [39] M. Miller. *Trends, Challenges, and Strategic Shifts in the Software Vulnerability Mitigation Landscape*. Vortrag, BlueHat IL 2019, Tel Aviv. Microsoft Security Response Center, BlueHat IL 2019. 7. Feb. 2019. URL: <https://www.youtube.com/watch?v=PjbGojJnBZQ> (besucht am 31.05.2026).
- [40] The Chromium Project. *Memory safety*. Undatierte, fortlaufend gepflegte Projektseite. The Chromium Project. URL: <https://www.chromium.org/Home/chromium-security/memory-safety/> (besucht am 31.05.2026).
- [41] National Security Agency. *Software Memory Safety*. Cybersecurity Information Sheet U/OO/219936-22. National Security Agency (NSA), 10. Nov. 2022. URL: https://media.defense.gov/2022/Nov/10/2003112742/-1/-1/0/CSI_SOFTWARE_MEMORY_SAFETY.PDF (besucht am 31.05.2026).
- [42] The Rust Project. *Rust Programming Language*. Undatierte, fortlaufend gepflegte Projektseite. URL: <https://www.rust-lang.org/> (besucht am 16.02.2026).
- [43] S. Klabnik, C. Nichols und C. Kryocho. *The Rust Programming Language*. Undatierte, fortlaufend gepflegte Online-Fassung. URL: <https://doc.rust-lang.org/book/> (besucht am 16.02.2026).
- [44] ISO/IEC JTC 1/SC 22/WG14. *Programming languages: C*. N2310. Frei verfügbarer Arbeitsentwurf zu ISO/IEC 9899:2018 (C17). ISO/IEC JTC 1/SC 22/WG14. 2018. URL: <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2310.pdf> (besucht am 31.05.2026).
- [45] The Linux kernel development community. *Development tools for the Linux kernel*. Version 5.10. The Linux Kernel Archives. 13. Dez. 2020. URL: <https://www.kernel.org/doc/html/v5.10/dev-tools/index.html> (besucht am 26.08.2026).
- [46] Zig Software Foundation. *Zig Language Reference*. Version 0.16.0. Zig Software Foundation. 2026. URL: <https://ziglang.org/documentation/0.16.0/> (besucht am 24.08.2026).
- [47] Linux man-pages project. *packet(7): packet interface on device level*. Version 6.18. 8. Feb. 2026. URL: <https://git.kernel.org/pub/scm/docs/man-pages/man-pages.git/plain/man/man7/packet.7?h=man-pages-6.18> (besucht am 26.08.2026).

- [48] Linux Kernel Community. *Linux 5.10: Universal TUN/TAP device driver*. Version 5.10. 13. Dez. 2020. URL: <https://www.kernel.org/doc/html/v5.10/networking/tuntap.html> (besucht am 26.08.2026).
- [49] Linux Kernel Community. *Linux 5.10: AF_XDP*. Version 5.10. 13. Dez. 2020. URL: https://www.kernel.org/doc/html/v5.10/networking/af_xdp.html (besucht am 26.08.2026).
- [50] Linux Kernel Community. *Linux 5.10: Building External Modules*. Version 5.10. 13. Dez. 2020. URL: <https://www.kernel.org/doc/html/v5.10/kbuild/modules.html> (besucht am 26.08.2026).
- [51] A. Crichton und T. de Zeeuw. *socket2: Advanced configuration options for sockets*. Version 0.6.4. 28. Mai 2026. URL: <https://docs.rs/socket2/0.6.4/> (besucht am 24.08.2026).
- [52] The Rust Project Developers. *libc: Raw bindings to platform APIs for Rust*. Version 0.2.186. 23. Apr. 2026. URL: <https://docs.rs/libc/0.2.186/> (besucht am 24.08.2026).
- [53] Linux man-pages project. *capabilities(7): overview of Linux capabilities*. Version 6.18. 8. Feb. 2026. URL: <https://git.kernel.org/pub/scm/docs/man-pages/man-pages.git/plain/man/man7/capabilities.7?h=man-pages-6.18> (besucht am 23.08.2026).

Eidesstattliche Erklärung

Ich versichere, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Alle Stellen der Arbeit, die wörtlich oder sinngemäß aus Veröffentlichungen oder aus anderweitigen fremden Quellen entnommen wurden, sind als solche kenntlich gemacht.

Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt.

.....

Ort, Datum

.....

Unterschrift